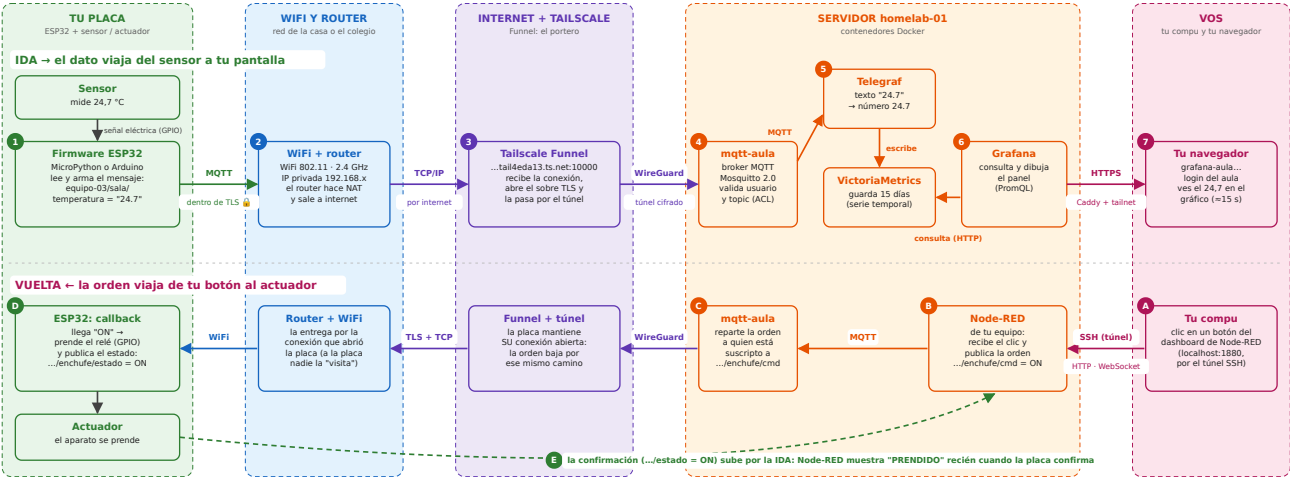


Manual de Arquitectura del Homelab

Arquitectura de software aplicada al diseño IoT,
desde cero, con el servidor del aula como caso real

El ciclo completo: ida (el dato) y vuelta (la orden)

Cada flecha dice QUÉ protocolo viaja por ese tramo. Los números y letras marcan el orden del recorrido.



Lucas D. Gómez — arquitecto de software
Versión del 03/10/2026 · generado desde el repositorio

Índice

Inicio

Parte I — Bases

1. Visión general
2. Fundamentos (compu, Linux, redes)
3. Docker y contenedores
4. Ansible e Infraestructura como Código

Parte II — La plataforma

5. Los servicios uno por uno
 6. Observabilidad
 7. Seguridad (DevSecOps en el host)
 8. El pipeline (Git, CI, tests)
 9. La plataforma de aula
- Guía práctica del equipo

Parte III — IoT aplicado

11. Arquitectura IoT del aula
- Red y accesos
12. Conectar tus sensores a Grafana
- ¿Node-RED, Grafana o ambos?
13. Usar Grafana
 14. Los proyectos de los equipos

Parte IV — Operar y evolucionar

Diagnóstico (¿dónde se cortó?)

10. Casos prácticos
15. Ciclo de vida y madurez

Glosario

Créditos y autoría

Mejoras futuras

Manual de Arquitectura del Homelab

Un libro de estudio de **arquitectura de software aplicada al diseño IoT**: cómo se piensa, se construye y se cuida un sistema que conecta placas, mensajes, datos y pantallas. Usa como caso real el servidor del aula (este repositorio) y los proyectos de los equipos. Está escrito **para principiantes**: no asume que sabés nada, y cada sigla se explica la primera vez que aparece.

Autor: Lucas D. Gómez, arquitecto de software · [Créditos y autoría](#)

 **Versión para imprimir o leer sin conexión:** el manual completo en PDF, ordenado desde Fundamentos y con todas las imágenes: [manual-arquitectura-homelab.pdf](#).

¿Qué necesitás hoy?

No hace falta leer todo en orden. Elegí tu camino:



Tengo que hacer funcionar mi proyecto para la muestra

1. [Guía del equipo: conectarte](#) (Tailscale + túnel)
2. [Conectar tu placa: las 3 reglas del topic](#)
3. [Comprobar que llega, con MQTT Explorer](#)
4. [Abrir tu dashboard de Node-RED](#)
5. [Chequeo rápido antes de la muestra](#)



Quiero entender la arquitectura completa

1. [Visión general](#) → los planos del sistema
2. [Arquitectura IoT](#) → las 6 capas y las 9 decisiones
3. [Seguí un dato: 24,7 °C](#) → el recorrido real
4. [Red y accesos](#) → la parte invisible
5. [Ciclo de vida y madurez](#) → cómo nace y crece



Necesito visualizar mis sensores

1. [Las 3 reglas del topic](#) → MQTT
2. [Seguí un dato](#) → cómo se guarda
3. [Usar Grafana](#) → tu dashboard
4. [Recetario de consultas](#)

Necesito procesar datos o automatizar algo

1. [¿Node-RED, Grafana o ambos?](#) → decidir
2. [Caso B: reglas y órdenes](#)
3. [Caso C: calcular y graficar](#)
4. [Los proyectos de los equipos](#) → cómo lo hicieron otros

No puedo acceder al servidor

1. [Diagnóstico: no puedo entrar](#) → paso a paso
2. [Red y accesos](#) → Tailscale, túneles y las 4 capas
3. [Tus credenciales, en una tabla](#)

Algo dejó de funcionar

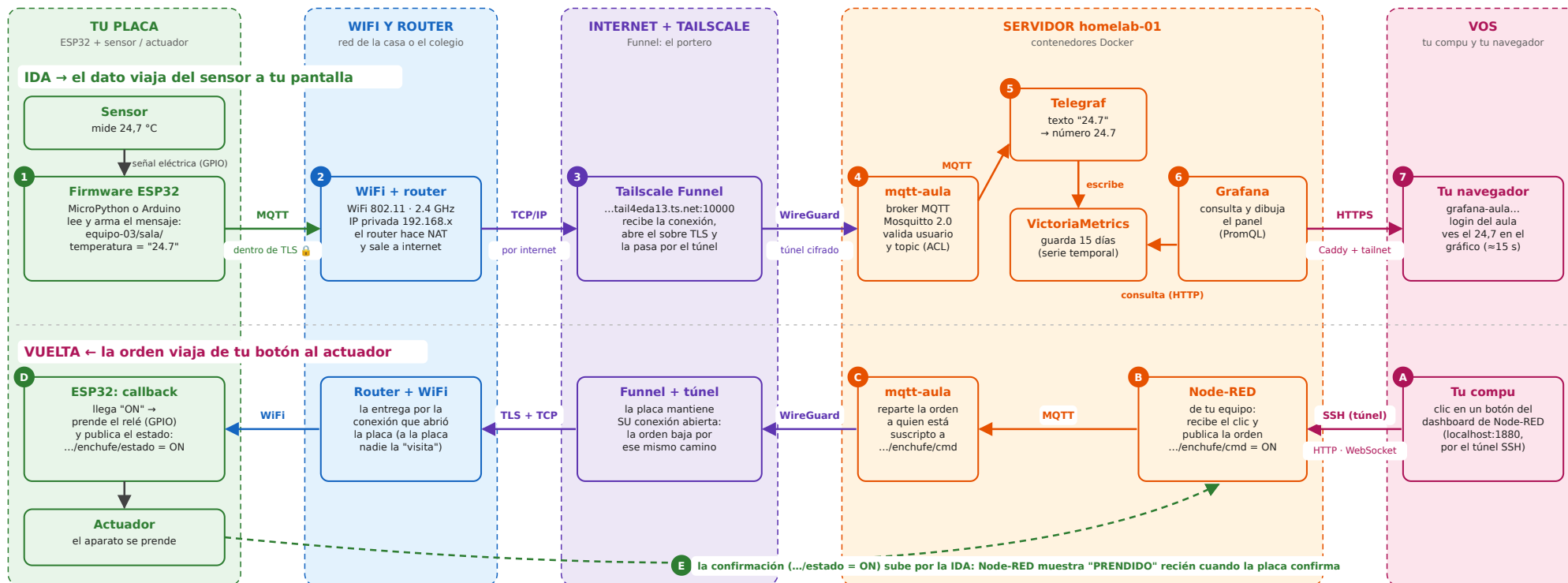
1. [Diagnóstico](#) → dónde se cortó el recorrido
2. [Casos prácticos](#) → errores reales y cómo se resolvieron
3. [¿Qué pasa si se cae cada pieza?](#)

Lo que vas a poder explicar al terminar

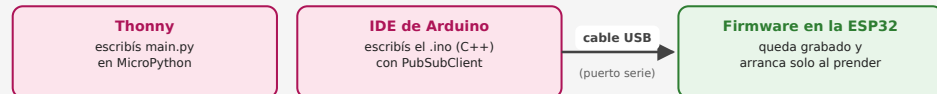
"Mi **ESP32** toma el dato del **sensor**, lo **publica por MQTT** en el topic de mi equipo; viaja por **WiFi e internet** y entra al servidor por **Funnel**; el **broker** lo reparte; **Telegraf** lo guarda en **VictoriaMetrics**, y **Grafana** lo consulta para dibujar mi dashboard. **Node-RED**, al costado, recibe los eventos, aplica mis reglas y manda órdenes a mis actuadores. Yo llego a todo eso por **Tailscale**, con un **túnel** para lo que no tiene login."

El ciclo completo: ida (el dato) y vuelta (la orden)

Cada flecha dice QUÉ protocolo viaja por ese tramo. Los números y letras marcan el orden del recorrido.



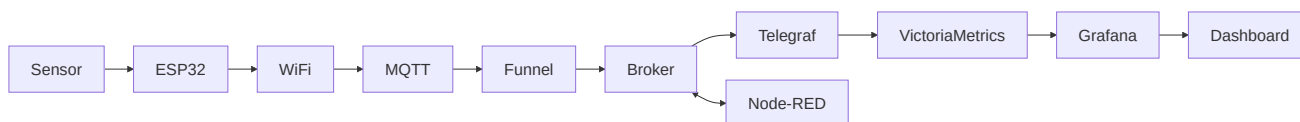
ANTES DE TODO: cómo llega tu código a la placa (y con qué trabajás vos)



Otras herramientas tuyas, y para qué:

- Tailscale: entrar a la red privada del aula • Terminal + SSH: entrar al servidor y abrir el túnel
- MQTT Explorer: escuchar tus mensajes en el broker • Navegador: Node-RED, tu página y Grafana
- Consola de Thonny / Monitor Serie: ver qué hace la placa (WiFi OK, MQTT OK, errores)

En versión corta:



Y también el ciclo del software: **código** → **repositorio** → **despliegue** → **ejecución** → **observabilidad** → **mantenimiento** ([capítulo 15](#)).

¿Para quién es este manual?

Está pensado para tres tipos de lectores:

1. **Alguien que nunca vio Docker ni un servidor.** Empezá por el principio y leé en orden. Cada concepto se explica desde cero.
2. **Alguien que conoce Linux pero no entiende cómo encaja todo.** Podés saltar los fundamentos e ir a los capítulos de arquitectura y servicios.
3. **Alguien con experiencia que quiere entender el diseño rápido.** Leé la [Visión general](#) y los "resúmenes" y diagramas de cada capítulo.

Cómo leerlo

Cada capítulo tiene siempre la misma estructura, para que sepas qué esperar:

- 🎯 **Objetivo** — qué vas a poder hacer/entender al terminar.
- 🧩 **Prerequisitos** — qué conviene haber leído antes.
- 🆕 **Conceptos nuevos** — las palabras nuevas que aparecen.
- 📖 **Desarrollo** — la explicación, con dibujos.
- 🧠 **Ideas clave** — lo que no te tenés que olvidar.
- ⚠️ **Errores comunes** — en qué se tropieza todo el mundo.
- ❓ **Preguntas de repaso** — para chequear que entendiste.
- 🛠️ **Ejercicios** — para practicar.

Convención: cuando veas una palabra en **negrita** por primera vez, casi siempre está también en el [Glosario](#) con una definición simple y una técnica.

Filosofía del manual

Aprendemos desde lo cercano. Cada tema arranca por algo que ya conocés (un grupo de WhatsApp, el tablero de un auto, una bici) o por un proyecto de un compañero, y recién después aparece la palabra técnica. Ninguna sigla se da por sabida: la primera vez se explica, y siempre está en el [Glosario](#).

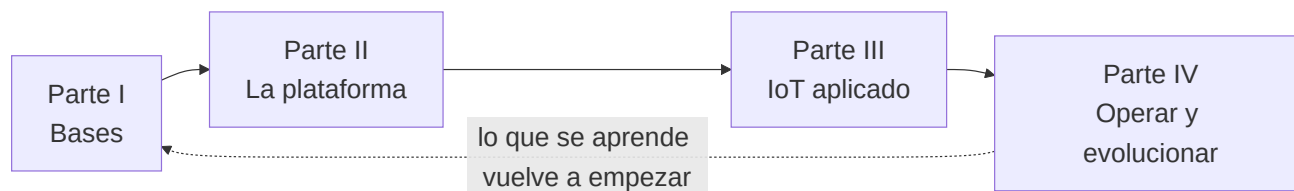
No describimos archivos: explicamos **por qué existen**. Para cada decisión de diseño respondemos:

- ¿Qué problema resuelve?
- ¿Cómo interactúa con lo demás?
- ¿Qué pasaría si no existiera?
- ¿Qué alternativas había y por qué se eligió ésta?

Cuando algo es una decisión mejorable, lo marcamos como 💡 **Posible mejora arquitectónica**.

Índice del libro

El libro se lee en cuatro partes. Si sos alumno y querés **conectar tu placa ya**, podés ir directo a la Parte III y volver a las otras cuando las necesites.



Parte I — Bases

#	Capítulo	De qué trata
1	Visión general	Qué es el laboratorio y sus "planos" (host, servicios, aula, IoT)
2	Fundamentos	Computación, Linux y redes desde cero
3	Docker y contenedores	Imágenes, contenedores, Compose, redes, volúmenes
4	Ansible e IaC	Infraestructura como código

Parte II — La plataforma

#	Capítulo	De qué trata
5	Los servicios uno por uno	Caddy, Grafana, Tailscale, Postgres, el broker...
6	Observabilidad	Medir, guardar y mostrar
7	Seguridad	Defensa en capas
8	El pipeline	Git, revisión, pruebas automáticas
9	La plataforma de aula	labctl, aislamiento, servicios compartidos
—	Guía práctica del equipo	Tu día a día: conectarte, tus puertos, MQTT, tu página web

Parte III — IoT aplicado

#	Capítulo	De qué trata
11	Arquitectura IoT del aula	La columna del libro: las 6 capas, el recorrido de un dato y las 9 decisiones
—	Red y accesos	Tailscale, Funnel, túneles y por qué hay tantas contraseñas
12	Conectar tus sensores a Grafana	Las 3 reglas del topic, ejemplos en MicroPython y Arduino
—	¿Node-RED, Grafana o ambos?	Quién procesa, quién muestra, y cómo se combinan
13	Usar Grafana	Leer y editar el tablero de tu equipo
14	Los proyectos de los equipos	El LED, la puerta y el enchufe como casos de diseño

Parte IV — Operar y evolucionar

#	Capítulo	De qué trata
—	Diagnóstico	Dónde se cortó el recorrido: árbol paso a paso, también para la muestra
10	Casos prácticos	Nueve problemas reales y cómo se resolvieron
15	Ciclo de vida y madurez	Cómo nace y crece el sistema, sus stacks y qué tan maduro está

Para consultar

—	Glosario	Todas las palabras y siglas, de la A a la Z
—	Créditos y autoría	Quién hizo qué
—	Mejoras futuras	Lo que falta, anotado

Cómo verlo como libro

Este manual está preparado para **[MkDocs Material](#)**:

```
pip install mkdocs-material
mkdocs serve      # abrí http://127.0.0.1:8000
```

También se lee directo en GitHub (los diagramas Mermaid se renderizan solos) y se puede exportar a PDF.

Parte I — Bases

1. Visión general

 **Objetivo:** entender **qué es** este laboratorio, de qué partes está hecho y cómo se relacionan, sin entrar todavía en los detalles.

 **Prerequisitos:** ninguno.

 **Conceptos nuevos:** servidor, plano (capa), host, servicio, contenedor, proxy inverso, infraestructura como código.

¿Qué es este laboratorio?

Es **una sola computadora** (un servidor) que hace tres trabajos a la vez:

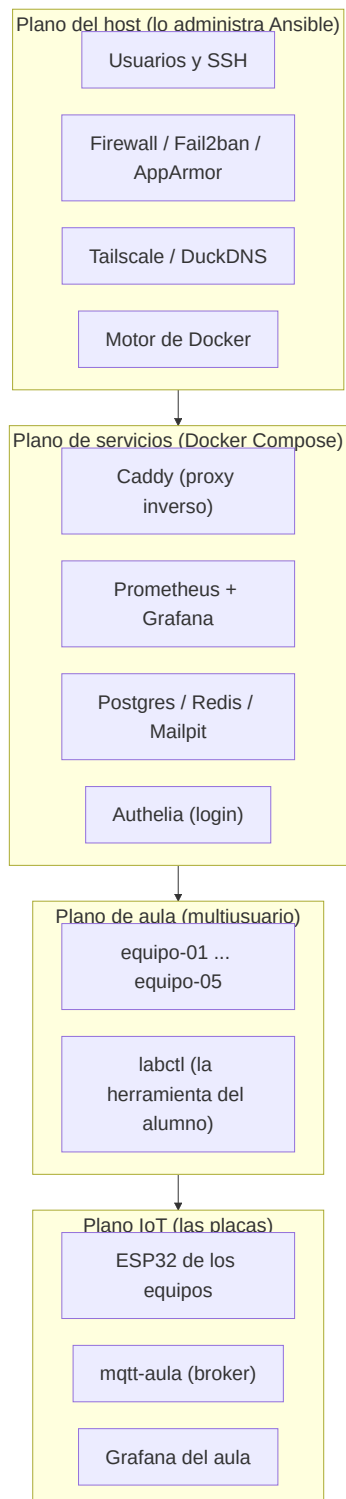
1. **Servidor personal** del dueño (dashboards, monitoreo, proyectos propios).
2. **Laboratorio para alumnos:** varios equipos despliegan sus propias aplicaciones con Docker, aislados entre sí.
3. **Workstation ocasional** (a veces se usa como escritorio con Firefox, etc.).

¿Qué es un servidor? Una computadora que está siempre prendida y ofrece "servicios" a otras computadoras por la red (mostrar una web, guardar datos, etc.). No tiene nada mágico: es una PC común con un rol.

El hardware es modesto a propósito (un Intel i5 de 4 núcleos, 15 GiB de RAM, un disco SSD). Todo el diseño está pensado para **rendir en hardware limitado**.

Los tres "planos"

Para no hacer un choclo, el sistema se organiza en **planos** (podés pensar "capas" o "áreas de responsabilidad"). Cada uno resuelve un problema distinto:



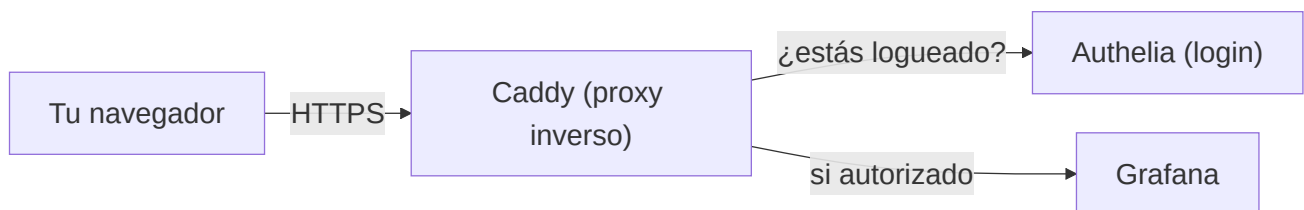
- **Plano del host:** el sistema operativo y su seguridad. Cambia poco y se quiere "declarativo" (definido en archivos). Lo maneja **Ansible**.
- **Plano de servicios:** las aplicaciones que corren en **contenedores** (Caddy, Grafana...). Cambian seguido, y por eso usan **Docker Compose**, que permite prender/apagar rápido.
- **Plano de aula:** los proyectos de los alumnos, cada equipo aislado, manejados con una herramienta propia (`labctl`).
- **Plano IoT:** las **placas** de los equipos (ESP32) que miden o hacen cosas en el mundo físico, el **broker** que reparte sus mensajes y el **Grafana del aula** donde se ven. Es el

corazón del trabajo de los equipos y tiene su propio capítulo: [11 — Arquitectura IoT del aula](#).

Idea central: separar lo que cambia lento (el host) de lo que cambia rápido (los servicios). Si mezclas todo, cada cambio chico te obliga a tocar el sistema entero.

¿Cómo entra el tráfico? (el camino de una petición)

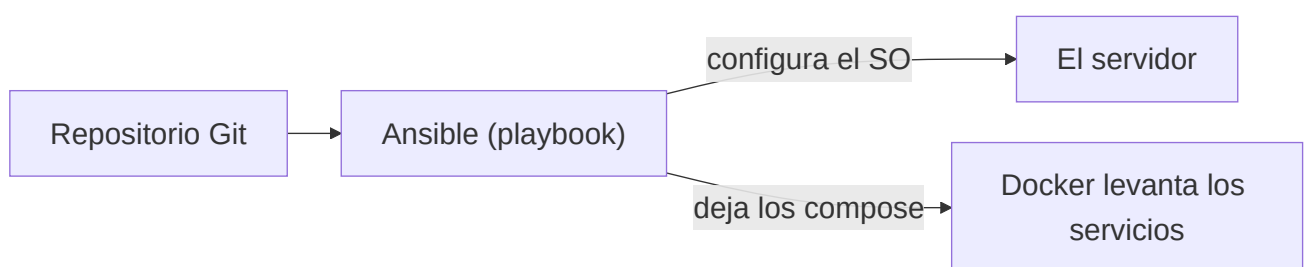
Cuando abris `https://grafana.tudominio` desde el navegador, pasa esto:



Caddy es la **única puerta de entrada**. Recibe todo el tráfico web, se encarga del **HTTPS** (el candado del navegador), pregunta a **Authelia** si el usuario tiene permiso, y recién ahí manda la petición al servicio correcto (Grafana, Prometheus, etc.). A esto se le llama **proxy inverso** (lo vemos en detalle en el [capítulo 5](#)).

¿Cómo se administra todo? (Infraestructura como Código)

No se configura el server "a mano". Todo está escrito en archivos de texto (**Ansible**) que se pueden versionar en Git y volver a aplicar. A esto se le llama **Infraestructura como Código (IaC)**.



La ventaja: si el server se rompe, se puede **reconstruir** corriendo los mismos archivos. Y como es **idempotente** (aplicarlo dos veces da el mismo resultado), es seguro re-ejecutarlo.

¿Cómo se accede de forma segura?

- **SSH** para entrar a la terminal del server (administración).
- **Tailscale** crea una red privada propia (una "VPN" simple) para llegar al server desde cualquier lado sin exponerlo a Internet.

- **DuckDNS** le da un nombre de dominio gratis (para que no tengas que recordar números de IP).

Todo esto se explica en los capítulos [7 \(Seguridad\)](#) y [5 \(Servicios\)](#).

Ideas clave

- El laboratorio es **una sola computadora** con tres roles.
- Como todo sistema, **evoluciona**: en qué etapa está cada parte, en el [capítulo 15](#).
- Se organiza en **planos**: host, servicios, aula e IoT.
- **Caddy** es la única puerta web; **Ansible** administra todo como código.
- Se separa lo que cambia lento de lo que cambia rápido.

Errores comunes

- Pensar que "servidor" es algo exótico: es una PC con un rol.
- Confundir el **plano del host** (SO, con Ansible) con el **de servicios** (contenedores, con Compose). Son mundos distintos.
- Creer que Caddy "es" la web: Caddy solo **enruta** hacia los servicios.

Preguntas de repaso

1. ¿Cuáles son los planos y qué problema resuelve cada uno?
2. ¿Por qué se separa el host de los servicios?
3. ¿Qué hace Caddy cuando llega una petición web?
4. ¿Qué quiere decir que Ansible sea "idempotente"?

Ejercicios

1. Dibujá, con tus palabras, el camino de una petición desde el navegador hasta Grafana.
2. Listá tres cosas que estén en el "plano del host" y tres en el "de servicios".

2. Fundamentos (computación, Linux y redes)

 **Objetivo:** aprender las palabras básicas que se usan en todo el resto del libro. Es un capítulo de "diccionario explicado". Podés volver a él cuando algo no te cierre.

 **Prerequisitos:** ninguno.

 **Conceptos nuevos:** CPU, RAM, disco, sistema operativo, proceso, servicio, daemon, kernel, systemd, boot, usuario, grupo, permisos, filesystem, inode, IP, puerto, TCP, DNS, HTTPS, protocolo, pila de protocolos, VPN, Tailscale, túnel.

Empecemos por lo cercano: una computadora es una cocina

Antes de cualquier palabra técnica, pensá en una cocina de restaurante:

En la cocina	En la computadora	Qué es
el cocinero	la CPU (<i>Central Processing Unit</i> , unidad central de procesamiento)	el que hace el trabajo: hace cuentas, sigue instrucciones. Si tiene "4 núcleos", son 4 cocineros
la mesada	la RAM (<i>Random Access Memory</i> , memoria de acceso aleatorio)	donde se pone lo que se está usando ahora. Es rápida pero chica, y se vacía al apagar
la heladera y la alacena	el disco (en el servidor, un SSD)	donde se guarda todo. Es grande y no se borra al apagar, pero es más lento de usar
el jefe de cocina	el sistema operativo (acá, Ubuntu Linux)	reparte cocineros, mesada y alacena entre todos los pedidos
cada pedido en curso	un proceso	un programa que se está ejecutando
el cocinero de guardia que siempre está	un servicio	un programa que corre siempre, esperando pedidos

Cuando más adelante leas "el servidor tiene 15 GiB de RAM", pensá "la mesada es de este tamaño": si todos los pedidos juntos no entran, algo se cae (eso se llama **OOM**, *Out Of Memory*, "sin memoria").

Parte A — Computación básica

¿Qué es un proceso?

Un **programa en ejecución**. Cuando abrís Firefox, se crea un *proceso* Firefox. El sistema operativo le da memoria (RAM) y turnos de CPU. Cada proceso tiene un número (**PID**, Process ID). Un programa guardado en disco es solo un archivo; cuando lo corrés, se vuelve un proceso.

¿Qué es un servicio?

Un proceso que corre **en segundo plano y de forma permanente** para ofrecer algo (una web, una base de datos). No tiene ventana; lo usás por la red o por comandos. En este proyecto, Grafana, Prometheus o la base de datos son servicios.

¿Qué es un daemon?

Es **otra palabra para "servicio de fondo"** en el mundo **Unix** (la familia de sistemas operativos de la que vienen Linux y macOS). Los daemons suelen terminar en `d`: `sshd` (el de SSH), `dockerd` (el de Docker), `tailscaled` (el de Tailscale). Cuando veas un nombre con `d` al final, pensá "servicio de fondo".

¿Qué es el kernel?

El **núcleo** del sistema operativo: el programa que habla directamente con el hardware (CPU, memoria, disco, red) y reparte esos recursos entre los procesos. En Linux, el kernel se llama, justamente, **Linux**. Todo lo demás (comandos, servicios) corre *encima* del kernel.

¿Qué es el boot?

El **arranque**: la secuencia que ocurre cuando prendés la máquina, desde que el hardware se inicializa hasta que el sistema operativo está listo y los servicios levantados. "Al boot" = "cuando arranca la máquina".

¿Qué es systemd?

El **administrador de servicios** de las distribuciones Linux modernas (como Ubuntu). Es el primer proceso que arranca (PID 1) y se encarga de **prender los servicios en orden** durante el boot, reiniciarlos si se caen, y manejarlos. Cuando escribís `systemctl start docker` o `systemctl status ssh`, le estás hablando a systemd.

En este proyecto, muchas cosas son **unidades de systemd**: los timers de backup, el daemon `labctld`, los "slices" de recursos por equipo (cap. 9).

Parte B — Linux

Linux organiza **todo como archivos** y controla el acceso con **usuarios, grupos y permisos**.

Usuarios y grupos

Un **usuario** es una cuenta (una persona o un servicio): `homelab`, `ansible`, `jessi`. Un **grupo** junta usuarios que comparten permisos: `grp-equipo-01`, `docker`, `sudo`. Un usuario puede estar en varios grupos.

- Cada usuario tiene un número (**UID**) y cada grupo un **GID**.
- `root` es el superusuario (UID 0): puede hacer todo. Por eso se lo evita.

Permisos, `chmod` y `chown`

Cada archivo/carpeta tiene permisos para tres categorías: **dueño (owner)**, **grupo** y **otros**. Cada categoría puede tener: **leer (r)**, **escribir (w)** y **ejecutar (x)**.

Se escriben como un número de tres dígitos (uno por categoría), donde `r=4`, `w=2`, `x=1` se suman:

- `750` = dueño `rw` (7), grupo `r-x` (5), otros nada (0).
- `2770` = como `770` pero con un bit especial adelante (ver **setgid**).

Comandos:

- `chmod` cambia los permisos: `chmod 750 archivo`.
- `chown` cambia el dueño/grupo: `chown root:grp-equipo-01 carpeta`.

setgid (el "2" adelante)

El bit **setgid** en una carpeta hace que **los archivos nuevos hereden el grupo de la carpeta** (en vez del grupo del usuario que los crea). En este proyecto, las carpetas de cada equipo son `2770` con setgid: así todo lo que crea un alumno queda con el grupo del equipo, y **nadie de otro equipo puede entrar**.

`sudo`

Permite a un usuario común ejecutar un comando **como root**, puntualmente. En este proyecto:

- La cuenta `ansible` tiene `sudo` **sin contraseña** (para que la automatización funcione sola).
- La cuenta `homelab` (vos) tiene `sudo` **con contraseña** (más seguro para una cuenta interactiva).
- Los alumnos **no tienen sudo** (por seguridad).

PATH y variables de entorno

Una **variable de entorno** es un valor con nombre que el sistema le pasa a los procesos (por ejemplo `TZ=America/Argentina/Buenos_Aires`). El **PATH** es una variable especial: la lista de carpetas donde el sistema busca los comandos. Por eso a veces `ansible-playbook` "no se encuentra": no está en el `PATH`, y hay que llamarlo por su ruta completa (`~/homelab/.venv/bin/ansible-playbook`).

Filesystem, jerarquía e inode

El **filesystem** (sistema de archivos) es cómo se organizan los datos en el disco: una jerarquía de carpetas que arranca en `/` (la raíz). Algunas ramas importantes:

- `/etc` → configuración del sistema.
- `/home` → carpetas de los usuarios.
- `/var` → datos variables (logs, etc.).
- `/opt` → software opcional (acá viven los stacks de Docker: `/opt/homelab/stacks`).
- `/srv` → datos servidos (acá viven los proyectos de los alumnos: `/srv/classroom`).

Un **inode** es la ficha interna que el filesystem usa para cada archivo (guarda permisos, tamaño, y **dónde** están los datos), identificada por un número. El *nombre* del archivo apunta a un inode. Esto importa por un detalle real de este proyecto: cuando Ansible "reemplaza" un archivo, en realidad crea uno **nuevo** (nuevo inode) y le pone el nombre viejo. Si un contenedor tenía montado el archivo por su inode, sigue viendo el viejo hasta que se reinicia (nos pasó con el Caddyfile — ver [cap. 10](#)).

Logs y `journalctl`

Un **log** es un registro de lo que fue pasando (mensajes de un servicio). En systemd, los logs se consultan con `journalctl` (ej: `journalctl -u ssh` muestra los del servicio SSH). Docker tiene sus propios logs: `docker logs <contenedor>`.

Parte C — Redes

¿Qué es una IP?

Una **dirección IP** (*Internet Protocol*, el protocolo de internet) es el "número de teléfono" de una computadora en una red (ej: `192.168.8.144`). Sirve para que los datos sepan a quién llegar.

Una compu puede tener **varias** IP, una por cada **interfaz** de red (cada "enchufe" por donde se conecta: el WiFi, el cable, la red privada de Tailscale). El servidor del aula, por ejemplo, tiene una IP de la red de la casa (`192.168.8.x`) y otra de Tailscale (`100.110.123.76`).

¿Qué es una IP privada?

Los rangos como `192.168.x.x`, `10.x.x.x` o `172.16-31.x.x` son **privados** (Tailscale usa además los `100.64` a `100.127`): solo existen dentro de tu red local (casa, aula). No son alcanzables desde Internet. Tu router tiene además una **IP pública** (la que te ve el mundo).

¿Qué es NAT?

Network Address Translation. Es el truco del router para que muchos dispositivos con IP privada compartan una sola IP pública. Cuando salís a Internet, el router traduce tu IP privada por la pública (y al revés). Por eso, para que alguien de afuera llegue a tu server, hay que configurar un **port-forward** (redirigir un puerto de la IP pública hacia la IP privada del server).

¿Qué es un puerto?

Si la IP es el teléfono, el **puerto** es el interno. Una misma computadora ofrece muchos servicios, cada uno en un puerto: **80** (web HTTP), **443** (web HTTPS), **22** (SSH), **5432** (Postgres), **9090** (Prometheus)... Publicar algo "en `127.0.0.1:8080`" significa: en el puerto 8080, pero **solo accesible desde la propia máquina** (`127.0.0.1` = "yo mismo", loopback).

¿Qué es TCP?

Transmission Control Protocol. La forma más común de enviar datos por la red de manera **confiable** (verifica que todo llegue y en orden). La web, SSH y las bases de datos usan TCP. (Hay otro, UDP, más rápido pero sin garantías.)

¿Qué es un protocolo? La pila de protocolos

Un **protocolo** es un **acuerdo** de cómo hablar: en qué orden, con qué formato y qué se responde. Igual que al mandar una carta hay reglas (sobre, estampilla, dirección), las computadoras siguen reglas para mandarse datos.

No hay un solo protocolo que haga todo: hay **varios, apilados**, y cada uno resuelve **un** problema. A eso se le dice **pila de protocolos**:

La pila de protocolos: cada capa con un solo trabajo

Como una carta: escribís el mensaje, lo metés en un sobre cerrado, le ponés la dirección y el correo lo lleva.

APLICACIÓN <i>qué dice el mensaje</i>	MQTT (placa ↔ broker) · HTTP/HTTPS (navegador) · SSH (terminal) En la carta: la carta
SEGURIDAD <i>que nadie lo lea en el camino</i>	TLS (sobre cerrado) · WireGuard (túnel de Tailscale) En la carta: el sobre cerrado
TRANSPORTE <i>que llegue completo y en orden</i>	TCP (MQTT, HTTPS, SSH) · UDP (WireGuard) En la carta: el remito con número de piezas
RED <i>a qué máquina va</i>	IP: 192.168.x (casa) · 100.110.123.76 (Tailscale) · IP pública En la carta: la dirección del sobre
ENLACE / FÍSICO <i>cómo viaja por el aire o el cable</i>	WiFi 802.11 a 2.4 GHz (placa, servidor) · Ethernet (cable) En la carta: el camión del correo

Cuando tu ESP32 publica "24.7", baja por todas estas capas en la placa y sube por todas en el servidor.

Leída de arriba hacia abajo, cuando tu ESP32 publica una temperatura:

1. **Aplicación:** MQTT arma el mensaje (equipo-03/sala/temperatura = 24.7).
2. **Seguridad:** TLS lo mete en un **sobre cerrado** para que nadie lo lea.
3. **Transporte:** TCP lo parte en pedazos numerados y se asegura de que lleguen todos y en orden.
4. **Red:** IP le pone la **dirección** de destino.
5. **Enlace/físico:** el WiFi lo manda por el aire, en la banda de 2.4 GHz.

En el servidor el camino es al revés: sube capa por capa hasta que el broker lee el mensaje. **Cada capa no sabe nada de las otras:** por eso se puede cambiar el WiFi por un cable sin tocar tu código MQTT. El recorrido completo, con todos los protocolos de cada tramo, está en el [capítulo 11](#).

¿Qué es una VPN? ¿Y Tailscale?

Una **VPN** (*Virtual Private Network*, red privada virtual) es una **red privada que viaja por adentro de internet**, cifrada. Es como si varias computadoras que están en casas distintas estuvieran enchufadas al **mismo router**: se ven entre ellas, y nadie de afuera las ve.

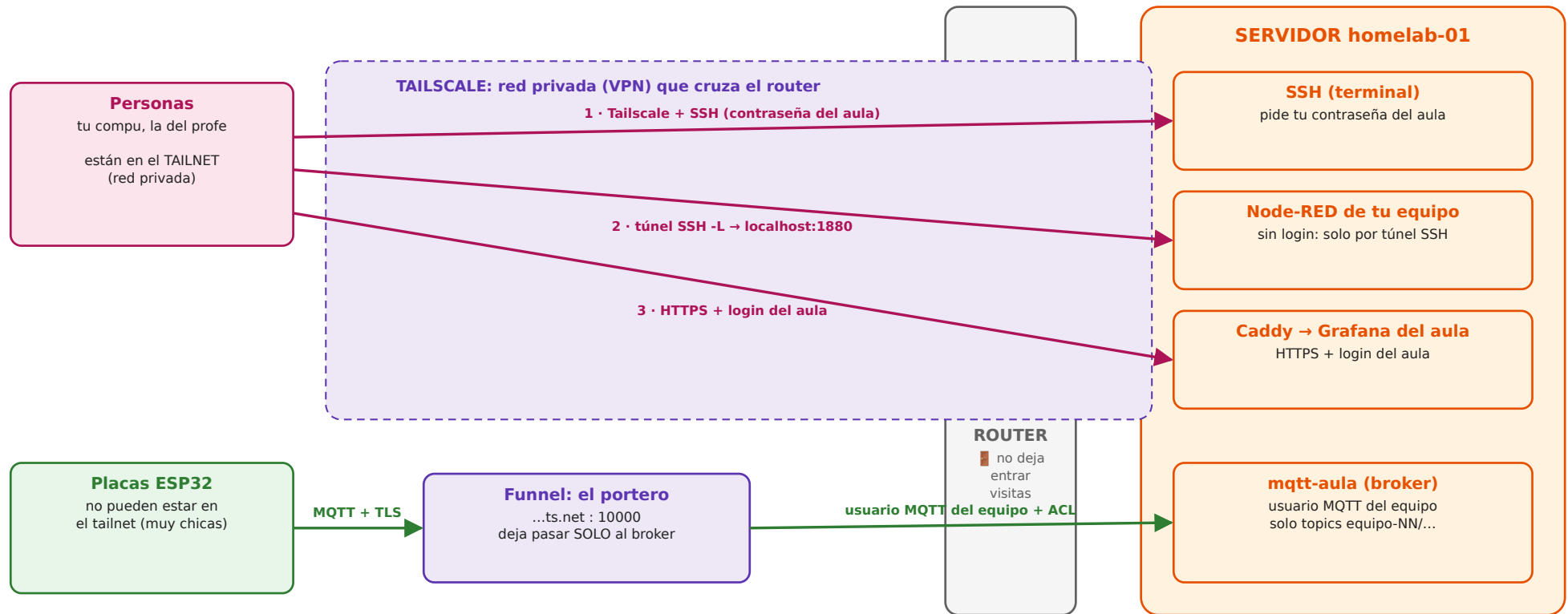
Tailscale es el programa que arma esa VPN en el aula. Lo instalás en tu compu, te unís con una clave, y tu compu queda en la misma red privada que el servidor (a esa red se le dice **tailnet**). El servidor tiene ahí una dirección propia: 100.110.123.76 (las direcciones de Tailscale empiezan con 100.).

¿Por qué hace falta? Porque el servidor está en una casa, detrás de un router que **no deja entrar a nadie de afuera**. Tailscale arma un **túnel** (un pasillo cifrado) que cruza ese router de forma segura. Por adentro usa **WireGuard**, un sistema de túneles cifrados muy liviano.

Las placas ESP32 son demasiado chicas para correr Tailscale: entran por otro camino, **Funnel**, un "portero" de Tailscale en internet que solo les abre la puerta del broker.

Dos caminos al servidor (y por qué son distintos)

El router de la casa no deja entrar a nadie de afuera. Cada uno llega por su propio camino privado.



Regla de diseño: lo que no tiene login no se publica; se llega por un túnel que sí tiene login.

Para entenderlo a fondo (qué ve cada máquina, qué es un túnel SSH, por qué te piden tantas contraseñas) → [Red y accesos](#).

¿Qué es DNS? ¿Y un dominio?

Un **dominio** es un nombre fácil de recordar (`grafana.lucasland.duckdns.org`). El **DNS** (Domain Name System) es la "guía telefónica" que traduce ese nombre a una IP. Cuando el navegador quiere entrar a un dominio, primero le pregunta al DNS "¿qué IP es?".

¿Qué es DuckDNS?

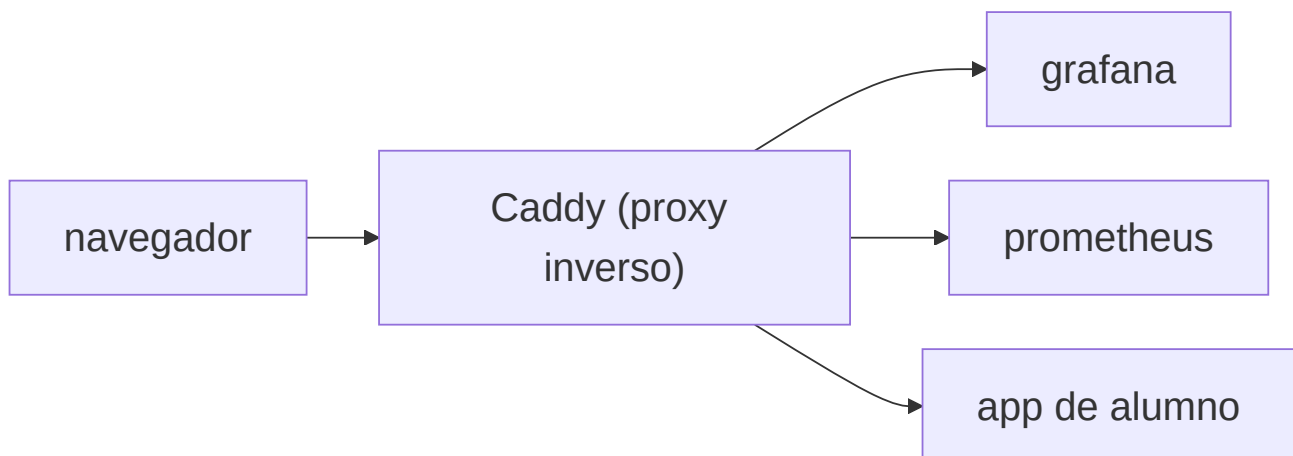
Un servicio **gratuito** de **DNS dinámico (DDNS)**. Te da un dominio (`tunombre.duckdns.org`) y, como tu IP pública de casa **cambia** cada tanto, DuckDNS la mantiene actualizada automáticamente (un timer en el server le avisa la IP nueva). Así el dominio siempre apunta a tu casa.

¿Qué es HTTPS? ¿Y TLS/certificados?

HTTP es el idioma de la web. **HTTPS** es HTTP **cifrado** con **TLS**: nadie en el medio puede leer ni modificar lo que va y viene (es el candado del navegador). Para eso, el servidor necesita un **certificado**: un documento firmado por una **autoridad de certificación** (una organización en la que los navegadores confían, como **Let's Encrypt** o **ZeroSSL**, que los dan gratis) que prueba que ese dominio es realmente ese servidor. Es como un DNI firmado por el Registro Civil. En este proyecto, **Caddy** obtiene y renueva los certificados **solo**, automáticamente.

¿Qué es un proxy inverso?

Un **proxy inverso** es un servidor que se pone **adelante** de otros y recibe todas las peticiones, para después repartirlas. Ventajas: un solo punto de entrada, un solo lugar donde manejar HTTPS, y un lugar donde poner el login. **Caddy** es el proxy inverso de este proyecto.



Ideas clave

- Un **proceso** es un programa corriendo; un **servicio/daemon** es un proceso de fondo permanente; **systemd** los administra.
- Linux controla el acceso con **usuarios, grupos y permisos** (`rwX` , números como `750` , `setgid`).
- Una **IP** identifica una máquina; un **puerto**, un servicio dentro de ella; el **DNS** traduce nombres a IPs; **HTTPS** es la web cifrada.
- Un **proxy inverso** (Caddy) es la puerta única de entrada web.
- Los datos viajan por una **pila de protocolos** (MQTT → TLS → TCP → IP → WiFi); cada capa resuelve un solo problema.
- **Tailscale** es una **VPN**: pone tu compu y el servidor en la misma red privada, cruzando el router con un túnel cifrado.

Errores comunes

- Confundir `127.0.0.1` (solo la propia máquina) con `0.0.0.0` (todas las interfaces, accesible desde afuera).
- Olvidar que las IP privadas no se alcanzan desde Internet sin port-forward.
- Editar permisos con `chmod` y romper el acceso de un servicio (ej: Grafana no podía leer una carpeta `750` — ver [cap. 10](#)).

Preguntas de repaso

1. Diferencia entre proceso, servicio y daemon.
2. ¿Qué hace el bit **setgid** en una carpeta y por qué importa acá?
3. ¿Qué traduce el DNS? ¿Y qué mantiene actualizado DuckDNS?
4. ¿Por qué `127.0.0.1:8080` no es accesible desde otra computadora?
5. Ordená de arriba hacia abajo: TCP, MQTT, WiFi, IP, TLS. ¿Qué hace cada uno?
6. ¿Por qué tu compu entra por Tailscale y tu ESP32 no?

Ejercicios

1. Convertí a `rwX` estos permisos: `640` , `755` , `700` .
2. Explicá con tus palabras qué pasa desde que escribís un dominio en el navegador hasta que ves la página (mencioná DNS, IP, puerto, TLS).
3. Buscá tres daemons en tu propia máquina Linux (pista: terminan en `d`).

3. Docker y contenedores

 **Objetivo:** entender qué es Docker, la diferencia entre imagen y contenedor, y cómo se describen aplicaciones con Docker Compose. Es la base para todo el plano de servicios y el de aula.

 **Prerequisitos:** [cap. 2 \(Fundamentos\)](#).

 **Conceptos nuevos:** imagen, contenedor, engine, Compose, red Docker, volumen, bind mount, healthcheck, límites de recursos, cgroup.

Empecemos por lo cercano: el contenedor de un barco

Antes de los contenedores de carga, cargar un barco era un caos: bolsas, cajas y barriles de todos los tamaños, cada uno se acomodaba distinto. El **contenedor** de metal lo cambió todo: **una caja estándar** que entra igual en un barco, un camión o un tren, sin importar qué lleva adentro.

Docker hace lo mismo con los programas: mete una aplicación **con todo lo que necesita** en una "caja" estándar que corre igual en cualquier computadora. De ahí el nombre "contenedor".

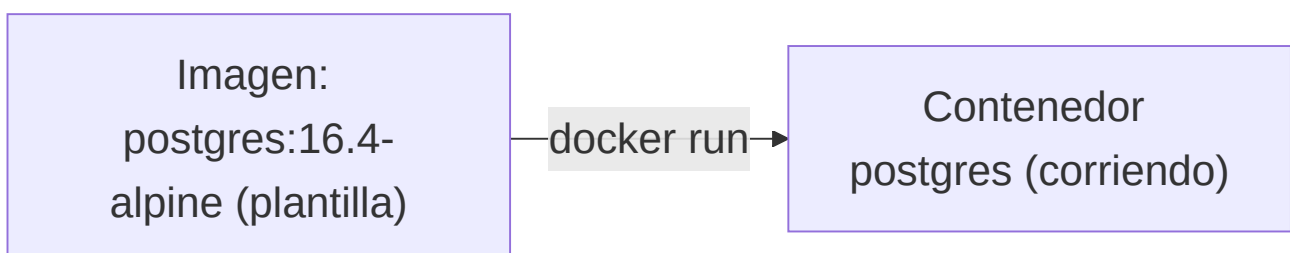
El problema que resuelve Docker

Antes, instalar una aplicación en un servidor era un dolor: dependencias que chocan, "en mi máquina funcionaba", versiones distintas. **Docker** empaqueta una aplicación **con todo lo que necesita** (librerías, configuración) en una unidad que corre igual en cualquier lado.

Imagen vs. contenedor

Ésta es **la** distinción clave:

- Una **imagen** es una **plantilla de solo lectura**: la app + sus dependencias, congeladas. Ejemplo: `postgres:16.4-alpine`. Es como el molde de una torta.
- Un **contenedor** es una **imagen en ejecución**: una copia viva y aislada. De una imagen podés prender muchos contenedores. Es la torta hecha con ese molde.



Otra analogía: la imagen es la **receta**, el contenedor es el **plato servido**. Con una receta cocinás muchos platos iguales.

¿Qué es alpine ? En `postgres:16.4-alpine`, lo que va después de `:` es la **etiqueta** (*tag*): la versión exacta. `alpine` es una versión de Linux muy chiquita que se usa de "base", para que la imagen ocupe poco.

El engine

El **motor de Docker** (`dockerd`, un daemon) es el que construye imágenes, y prende/apaga contenedores. En este proyecto hay **un solo engine**, administrado por el operador. Los alumnos **no** lo tocan directo (ver [cap. 9](#)).

¿Por qué "aislado"?

Cada contenedor corre como si tuviera su propio mini-sistema: su propio filesystem, su propia red, sus propios procesos. Ese aislamiento lo da el **kernel** de Linux con dos mecanismos:

- **namespaces** ("espacios de nombres"): son como **anteojeras**. Deciden **qué ve** el contenedor: solo sus propios archivos, sus procesos y su red, no los de los demás.
- **cgroups** ("grupos de control"): son como un **medidor con corte**. Deciden **cuánto usa**: cuánta CPU, RAM y procesos (ver abajo).

Docker Compose

Prender un contenedor a mano (`docker run` con veinte flags) es incómodo. **Docker Compose** permite describir uno o varios contenedores en un archivo **YAML** (`compose.yml`) y prenderlos juntos con `docker compose up`.

YAML (*YAML Ain't Markup Language*) es un formato de texto para escribir configuración, que se lee casi como una lista hecha a mano: `nombre: valor`, y lo que va "adentro" de algo se escribe **más a la derecha** (con espacios, no con tabulación). Si la sangría está mal, el archivo no se entiende.

Ejemplo mínimo (de este proyecto):

```

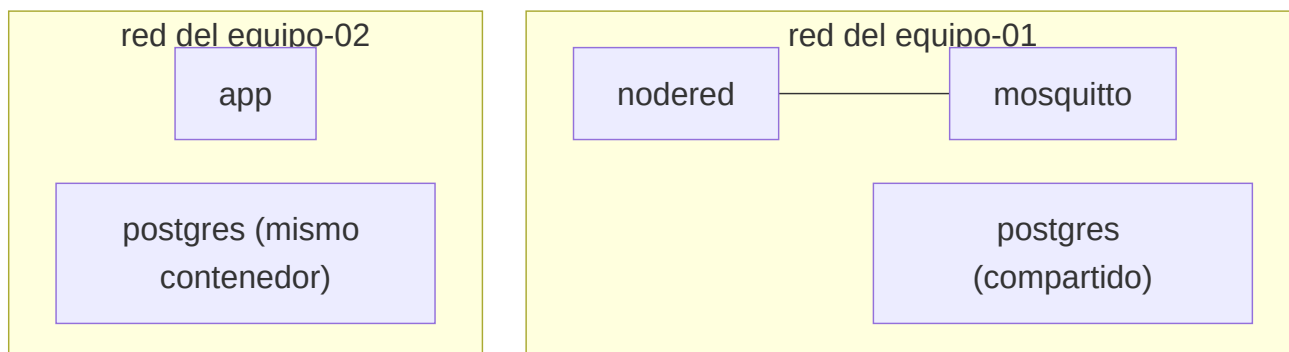
services:
  web:
    image: nginxdemos/hello:plain-text    # qué imagen usar
    restart: unless-stopped               # reiniciar si se cae
    logging: { driver: json-file, options: { max-size: 10m, max-file: "3" } }
    deploy:
      resources:
        limits: { cpus: "0.25", memory: 128M, pids: 100 }    # topes
    ports: ["127.0.0.1:8080:80"]          # publicar puerto (solo local)

```

- **services:** — la lista de contenedores.
- **image:** — la imagen (siempre con versión fija, nunca `latest`).
- **ports:** — "publica" un puerto del contenedor en la máquina **anfitriona** (el *host*: la computadora real donde corre Docker, en nuestro caso el servidor). `127.0.0.1:8080:80` = el puerto 80 del contenedor se ve como 8080 del servidor, **solo en loopback** (`127.0.0.1`, "esta misma máquina": nadie de afuera lo ve).

Redes Docker

Cuando levantas un proyecto Compose, Docker le crea una **red privada** propia. Los contenedores de ese proyecto se ven entre sí **por su nombre** (`web` puede hablar con `db`), pero **no ven** a los de otro proyecto. Este aislamiento por red es la base del aula: cada equipo tiene su red, y los servicios compartidos (Postgres) se conectan a la red de cada equipo sin que los equipos se vean entre sí.



Ejemplo completo: [ejemplos/nodered-mqtt/](#) — un `compose.yml` de laboratorio (Node-RED + Mosquitto) que usa la **Postgres compartida** del aula vía `.shared-services.env` (no una db propia), cumple la política y se levanta con `labctl validate && labctl up`.

Volúmenes y bind mounts

Un contenedor es **efímero** (dura poco, es descartable): si lo borrás, se pierde lo que escribió adentro. Es como una mesada que se limpia al final del día. Para **guardar datos**, se montan carpetas desde afuera:

- **Bind mount:** montás una carpeta del host dentro del contenedor (`./datos:/var/lib/algo`). Los datos viven en el host, en una ruta que vos controlás. En el aula, la política **obliga** a usar bind mounts dentro de la carpeta del equipo (para que cuenten contra su cuota de disco).
- **Volumen con nombre:** Docker administra un espacio en `/var/lib/docker`. Es cómodo, pero los datos quedan "escondidos" en un lugar compartido. En el aula están **prohibidos** para los alumnos (romperían la cuota).

healthcheck

Un **healthcheck** es un comando que Docker corre periódicamente para saber si el contenedor está **sano** (no solo "prendido", sino "funcionando"). Si falla varias veces, Docker lo marca `unhealthy`. Por ejemplo, Postgres usa `pg_isready`.

restart

La política **restart** dice qué hacer si el contenedor se cae: `unless-stopped` = reinicialo solo, salvo que yo lo haya apagado a propósito. Así los servicios vuelven después de un reinicio del server.

Límites de recursos y cgroups

Un **cgroup** (control group) es un mecanismo del kernel para **limitar** cuánta CPU, RAM o cuántos procesos puede usar un grupo de contenedores. En Compose se declaran en `deploy.resources.limits`:

```
deploy:
  resources:
    limits: { cpus: "0.5", memory: 256M, pids: 200 }
```

Esto es clave en hardware modesto: sin límites, un error de un alumno (un bucle infinito, una fuga de memoria) podría **tumbar el servidor entero**. En el aula se usa una **doble capa**: los límites del compose **y** un cgroup de systemd por equipo (ver [cap. 9](#)).

logging

Sin límite, los logs de un contenedor pueden **llenar el disco**. Por eso se configura **rotación**: `max-size: 10m, max-file: 3` = guardá hasta 3 archivos de 10 MB y después descartá lo viejo.



Ideas clave

- **Imagen = plantilla; contenedor = imagen corriendo.** De una imagen salen muchos contenedores.
- **Compose** describe contenedores en YAML y los prende juntos.
- El aislamiento (namespaces + cgroups) permite correr muchos proyectos en una sola máquina sin que se pisen.

- **Bind mounts** guardan datos; **límites** y **rotación de logs** evitan que un contenedor se coma la máquina.

Errores comunes

- Usar imágenes `:latest` → despliegues no reproducibles (hoy anda, mañana no).
- Olvidar los **límites** → un contenedor se come toda la RAM.
- Guardar datos **dentro** del contenedor (se pierden) en vez de en un bind mount.
- Publicar en `0.0.0.0` sin querer → exponés el servicio a toda la red.

Preguntas de repaso

1. Explicá la diferencia entre imagen y contenedor con una analogía propia.
2. ¿Para qué sirve un healthcheck? Dá un ejemplo.
3. ¿Qué diferencia hay entre un bind mount y un volumen con nombre?
4. ¿Qué es un cgroup y por qué importa en hardware limitado?

Ejercicios

1. Escribí un `compose.yml` mínimo que cumpla la política (imagen con tag, límites, logging, restart, puerto en 127.0.0.1).
2. Explicá qué pasaría si dos equipos usan la imagen `postgres:16.4-alpine`: ¿se descarga dos veces? (pista: una imagen está hecha de **capas**, como un sándwich de pisos: la base de Linux, después Postgres, después la configuración. Si dos imágenes comparten capas, esas se guardan una sola vez).

4. Ansible e Infraestructura como Código

 **Objetivo:** entender cómo se configura todo el servidor **escribiendo archivos** en vez de tocar cosas a mano, cómo está organizado este repositorio y cómo se aplica un cambio sin miedo a romper.

 **Prerequisitos:** [cap. 2 \(Fundamentos\)](#), [cap. 3 \(Docker\)](#).

 **Conceptos nuevos:** infraestructura como código (IaC), Ansible, playbook, rol, tarea, handler, template, inventario, variable, vault, tag, idempotencia, drift.

Empecemos por lo cercano: armar un mueble

Hay dos formas de armar un mueble:

1. **"A ojo":** vas probando, atornillás donde parece, y queda. Funciona... hasta que querés armar **otro igual**, o se desarma y no te acordás cómo era.
2. **Con el manual de armado:** paso a paso, con las piezas numeradas. Cualquiera lo arma igual, las veces que quiera.

Configurar un servidor "a mano" (entrar, instalar, editar archivos) es armar a ojo.

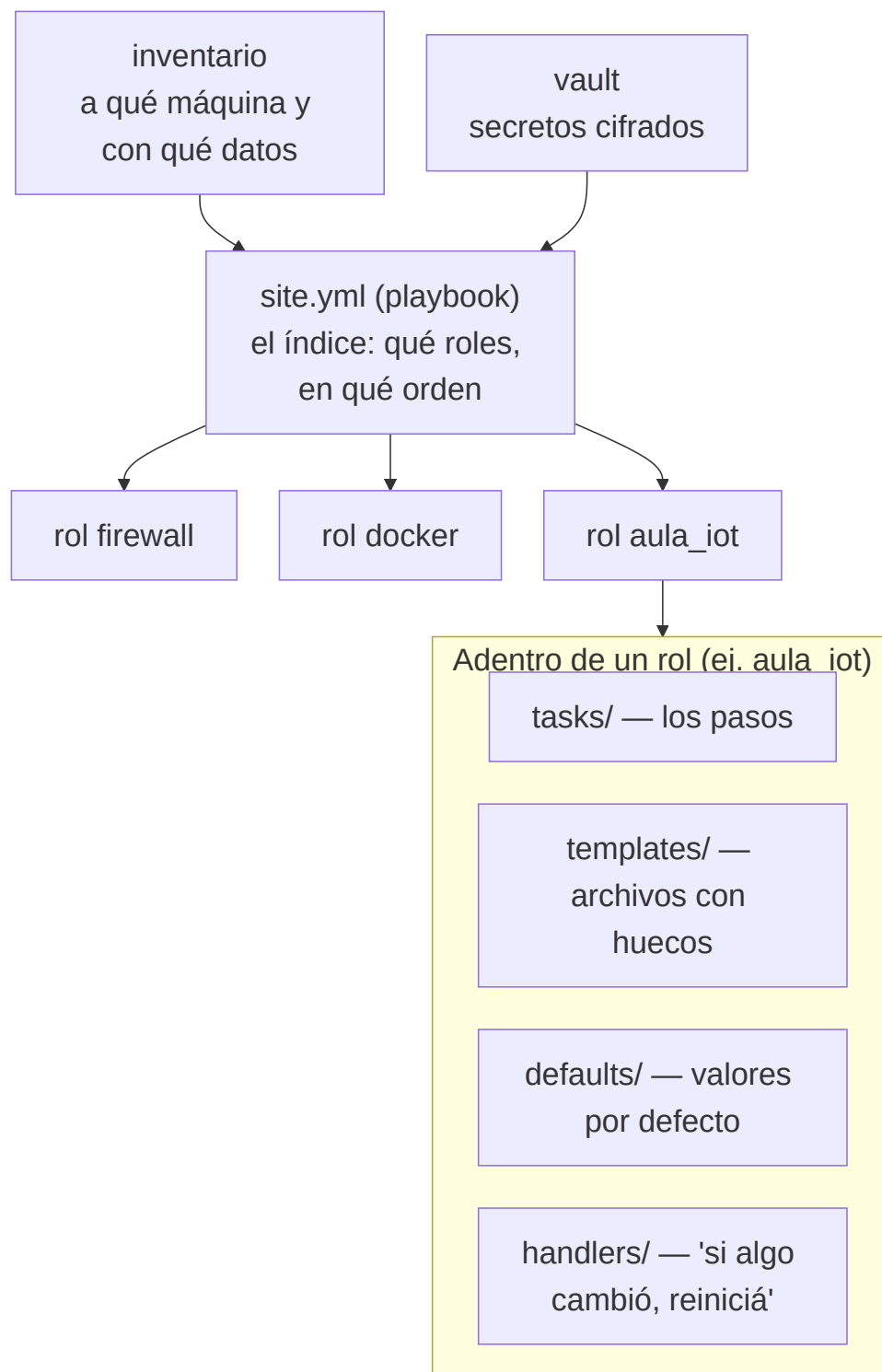
Infraestructura como Código es escribir el **manual de armado** del servidor, en archivos de texto, y dejar que un programa lo siga al pie de la letra. Ese programa es **Ansible**.

¿Qué es Infraestructura como Código (IaC)?

IaC (*Infrastructure as Code*) es describir **cómo tiene que estar** un servidor (qué programas, qué usuarios, qué reglas de firewall, qué servicios) en **archivos**, en vez de configurarlo a mano. Las ventajas, con ejemplos de este proyecto:

Ventaja	Qué significa	Ejemplo real
Reproducible	se puede reconstruir igual	si el servidor se rompe, se reinstala Ubuntu y se aplica el repo
Revisable	los cambios se leen antes de aplicarse	cada cambio pasa por una rama y una revisión (cap. 8)
Con historia	se sabe qué cambió, cuándo y por qué	la línea de tiempo del cap. 15 sale de ahí
Escalable	lo mismo sirve para otro servidor	<code>make new-site NAME=colegio-norte</code> arma otro sitio igual

Las piezas de Ansible (con ejemplos de este repo)



Playbook: el índice

El **playbook** es el archivo principal: dice **qué roles** se aplican y **en qué orden**. El de este repo es `ansible/site.yml`, y está ordenado por "planos" (ver [cap. 1](#)):

Orden	Plano	Roles
1	Base del host	<code>bootstrap</code> , <code>users_ssh</code>
2	Red	<code>tailscale</code> , <code>dns</code> , <code>ddns</code>
3	Seguridad	<code>firewall</code> , <code>fail2ban</code> , <code>apparmor</code> , <code>hardening</code> , <code>auto_updates</code> , <code>audit</code>
4	Servicios	<code>docker</code> , <code>authelia</code> , <code>monitoring</code> , <code>backups</code> , <code>panol</code>
5	Aula	<code>classroom</code> , <code>shared_services</code> , <code>aula_iot</code> , <code>labctl</code> , <code>classroom_publish</code>

El orden importa: no se puede levantar un contenedor (plano 4) sin haber instalado Docker antes.

Rol: una caja con un solo trabajo

Un **rol** es una carpeta con todo lo necesario para **una** responsabilidad. Está en `ansible/roles/<nombre>/`. Por ejemplo, `aula_iot` levanta la capa de Grafana del aula. Adentro:

- `tasks/main.yml` — **las tareas**: los pasos, en orden. Cada tarea usa un **módulo** (una herramienta de Ansible: copiar un archivo, instalar un paquete, levantar un compose). Ejemplo real:

```
- name: Bring up the aula-iot stack
  community.docker.docker_compose_v2:
    project_src: "{{ aula_iot_dir }}"
    state: present
```

Se lee: "que el stack de `aula_iot_dir` esté **presente** (levantado)". Fijate que no dice "levantalo": dice **cómo tiene que quedar**. Eso se llama ser **declarativo**.

- `templates/` — **archivos con huecos**: un archivo de configuración con partes que se completan solas. En `telegraf.conf.j2` hay una lista de topics que se arma con los equipos del aula: si se suma un equipo, el archivo se actualiza solo. Los huecos se escriben `{{ así }}` (es el formato **Jinja2**, de ahí la extensión `.j2`).
- `defaults/main.yml` — **valores por defecto**: los "ajustes de fábrica" del rol. Ejemplo: `aula_iot_retention: "15d"` (cuánto historial se guarda).
- `handlers/` — **"si algo cambió, hacé esto"**: una tarea que corre **solo** si otra avisó que cambió algo. Ejemplo real: si cambia el `Caddyfile`, se **reinicia** Caddy. Si no cambió nada, no se reinicia nadie (así no se cortan servicios de gusto).

Inventario: a qué máquina y con qué datos

El **inventario** dice **a qué servidores** se aplica y con **qué datos propios** de ese lugar. Hay dos en `ansible/inventories/`:

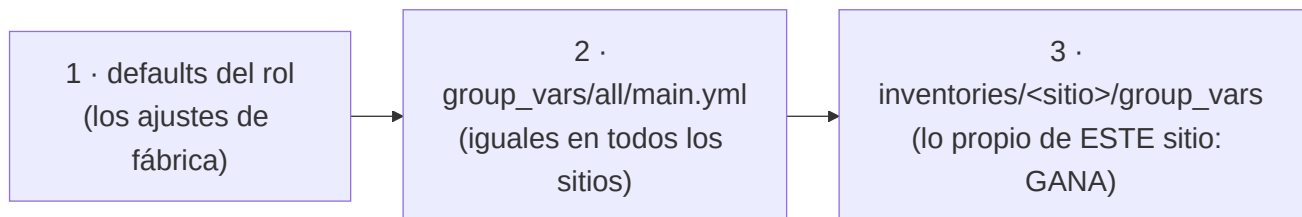
- `production/` — el servidor real del aula (`homelab-01`).

- `staging/` — un servidor de **prueba**, para ensayar antes.

Adentro de cada uno, `group_vars/all/` tiene los datos del sitio: el dominio (`lucasland.duckdns.org`), la red de la casa (`lan_cidr`), y el **roster** del aula (`classroom.yml` : equipos, alumnos y lo que se publica).

Las tres capas de variables

Una **variable** es un dato con nombre que se usa en muchos lados (`aula_iot_retention` , `lan_cidr`). En este repo vienen de tres lugares, y gana el más específico, como un estatuto escolar, el reglamento del curso y lo que acuerda tu equipo:



Vault: la caja fuerte

Las contraseñas no pueden estar en texto común dentro del repositorio. **Ansible Vault** guarda los secretos **cifrados** en `vault.yml` , y solo se abren con una clave que vive **en el servidor** (`~/.vault_pass`), nunca en el repo.

Algunos secretos ni siquiera van al vault: se **generan en el servidor** la primera vez (por ejemplo, la clave MQTT de cada equipo, en `/etc/classroom/secrets/`). Así sumar un equipo no obliga a tocar secretos cifrados.

Tags: aplicar solo una parte

Cada rol tiene **etiquetas** (`tags`). Con `--tags` se aplica solo lo que tiene esa etiqueta. Ejemplo: `--tags classroom` aplica todo lo del aula (usuarios, servicios compartidos, broker, Grafana del aula, labctl) y no toca el firewall.

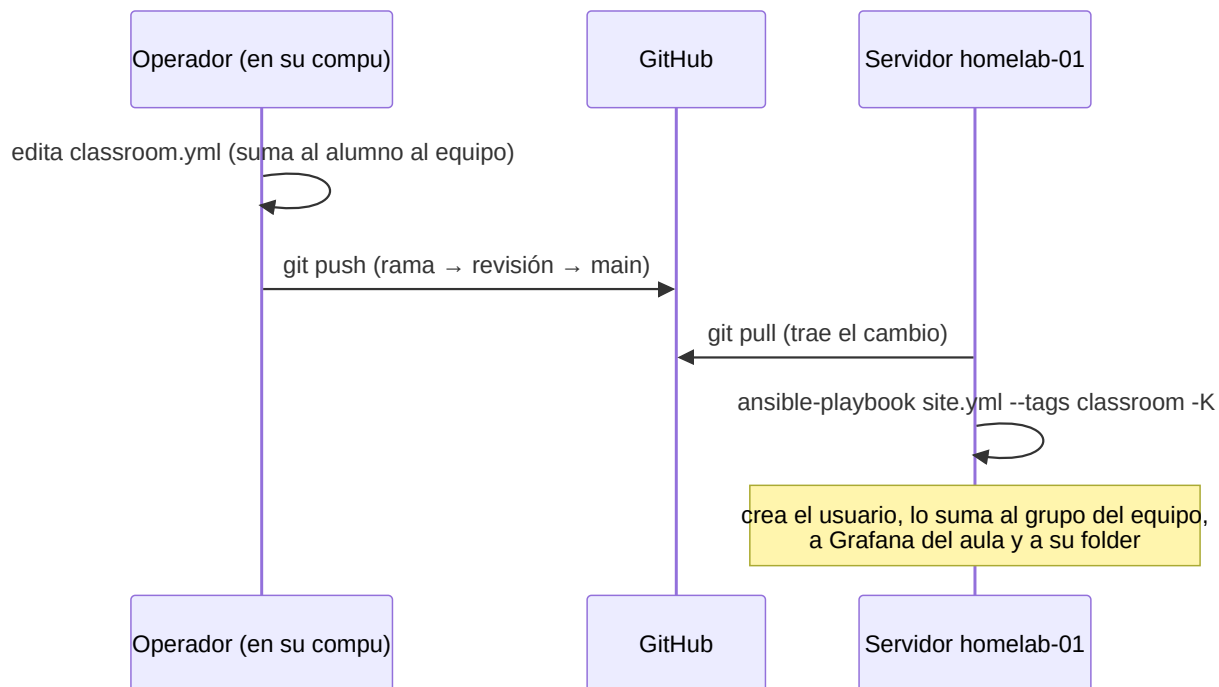
Idempotencia: aplicar dos veces da lo mismo

Un interruptor de luz común **cambia** de estado cada vez que lo apretás: si lo apretás dos veces, volvés al principio. Ansible no funciona así: funciona como decir "**que la luz quede prendida**". Si ya está prendida, no hace nada.

Eso es **idempotencia**: aplicar el playbook **una o diez veces da el mismo resultado**. Por eso es seguro volver a correrlo. Hay una prueba automática que lo verifica: `make idempotence` corre todo **dos veces** y falla si la segunda vez cambió algo.

Cómo se aplica un cambio, paso a paso

El ejemplo real: dar de alta a un alumno.



En el servidor, como usuario `homelab`:

```
cd ~/homelab && git pull
cd ~/homelab/ansible
~/homelab/.venv/bin/ansible-playbook site.yml --tags classroom -K
```

Cada parte del último comando:

Parte	Qué es
<code>~/homelab/.venv/bin/ansible-playbook</code>	el programa, por su ruta completa (está instalado en un entorno propio, <code>.venv</code> , y no en el <code>PATH</code> ; ver cap. 2)
<code>site.yml</code>	el playbook
<code>--tags classroom</code>	solo la parte del aula
<code>-K</code>	"preguntame la contraseña de <code>sudo</code> " (la cuenta <code>homelab</code> necesita contraseña para hacer cosas de administrador)

Antes de aplicar algo grande: `make dry-run` muestra **qué cambiaría** sin cambiar nada (modo *check*). Es mirar el plano antes de mover paredes.

Drift: cuando el servidor y el repo no coinciden

Drift (deriva) es cuando el servidor real **ya no es** lo que dicen los archivos, porque alguien cambió algo a mano. Es peligroso porque la próxima vez que se aplique Ansible, **pisa** lo hecho a mano, o al revés: lo de a mano queda como un misterio que nadie puede reconstruir.

Caso real (septiembre 2026): el broker del aula (`mqtt-aula`) se levantó primero a mano, para que las placas pudieran conectarse ese mismo día. Durante unos días existió **solo en el servidor**: si alguien corría Ansible desde la rama principal, las placas de tres equipos se iban a desconectar. Se resolvió pasando todo al repositorio y verificando que el servidor y el repo coincidieran. La regla que quedó: **lo de a mano es provisorio y se anota**.

Comandos del operador (`make`)

El `Makefile` de la raíz junta los comandos más usados con nombres simples. Algunos:

Comando	Qué hace
<code>make dry-run</code>	muestra qué cambiaría, sin cambiar nada
<code>make apply</code>	aplica todo
<code>make harden</code>	aplica solo la seguridad
<code>make firewall</code>	aplica solo el firewall
<code>make panol</code>	redespliega el pañol
<code>make idempotence</code>	prueba que una segunda corrida no cambia nada
<code>make test</code>	pruebas sobre el servidor ya armado
<code>make logs-on</code> / <code>make logs-off</code>	prende o apaga los logs en vivo
<code>make como-conectar</code>	dice cuál es la mejor forma de entrar al servidor ahora



Ideas clave

- **laC** = el servidor descrito en archivos: reproducible, revisable, con historia.
- **Playbook** (el índice) → **roles** (una responsabilidad cada uno) → **tareas** (los pasos), **templates** (archivos con huecos), **handlers** (reiniciar solo si cambió algo).
- El **inventario** dice dónde y con qué datos; el **vault** guarda los secretos.
- **Idempotencia**: aplicar dos veces da lo mismo, por eso es seguro.
- **Drift**: lo hecho a mano y no anotado es una bomba de tiempo.

Errores comunes

- **Cambiar cosas a mano en el servidor** y no pasarlas al repo (drift).
- Olvidar el `-K` → Ansible no puede usar `sudo` y falla.
- Correr Ansible desde otra carpeta que no sea `~/homelab/ansible` → no encuentra su configuración (`ansible.cfg`) ni la clave del vault.
- Poner un secreto en un archivo común del repo en vez del vault.

Preguntas de repaso

1. ¿Qué diferencia hay entre configurar "a mano" y con IaC? Usá la analogía del mueble.
2. ¿Qué es un rol y qué partes tiene?
3. ¿Qué quiere decir que Ansible sea idempotente? ¿Por qué es importante?
4. ¿Qué es el drift y cómo se evita?

Ejercicios

1. Abrió `ansible/site.yml` y ubicá cada rol en uno de los cinco planos.
2. Abrió `ansible/roles/aula_iot/defaults/main.yml` y explicá con tus palabras tres de sus variables.
3. Escribí, en orden, los pasos para sumar un alumno nuevo a un equipo (dónde se edita, cómo llega al servidor, qué comando se corre).

Parte II — La plataforma

5. Los servicios uno por uno

 **Objetivo:** conocer cada servicio del laboratorio: qué es, para qué sirve, dónde corre, qué expone, de qué depende y **qué pasa si se cae**.

 **Prerequisitos:** [cap. 3 \(Docker\)](#). Para los servicios de las placas, conviene haber leído el [cap. 11 \(Arquitectura IoT\)](#).

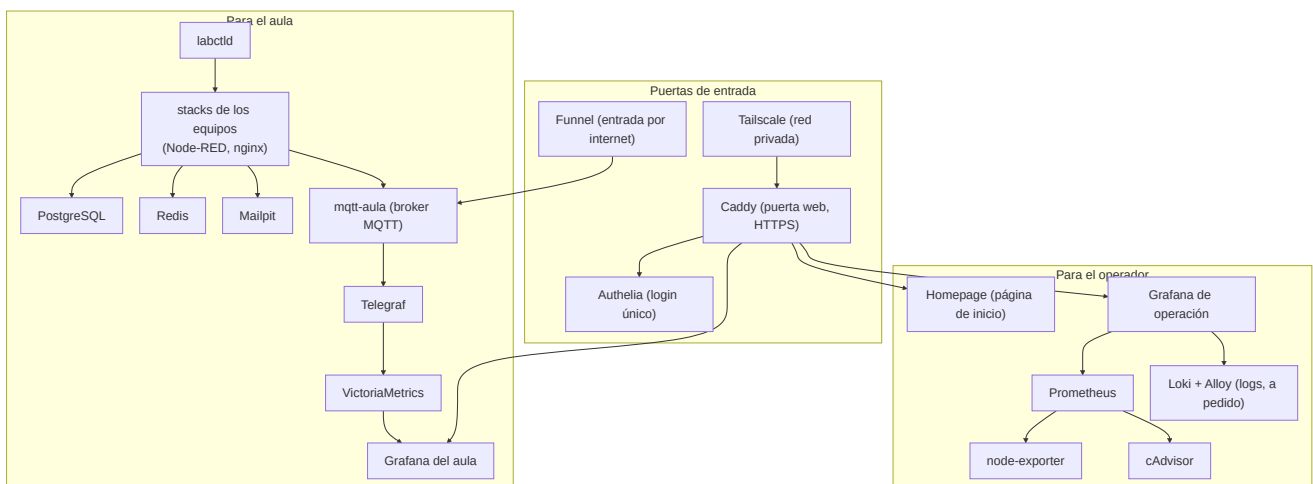
 **Conceptos nuevos:** proxy inverso, SSO, exporter, scrape, ficha de servicio, dependencia, punto único de falla.

Empecemos por lo cercano: los oficios de un barrio

En un barrio cada uno tiene su oficio: el portero, el cartero, el que anota en el libro de actas, la biblioteca. Si se enferma el cartero, las cartas no llegan, pero la biblioteca sigue abierta. Si se enferma el portero, **nadie entra** a ningún lado.

En el servidor pasa lo mismo: cada **servicio** tiene un oficio, y algunos son "porteros" de los que dependen muchos otros. Conocer **quién depende de quién** es lo que permite saber qué se rompe cuando algo se cae.

El mapa completo



Las fichas

Cada ficha responde siempre lo mismo: **qué es** (con algo cercano), **para qué**, **dónde corre**, **qué expone**, **de qué depende** y **qué pasa si se cae**.

Puertas de entrada

Tailscale — la red privada del aula

- **Qué es:** una red privada entre las computadoras de ustedes y el servidor, como si estuvieran en la misma casa aunque estén lejos. Por dentro usa **WireGuard**, un sistema de túneles cifrados.
- **Dónde corre:** en el servidor (`tailscaled` , un daemon) y en cada compu.
- **Expone:** la IP privada `100.110.123.76` .
- **Si se cae:** nadie entra desde afuera de la casa (ni SSH ni web). La consola física del servidor sigue funcionando.
- **Para entenderlo desde cero** (qué problema resuelve, qué es un túnel) → [Red y accesos](#).

Tailscale Funnel — el portero de internet

- **Qué es:** publica un puerto del servidor **en internet** sin abrir puertas en el router.
- **Expone:** `:10000` → broker del aula (placas); `:8443` → API del pañol.
- **Si se cae:** las placas no llegan al broker. Ojo: lo opera la empresa Tailscale; a veces falla **una** de sus entradas (ver [caso 7](#)).

Caddy — la puerta web

- **Qué es:** un **proxy inverso**: recibe todos los pedidos web y los manda al servicio que corresponde (como una recepción que deriva a cada oficina). Además pone el **HTTPS** (el candado) solo, con certificados gratuitos que renueva solo.
- **Dónde corre:** contenedor `caddy` . Configuración en `compose/proxy/Caddyfile` .
- **Expone:** puertos 80 y 443.
- **Si se cae:** **ninguna página web** del servidor responde. Es un **punto único de falla** (si falla, falla todo lo que pasa por él).

Authelia — el login único

- **Qué es:** un **SSO** (*Single Sign-On*, "inicio de sesión único"): te logueás una vez y entrás a todo lo que tenés permitido. Sabe a qué **grupo** pertenecés (`operators` u `students`) y decide.
- **Dónde corre:** contenedor `authelia` . Caddy le pregunta antes de dejar pasar.
- **Si se cae:** las páginas protegidas no dejan entrar a nadie.

Para el operador

Homepage — página de inicio con enlaces a todo. Si se cae, se pierde el "menú", pero cada servicio sigue andando.

Prometheus — el que pasa a leer los medidores

- **Qué es:** cada 15 segundos **va a buscar** (*scrape*) números a los *exporters* y los guarda 15 días. Como el empleado que pasa a leer el medidor de luz de cada casa.
- **Expone:** solo `127.0.0.1:9090` (adentro del servidor).

- **Si se cae:** los tableros de operación se quedan sin datos nuevos.

node-exporter y **cAdvisor** — *los medidores*

- **node-exporter** mide el **servidor** (CPU, RAM, disco, red).
- **cAdvisor** mide **cada contenedor** (cuánto usa cada uno). De ahí salen los tableros por equipo.
- **Ojo:** cAdvisor anda cerca de su tope de memoria; si se pasa, el sistema lo corta y se pierden las métricas por contenedor un rato.

Grafana de operación — tableros del servidor, del aula (consumo por equipo) y del pañol. **Solo operadores**, porque muestra la auditoría del pañol (quién entró y cuándo).

Loki + Alloy — *logs a pedido*

- **Qué son:** Alloy junta los **logs** (registros) de todos los contenedores y Loki los guarda para buscarlos desde Grafana.
- **Por qué a pedido:** guardar todos los logs todo el tiempo gasta RAM y disco que necesita el aula. Se prenden con `make logs-on` y se apagan solos.

Para el aula

PostgreSQL, Redis, Mailpit — *los servicios comunes*

- **PostgreSQL:** base de datos; cada equipo tiene la suya, con su usuario.
- **Redis:** memoria rápida de "clave → valor" (para caché o colas chicas).
- **Mailpit:** un correo de mentira que **atrapa** los mails para probar envíos sin mandarlos de verdad.
- **Si se cae** uno: se rompe solo lo de los equipos que lo usan.

mqtt-aula — *el cartero de las placas*

- **Qué es:** el **broker MQTT** del aula (Mosquitto 2.0). Detalle en el [cap. 11](#).
- **Expone:** `127.0.0.1:18830`, que Funnel publica en `:10000` con TLS.
- **Depende de:** Funnel (para las placas), la red de cada equipo (para su Node-RED).
- **Si se cae:** ninguna placa puede mandar ni recibir. Las placas reintentan solas y vuelven cuando el broker vuelve.

Telegraf, VictoriaMetrics y el exporter del broker — *el escribano y el archivo*

- **Telegraf** escucha los topics de los equipos (solo puede **leer**), traduce a números y escribe en **VictoriaMetrics**, que guarda 15 días.
- **El exporter** lee las estadísticas del propio broker (cuántos clientes, cuántos mensajes).
- **Si se caen:** las placas siguen funcionando entre ellas y con Node-RED; solo se deja de **guardar** historial para Grafana.

Grafana del aula — un tablero por equipo, que solo ese equipo ve y edita, y uno general para el operador. Ver [cap. 13](#).

labctl — el encargado del edificio (ver [cap. 9](#)). Si se cae, los alumnos no pueden hacer `labctl up/down`, pero lo que ya está corriendo **sigue corriendo**.

Stacks de los equipos — el Node-RED y el nginx de cada equipo, adentro de su carpeta, con sus propios puertos (ver la [guía del equipo](#)).

En el servidor, sin contenedor

- **DuckDNS** (un temporizador cada 5 minutos): mantiene el nombre `lucasland.duckdns.org` apuntando a la IP pública de la casa.
- **dnsmasq** (rol `dns`): adentro de la red privada, hace que esos nombres apunten a la IP de Tailscale (eso se llama **Split DNS**: el mismo nombre responde distinto según desde dónde preguntes).

¿Qué pasa si se cae...? (resumen)

Si se cae...	Se rompe	Sigue andando
Tailscale	entrar desde afuera de la casa	todo lo que corre adentro, las placas (van por Funnel)
Funnel	las placas no llegan al broker	todo lo web y el SSH por Tailscale
Caddy	todas las páginas web	SSH, placas, servicios internos
Authelia	entrar a páginas protegidas	lo demás
<code>mqtt-aula</code>	mensajes de todas las placas	lo web, las bases de datos
Telegraf / VictoriaMetrics	el historial en Grafana	las placas y Node-RED
PostgreSQL	apps de los equipos que guardan datos	placas, tableros

Ideas clave

- Cada servicio tiene **un oficio**. Saber **de qué depende** es saber qué se rompe.
- **Caddy** y `mqtt-aula` son **puntos únicos de falla**: si caen, afectan a muchos.
- Prometheus **va a buscar** los datos (los medidores); las placas, en cambio, **mandan** sus mensajes (MQTT).

? Preguntas de repaso

1. ¿Qué es un proxy inverso? Usá la analogía de la recepción.
2. ¿Por qué el Grafana de operación es solo para operadores?
3. Si se cae Funnel, ¿pueden los alumnos entrar a su Node-RED? ¿Y sus placas al broker?
4. ¿Qué es un punto único de falla? Nombrá dos en este sistema.

Ejercicios

1. Elegí un servicio y armá su ficha completa con tus palabras.
2. Dibujá de qué servicios depende **tu** proyecto, desde la placa hasta Grafana.

6. Observabilidad

 **Objetivo:** entender qué significa **observar** un sistema, cómo se hace en el servidor (Prometheus, Grafana, logs) y en las placas (MQTT → Grafana del aula), y cómo leer los tableros.

 **Prerequisitos:** [cap. 3 \(Docker\)](#), [cap. 5 \(Servicios\)](#).

 **Conceptos nuevos:** observabilidad, métrica, serie temporal, etiqueta, log, exporter, scrape, *pull* y *push*, PromQL, dashboard, alerta.

Empecemos por lo cercano: el médico

Cuando vas al médico, no te abre para ver qué pasa adentro. **Observa desde afuera** con tres herramientas:

El médico usa...	En un sistema se llama...	Qué es
el termómetro y la presión	métricas	números medidos cada tanto: temperatura, RAM usada, mensajes por minuto
la historia clínica	logs (registros)	lo que fue pasando, anotado con fecha: "10:32 se conectó la placa de Jorge"
un estudio con contraste que sigue un recorrido	trazas (<i>traces</i>)	el camino completo de un pedido por todos los servicios

Observabilidad es poder entender **qué pasa adentro** de un sistema mirando lo que muestra **desde afuera**. En este sistema usamos métricas y logs (las trazas quedan para más adelante).

Métricas y series temporales

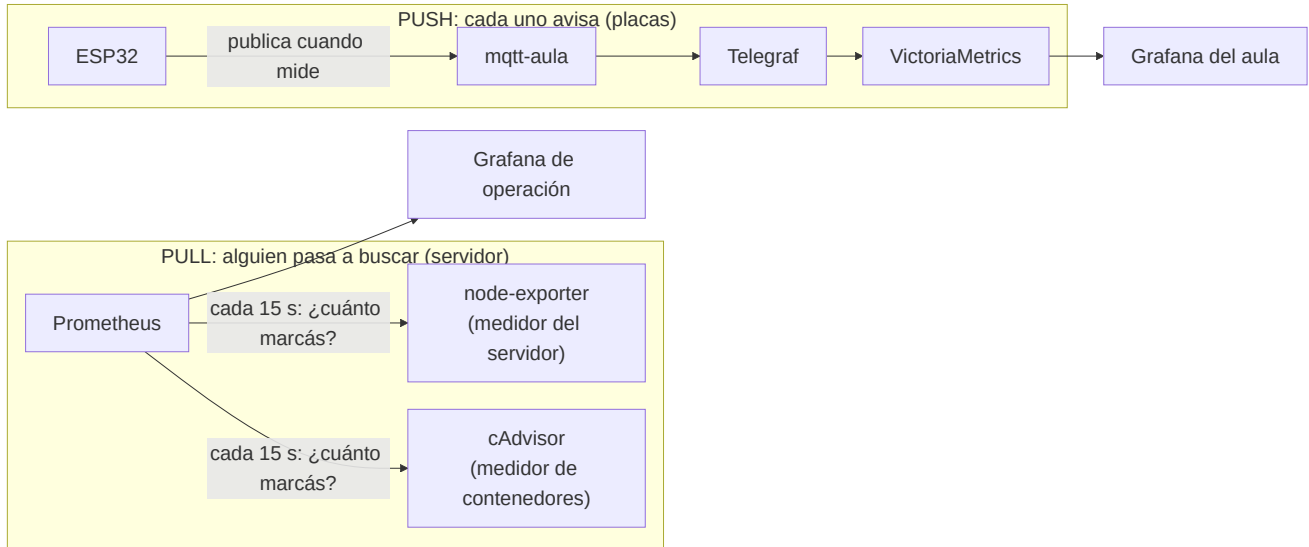
Una **métrica** es un número que se mide **muchas veces**. Guardado con la hora de cada medición, forma una **serie temporal**: una planilla de "hora | valor".

Cada serie tiene **etiquetas** (*labels*): datos que la acompañan para poder filtrarla, como las columnas de una planilla. Por ejemplo:

```
container_memory_usage_bytes{name="equipo-04-nodered-1"} = 69 000 000
mqtt_valor{equipo="equipo-04", dispositivo="enchufe", magnitud="estado"} = 1
```

La primera dice "el Node-RED del equipo-04 usa 69 MB de RAM". La segunda, "el enchufe de Jorge está prendido".

Dos formas de juntar datos: *pull* y *push*



- **Pull ("ir a buscar"):** como el empleado que pasa a leer el medidor de luz de cada casa. **Prometheus** visita cada 15 segundos a los **exporters** (programas que exponen números: `node-exporter` mide el servidor; `cAdvisor`, cada contenedor). A esa visita se le dice **scrape**.
- **Push ("avisar"):** como mandar un mensaje cuando pasa algo. Las **placas** publican por MQTT cuando miden o cambian; **Telegraf** los anota en **VictoriaMetrics**.

¿Por qué distinto? Porque una placa puede estar detrás de cualquier router y apagarse cuando quiera: nadie podría "pasar a buscarla". Es más simple que ella avise.

Logs: la historia clínica

Un **log** es una línea de texto con fecha que un programa escribe cuando pasa algo. Los logs **cuentan lo que pasó**; las métricas, **cuánto**.

El log del broker fue la herramienta clave de casi todos los [casos prácticos](#) de las placas:

```
New client connected from 172.21.0.1 as esp32-equipo-04-enchufe (u'equipo_04').
Client esp32-equipo-03 disconnected, not authorised.
```

Dónde se miran:

Qué	Cómo
un contenedor	<code>docker logs <nombre></code> (lo corre el operador)
tus contenedores	<code>labctl logs</code> (lo corrés vos)
un servicio del servidor	<code>journalctl -u <servicio></code>
todos juntos, con búsqueda	Loki en Grafana, prendiéndolo con <code>make logs-on</code>

Loki está **apagado por defecto** a propósito: guardar todos los logs todo el tiempo cuesta RAM y disco que necesita el aula. "Un observador que consume más que lo observado no sirve."

PromQL: preguntarle a los datos

PromQL (*Prometheus Query Language*) es el idioma para hacer **preguntas** a las series temporales. Lo usan Prometheus y VictoriaMetrics. Con tres ideas alcanza para empezar:

Idea	Ejemplo	Se lee
Elegir una serie	<code>mqtt_valor</code>	"todos los valores"
Filtrar con etiquetas <code>{}</code>	<code>mqtt_valor{dispositivo="enchufe"}</code>	"los del enchufe"
Resumir en el tiempo <code>[...]</code>	<code>avg_over_time(mqtt_valor[1h])</code>	"el promedio de la última hora"
Agrupar	<code>sum by (equipo) (...)</code>	"sumado por equipo"

Un ejemplo real del tablero de consumo por equipo: los contenedores se llaman `equipo-04-nodered-1`, `equipo-04-web-1` ... La consulta saca el `equipo-04` del nombre y suma la RAM de todos los suyos:

```
sum by (team) (
  label_replace(container_memory_usage_bytes{name=~"equipo-.*"},
    "team", "$1", "name", "(equipo-[0-9]+)-.*")
)
```

No hace falta escribir esto de memoria: el recetario del [cap. 13](#) tiene las consultas más usadas para copiar.

Los tableros (dashboards)

Un **dashboard** es una pantalla con varios gráficos (**paneles**), cada uno haciendo una consulta. Hay dos Grafana, para dos públicos:

Grafana	Quién lo ve	Tableros
De operación (grafana.)	solo operadores	<i>Homelab Overview</i> (el servidor), <i>Classroom Overview / Team Detail / Capacity</i> (consumo de cada equipo), <i>Pañol IoT</i> (auditoría y logs)
Del aula (grafana-aula.)	operadores y alumnos	<i>Aula — todos los equipos</i> (operador) y un tablero por equipo con sus placas

Cómo leer el tablero de un equipo: [capítulo 13](#).

Alertas: lo que todavía falta

Hoy el sistema **muestra**, pero no **avisa**. Una **alerta** es una regla que mira una métrica y manda un aviso si pasa algo ("hace 15 minutos que la placa del equipo-04 no publica", "el disco está al 90%"). Es lo que convierte un tablero que alguien tiene que mirar en un sistema que **te busca a vos** cuando algo anda mal. Está anotado en la hoja de ruta del [capítulo 15](#).



Ideas clave

- **Observar** = entender lo de adentro mirando desde afuera: **métricas** (cuánto), **logs** (qué pasó), **trazas** (por dónde pasó).
- **Pull**: Prometheus va a buscar a los exporters. **Push**: las placas avisan por MQTT.
- **PromQL**: elegir, filtrar `{}`, resumir `[...]`, agrupar `by`.
- Dos Grafana: el de **operación** (solo operadores) y el **del aula** (un tablero por equipo).
- Falta: **alertas**.



Errores comunes

- Mirar solo las métricas y no los **logs**: el número te dice que algo falla, el log te dice **por qué**.
- Guardar **todo** "por las dudas": cuesta recursos (por eso Loki va a pedido).
- Un tablero que nadie mira no observa nada: hacen falta **alertas**.

? Preguntas de repaso

1. ¿Qué diferencia hay entre una métrica y un log? Usá la analogía del médico.
2. ¿Por qué las placas **avisan** (push) en vez de que alguien las vaya a buscar (pull)?
3. ¿Qué hace `avg_over_time(mqtt_valor[1h])`?
4. ¿Por qué hay dos Grafana distintos?

Ejercicios

1. Abrí el tablero de tu equipo y explicá qué responde cada panel.
2. Escribí una consulta para ver **solo** la magnitud `estado` de tu dispositivo.
3. Proponé **una** alerta útil para tu proyecto: qué mira y cuándo avisa.

7. Seguridad (defensa en profundidad)

 **Objetivo:** entender cómo se protege el sistema **en varias capas**, de modo que si una falla, otra sigue cubriendo, y qué protege a cada cosa que se ve desde afuera (incluido el broker de las placas).

 **Prerequisitos:** [cap. 2 \(Fundamentos\)](#).

 **Conceptos nuevos:** defensa en profundidad, superficie de ataque, firewall, fuerza bruta, clave pública, mínimo privilegio, cifrado, secreto, endurecimiento (*hardening*).

Empecemos por lo cercano: un boliche

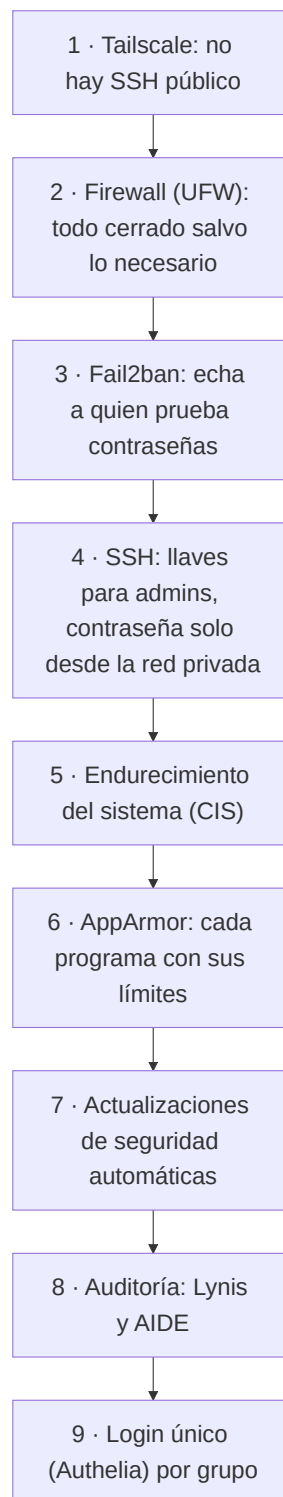
Pensá en cómo cuida un boliche a la gente de adentro:

1. Hay **una sola puerta** de entrada (no se entra por la ventana).
2. En la puerta, un **patovica** mira la lista: si no estás, no pasás.
3. Si alguien intenta colarse varias veces, **lo echan** por un rato.
4. Adentro, la **pulsera** dice a qué sector podés ir (pista o VIP).
5. Hay **cámaras** que graban lo que pasa.
6. Si algo falla, hay **salidas de emergencia** y un plan.

Ninguna de esas medidas sola alcanza: un patovica distraído no sirve si no hay pulseras, y las pulseras no sirven si se puede entrar por la ventana. **Juntas**, sí. Eso se llama **defensa en profundidad**: varias capas **independientes**, para que romper una no alcance.

Este capítulo mira la seguridad desde el servidor. Si querés la vista **del alumno** (qué puertas cruzás vos, con qué llave y por qué son distintas) → [Red y accesos: ¿por qué me autentico tantas veces?](#).

Las capas de este sistema



Las vemos con lo que hay **realmente configurado**.

1 · Tailscale: la puerta no da a la calle

El acceso de administración (**SSH**, la terminal remota) **no está en internet**. Solo se llega desde la red de la casa o desde la red privada de Tailscale. Así, los miles de robots que prueban contraseñas en internet **ni siquiera encuentran la puerta**. A todo lo que se

puede atacar desde afuera se le llama **superficie de ataque**: esta capa la achica al mínimo.

2 · Firewall: el patovica de la red

Un **firewall** ("cortafuegos") decide qué conexiones dejar entrar. En el servidor se llama **UFW** (*Uncomplicated Firewall*). La regla base: **todo lo que entra está prohibido**, salvo lo que se habilita a propósito:

Permitido	Desde	Para qué
SSH (22)	red de la casa y Tailscale	administrar
Web (80, 443)	cualquiera	Caddy (si el router lo deja pasar)
Todo	la red de Tailscale	es nuestra red privada

La trampa de Docker: Docker escribe sus propias reglas de red y puede "saltarse" el firewall. Por eso se instala **ufw-docker**, que hace que el firewall también mande sobre los contenedores, y por eso los servicios se publican en 127.0.0.1 (solo adentro del servidor).

3 · Fail2ban: el que echa a los insistentes

Fail2ban lee los registros y, si una IP **falla 4 veces en 10 minutos**, la **bloquea 1 hora**. Frena la **fuerza bruta** (probar contraseñas una tras otra hasta acertar). No bloquea a la red de la casa ni a Tailscale, para no dejar afuera a los propios.

4 · SSH: quién entra y cómo

- **Administradores:** entran **solo con llave** (*clave pública*). Es un par de archivos: uno **público** que se deja en el servidor (como un candado) y uno **privado** que queda en tu compu (la única llave que lo abre). Sin la llave privada no se entra, aunque se sepa la contraseña.
- **Alumnos:** entran con **contraseña**, pero **solo** desde la red de la casa o Tailscale, y solo pueden abrir túneles **locales** (-L, ver la [guía del equipo](#)).
- Para todos: `root` no puede entrar, máximo **3 intentos** por conexión, y cifrados modernos.

5 · Endurecimiento (*hardening*)

Endurecer es sacar o ajustar todo lo que un sistema trae "por si acaso" y no hace falta. Se sigue la guía **CIS** (*Center for Internet Security*, una organización que publica recetas de seguridad), nivel 1:

- ajustes del kernel para la red (no aceptar redirecciones raras, etc.),
- contraseñas fuertes obligatorias,

- archivos sensibles con permisos estrictos,
- sin volcados de memoria (*core dumps*) de programas privilegiados,
- módulos del kernel que no se usan, bloqueados,
- un cartel legal al entrar.

6 · AppArmor: cada programa en su carril

AppArmor le pone a cada programa una **lista de lo que puede tocar**. Si un programa es engañado, igual no puede salirse de su lista. Está en modo **enforce** (obligatorio), no solo "avisar".

7 · Actualizaciones automáticas

Las **actualizaciones de seguridad** se instalan solas, y si hace falta reiniciar, el servidor lo hace a las **04:30**, cuando nadie lo usa.

8 · Auditoría

- **Lynis** revisa la configuración **una vez por semana** y da un puntaje.
- **AIDE** guarda una "huella" de los archivos del sistema y avisa **todos los días** si alguno cambió sin permiso (como un precinto).

9 · Login único por grupo (Authelia)

Para las páginas web, **Authelia** pide usuario y contraseña **una vez** y decide según tu **grupo**:

Página	operators	students
Grafana de operación (incluye la auditoría del pañol)	✓	✗
Grafana del aula	✓	✓ (solo el folder de su equipo)
Tablero del pañol	✓	✗
Prometheus, cAdvisor	✓	✗

Lo que se ve desde internet, y qué lo protege

Con **Tailscale Funnel**, dos cosas quedan alcanzables desde internet. Cada una tiene su propia defensa en capas:

Qué	Qué lo protege
Broker de las placas (:10000)	cifrado (TLS) · usuario y clave por equipo · reglas (ACL) que encierran a cada equipo en sus topics · sin anónimos
API del pañol (:8443)	cifrado · token secreto en cada pedido · límite de pedidos por minuto · solo las rutas que usa un nodo

Mínimo privilegio: cada uno con lo justo

Mínimo privilegio es darle a cada persona o programa **solo** el permiso que necesita para su trabajo, y nada más. Ejemplos de este sistema:

- Los alumnos **no** tienen `sudo` ni acceso a Docker: piden las cosas por `labctl`.
- **Telegraf** puede **leer** los topics de los equipos, pero **no escribir**: si alguien lo engañara, no podría prender ningún LED.
- **Grafana del aula** confía en el usuario que le pasa el login **solo** si el pedido viene de Caddy, por una red que no comparte con nadie más.
- El tablero del pañol en Grafana lee la base con un usuario que **solo puede leer**.

Secretos: dónde viven las claves

Un **secreto** es cualquier dato que da acceso (contraseñas, tokens, llaves). Regla: **nunca** en un archivo común del repositorio.

Dónde	Qué
Ansible Vault (cifrado)	contraseñas del aula, secretos de Authelia, base de datos
<code>/etc/classroom/secrets/</code> (solo <code>root</code>)	claves MQTT de cada equipo, claves de servicio
<code>/etc/panol/secrets/</code> (solo <code>root</code>)	claves del pañol
<code>.shared-services.env</code> de cada equipo	lo que el equipo necesita, legible solo por su equipo

Lección real: durante la puesta en marcha, algunas claves MQTT se pasaron por chat y en archivos comprimidos. Funciona, pero una clave que viajó por canales comunes conviene **rotarla** (cambiarla) cuando el sistema se estabiliza.

Lo que todavía falta (honestidad)

- **Copias de seguridad:** están diseñadas (cifradas, con Borg) pero **no corren**: falta el disco. Sin copias, un disco roto es perder todo. Es lo más urgente del [capítulo 15](#).
- **El firewall quedó con la red vieja de la casa:** hoy se entra solo por Tailscale ([caso 9](#)).
- **Alertas de seguridad:** hoy Lynis y AIDE anotan, pero nadie recibe un aviso.



Ideas clave

- **Defensa en profundidad:** muchas capas independientes; romper una no alcanza.

- Achicar la **superficie de ataque**: lo que no está expuesto no se puede atacar.
- **Llave** para administradores; **contraseña** para alumnos, solo desde la red privada.
- **Mínimo privilegio**: cada uno con lo justo.
- Los **secretos** nunca en archivos comunes del repositorio.

Errores comunes

- Publicar el **editor de Node-RED** en internet: quien lo abre puede ejecutar cualquier cosa en el servidor. Nunca se publica.
- Mandar una contraseña por chat y no cambiarla después.
- Pensar que "como tengo firewall, estoy seguro": es **una** capa.

Preguntas de repaso

1. Explicá la defensa en profundidad con la analogía del boliche.
2. ¿Qué diferencia hay entre entrar por SSH con llave y con contraseña?
3. ¿Qué protege al broker de las placas, que está en internet?
4. Da dos ejemplos de mínimo privilegio en este sistema.

Ejercicios

1. Elegí una capa y explicá qué pasaría si **no** existiera.
2. Revisá tu proyecto: ¿dónde guardás tus claves (WiFi, MQTT)? ¿Hay alguna en un lugar público (por ejemplo, un repositorio)?

8. El pipeline (Git, revisión y pruebas automáticas)

 **Objetivo:** entender cómo se cuida la calidad del sistema **antes** de que un cambio llegue al servidor: cómo se guardan las versiones, cómo se revisan los cambios y qué pruebas corren solas.

 **Prerequisitos:** [cap. 4 \(Ansible e IaC\)](#).

 **Conceptos nuevos:** Git, commit, rama, merge, *pull request*, repositorio remoto, CI, test, lint, *shift-left*, staging.

Empecemos por lo cercano: entregar un trabajo práctico

Cuando entregás un trabajo práctico importante, no lo mandás recién escrito:

1. guardás **versiones** ("tp_v1", "tp_v2_corregido") por si tenés que volver atrás,
2. trabajás en un **borrador** sin tocar la versión buena,
3. le pedís a un compañero que lo **lea** antes de entregar,
4. pasás el **corrector ortográfico**,
5. y recién ahí lo **entregás**.

El *pipeline* (la "cañería" por la que pasa cada cambio) es exactamente eso, pero automático y para código.

Git: las versiones, bien hechas

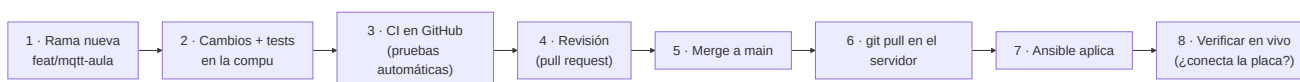
Git es el programa que guarda la **historia** de un proyecto: cada cambio, quién lo hizo, cuándo y por qué. En vez de "tp_v2_final_ahora_sí", guarda cada versión con un mensaje.

Palabra	Qué es	En el TP sería...
Repositorio	la carpeta del proyecto con toda su historia	la carpeta del TP con todas sus versiones
Commit	una "foto" de los cambios, con un mensaje que explica por qué	guardar "tp_v3: agregué la conclusión"
Rama (<i>branch</i>)	una línea de trabajo aparte, para no tocar lo que funciona	un borrador
Merge	juntar una rama con la principal	pasar el borrador corregido a la versión final
Remoto (GitHub)	una copia del repositorio en internet	el aula virtual donde se entrega
Push / pull	subir tus cambios al remoto / bajar los de otros	subir / descargar

La rama principal se llama `main`: es la versión "buena", la que va al servidor.

El recorrido de un cambio

Un ejemplo real: cuando se armó el broker del aula para las placas.



Un **pull request** (PR, "pedido de incorporación") es pedir formalmente "por favor, sumen mi rama a `main`": ahí se ven todos los cambios, corren las pruebas y alguien los revisa.

CI: las pruebas que corren solas

CI (*Continuous Integration*, integración continua) es un robot que, **cada vez** que se sube un cambio, corre todas las pruebas. En este proyecto el robot es **GitHub Actions**, y su receta está en `.github/workflows/ci.yml`. Hace cinco trabajos:

Trabajo	Qué revisa	En el TP sería...
Tests unitarios	la política de Compose del aula, los tableros, y que el broker del pañol, el del aula y la capa de Grafana estén bien armados (por ejemplo: "el broker no acepta conexiones sin clave", "un equipo no puede escribir en otro", "Grafana no confía en cualquiera")	verificar cada cuenta del TP
Búsqueda de secretos (gitleaks)	que nadie haya subido una contraseña por error, en toda la historia	revisar que no quedó tu DNI en el archivo
Lint (yamllint, ansible-lint)	el estilo y las buenas prácticas	el corrector ortográfico
Chequeo de sintaxis	que el playbook se pueda leer, en producción y en staging	que el archivo abra
Molecule	prueba roles de verdad , en un Ubuntu 24.04 armado en un contenedor descartable	ensayar la presentación

Algunas palabras:

- Un **test** (prueba) es un pequeño programa que comprueba que otro hace lo que debe. Si falla, avisa **antes** de que el error llegue al servidor.
- **Lint** viene de la pelusa que se le saca a la ropa: es revisar el código buscando "pelusas" (errores de estilo, cosas sospechosas) sin ejecutarlo.

Caso real: en octubre de 2026, al revisar la documentación, se encontró que los tests nuevos del broker del aula y de Grafana **no estaban en el CI**: existían, pero el robot no los corría. Se agregaron. Una prueba que no corre sola es una prueba que, tarde o temprano, nadie corre.

Pruebas sobre el servidor de verdad

Algunas cosas solo se pueden comprobar **en el servidor armado**:

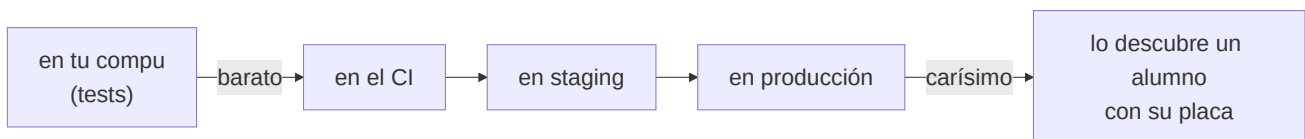
Comando	Qué hace
<code>make verify</code>	Ansible revisa la postura de seguridad (SSH, firewall, AppArmor)
<code>make test</code>	testinfra : pruebas que miran el servidor real (permisos de carpetas, servicios corriendo, contenedores sanos)
<code>make idempotence</code>	aplica todo dos veces y falla si la segunda cambió algo (cap. 4)

Staging: ensayar antes del estreno

Staging es un servidor de **ensayo**, con su propio inventario (`inventories/staging`), donde se prueba un cambio grande antes de llevarlo al servidor real (**producción**). Es el ensayo general antes del acto.

Shift-left: encontrar los errores temprano

Shift-left ("correr hacia la izquierda") es la idea de buscar los errores **lo antes posible** en el recorrido de arriba: cuanto más a la izquierda se encuentra un error, más barato es arreglarlo.



La regla del proyecto: **nada se declara "funcionando" sin una prueba que lo demuestre.**



Ideas clave

- **Git** guarda la historia; las **ramas** permiten trabajar sin romper `main`.
- Un cambio recorre: rama → pruebas → revisión → merge → servidor → verificación.
- El **CI** corre las pruebas solo, en cada cambio: tests, secretos, lint, sintaxis, Molecule.
- **Shift-left**: cuanto antes se encuentra un error, más barato.



Errores comunes

- Trabajar directo en `main` "porque es un cambio chiquito".
- Escribir un test y no sumarlo al CI.
- Mensajes de commit que dicen **qué** ("cambios") en vez de **por qué** ("el broker del pañol quedó sin puerto al mudarse la red").

? Preguntas de repaso

1. ¿Qué es un commit y qué tiene que decir su mensaje?
2. ¿Para qué sirve trabajar en una rama?
3. Nombrá tres cosas que revisa el CI de este proyecto.
4. ¿Qué es *shift-left* y por qué conviene?

Ejercicios

1. Abrió `.github/workflows/ci.yml` y ubicó los cinco trabajos de la tabla.
2. Pensó **un test** para tu proyecto (por ejemplo: "si llega `ON`, el relé se prende"): qué pondrías a prueba y qué resultado esperarás.
3. Escribí un buen mensaje de commit para el último cambio que hiciste en tu placa.

9. La plataforma de aula

 **Objetivo:** entender cómo varios equipos de alumnos despliegan sus propias aplicaciones en un **único** servidor, aislados entre sí y **sin** acceso a Docker, sudo ni al socket.

 **Prerequisitos:** [cap. 3 \(Docker\)](#), [cap. 2 \(Linux\)](#).

 **Conceptos nuevos:** aislamiento multiusuario, intermediario (`labctld`), socket, política de Compose, cuota de disco, slice de recursos, auditoría.

Empecemos por lo cercano: un edificio de departamentos

Pensá en un edificio donde vive una familia por departamento:

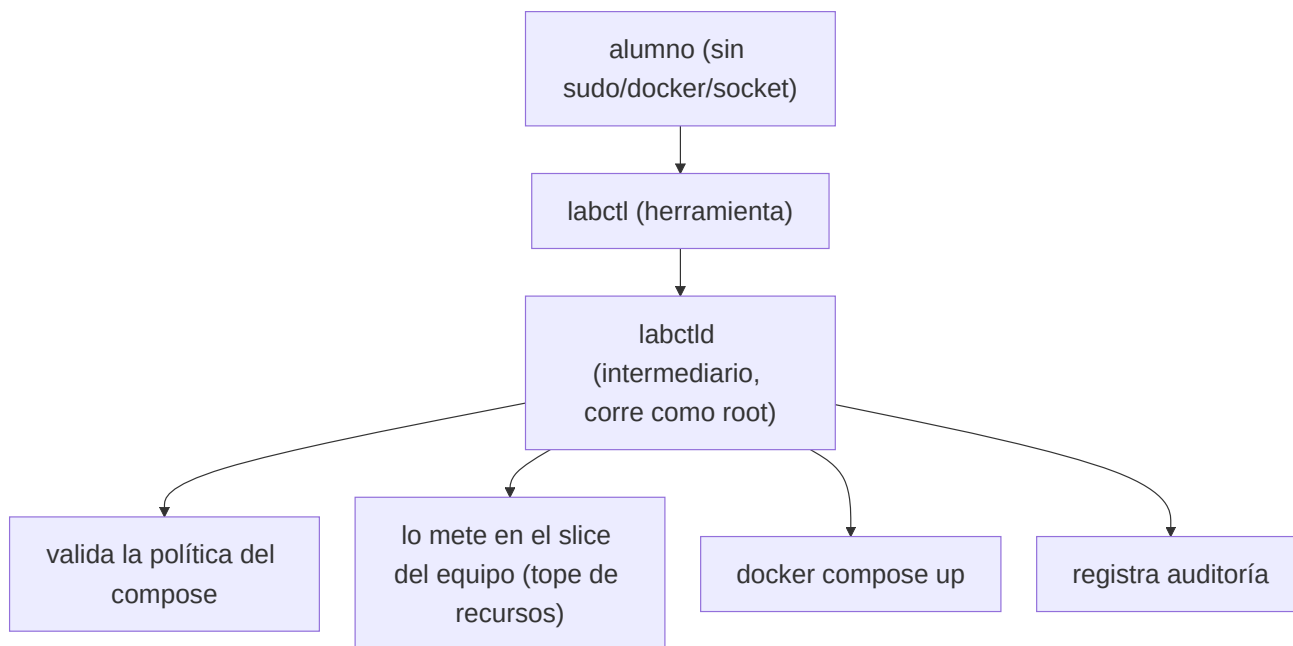
En el edificio	En la plataforma	
cada departamento , con su llave	la carpeta de cada equipo (<code>/srv/classroom/equipo-NN</code>)	solo entra quien tiene la llave
el medidor de luz con corte de cada depto	el slice de recursos de cada equipo	si un depto se pasa de consumo, se le corta a él, no a todo el edificio
la baulera de tamaño fijo	la cuota de disco (20 GB por equipo)	no podés guardar más de lo que entra
el encargado del edificio	<code>labctld</code>	los vecinos no tocan la sala de máquinas: le piden al encargado
el reglamento del consorcio	la política de Compose	el encargado revisa que el pedido cumpla el reglamento
el libro de novedades	la auditoría	queda anotado quién pidió qué y cuándo
los servicios comunes (gas, agua)	los servicios compartidos (base de datos, broker MQTT)	llegan a cada depto por su propio caño

El problema

Querés que 5 equipos desplieguen stacks de Docker Compose reales (por ejemplo un stack IoT con Mosquitto + Node-RED + n8n), en **una sola máquina modesta**, sin que:

- puedan **romper** el servidor,
- se **vean entre sí**,
- ni tengan **acceso directo a Docker** (que equivale a ser root).

La solución: tres barreras de aislamiento



1. **Linux:** cada alumno es un usuario común, en el grupo de su equipo (`grp-equipo-NN`). La carpeta del equipo (`/srv/classroom/equipo-NN`) es `2770` con **setgid**: solo su equipo entra. Está respaldada por un **filesystem loopback** (un "disco virtual" guardado adentro de un archivo, de tamaño fijo) que le pone una **cuota de disco dura**: no puede pasarse de 20 GB.
2. **Recursos (cgroups):** cada equipo tiene un **slice de systemd** (`classroom-equipo-NN.slice`): una "caja" de recursos con topes de RAM, CPU y cantidad de procesos (los **cgroups** del [cap. 3](#), aplicados a todo el equipo junto). El broker corre los contenedores **dentro** de ese slice, así el **kernel** garantiza el tope pase lo que pase.
3. **La herramienta labctl** : los alumnos **no** tocan Docker. Usan `labctl`, que le habla a un daemon (`labctld`, un programa de fondo: ver [cap. 2](#)) por un **socket** (un "enchufe" para que dos programas de la misma máquina se hablen; es un archivo especial, `/run/labctld.sock`). A `labctld` se le dice **intermediario** o *broker*, pero **no** tiene nada que ver con el broker MQTT de las placas ([cap. 11](#)): es el "encargado del edificio". El daemon: - identifica **quién** llama (por el **UID**, el número de usuario que le pasa el propio kernel: no se puede falsificar), - **enjaula** todo a la carpeta del equipo, - **valida** el compose contra una política (rechaza `privileged`, el socket de Docker, imágenes `latest`, puertos públicos, falta de límites, más de 5 servicios, etc.), - inyecta el **slice** del equipo, - **audita** cada acción.

Comandos del alumno

```

cd /srv/classroom/equipo-01
labctl validate    # revisa el compose contra la política
labctl up         # despliega
labctl ps | logs | usage | status | restart | down

```

Guía práctica: el paso a paso para trabajar en tu proyecto (conectarte por túnel, rutas, ejemplos de `compose.yml` verificados, base compartida y cómo salir a la web) está en la [Guía práctica del equipo](#).

Servicios compartidos

Los alumnos **no** levantan su propia base de datos ni su propio broker: el laboratorio les da servicios **compartidos**, con **credenciales propias por equipo** (en `.shared-services.env`, un archivo de solo lectura en la carpeta del equipo):

Servicio	Qué es	Para qué lo usa un equipo
PostgreSQL	base de datos	guardar datos de su aplicación
Redis	memoria rápida de "clave → valor"	caché, colas chicas
Mailpit	un correo de prueba (atrapa los mails, no los manda)	probar envíos de mail
<code>mqtt-aula</code>	el broker MQTT del aula	que sus placas manden y reciban mensajes (ver cap. 11)
Grafana del aula	tableros	ver los datos de sus placas (ver cap. 13 ; en puesta en marcha)

El broker de la plataforma (`labctl`) conecta esos servicios a la **red privada** de cada equipo cuando hace `labctl up`, así los alcanzan por nombre (`postgres` , `mqtt-aula`) **sin verse entre sí**.

Publicación

Los alumnos **nunca** exponen puertos públicos. Si un proyecto tiene que verse desde afuera, lo publica el **operador** con un registro declarativo (`student_exposures`) → un vhost de Caddy.



Ideas clave

- **Tres barreras:** permisos Linux (2770+setgid+loopback), cgroups (slice por equipo) y el broker `labctl` (validación + auditoría).
- Los alumnos **nunca** tienen Docker/sudo/socket.
- Servicios **compartidos** (base de datos, broker MQTT, Grafana), con aislamiento por red y credenciales por equipo.

Errores comunes

- Querer usar `privileged`, el socket de Docker o volúmenes con nombre → la política los rechaza (y te dice por qué).
- Publicar en `0.0.0.0` → rechazado; solo `127.0.0.1`.
- Guardar datos fuera de la carpeta del equipo → no cuenta contra la cuota y se rechaza si es fuera del proyecto.

Preguntas de repaso

1. Nombrá las tres barreras de aislamiento y qué garantiza cada una.
2. ¿Por qué los alumnos usan `labctl` en vez de `docker` directamente?
3. ¿Cómo llega un equipo a su base de datos sin ver a los otros equipos?

Ejercicios

1. Escribí un `compose.yml` que la política **rechace** y explicá por qué.
 2. Escribí uno que la política **acepte** y que use la Postgres compartida (`env_file: .shared-services.env`).
-

Ahora deberías entender

- Por qué trabajás con `labctl` y no con Docker directo, y qué te protege a vos y a los demás.
- Qué servicios te da el aula (base de datos, broker MQTT, Grafana) y cómo llegan a tu proyecto.

Seguí por acá:

- Para **trabajar en tu proyecto** paso a paso → [Guía práctica del equipo](#).
- Para entender **cómo llegás** a tus servicios (túnel, contraseñas) → [Red y accesos](#).

Guía práctica del equipo

 **Objetivo:** poder trabajar de punta a punta en el proyecto de tu equipo: conectarte, saber **dónde vive cada archivo**, escribir un `compose.yml` que pase la política, usar la base de datos compartida, y dejar un servicio listo para **salir a la web**.

 **Prerequisitos:** [cap. 3 \(Docker\)](#), [cap. 9 \(Plataforma de aula\)](#).

 **Conceptos nuevos:** túnel SSH, directorio del equipo, `.shared-services.env`, publicación (`student_exposures`), terminación TLS en el borde.

Todos los ejemplos de `compose.yml` de esta guía están **verificados contra la política del aula** (`labctl validate`, el comando que revisa tu proyecto). Ejemplos completos y probados: [ejemplos/nodered-mqtt/](#) y [ejemplos/nginx-ui/](#) (tu propia página web).

El modelo en una frase: trabajás dentro de la carpeta de tu equipo con `labctl` (nunca tocás Docker ni `sudo`). Tus servicios escuchan **solo en loopback** (`127.0.0.1`) y los alcanzáis por un **túnel SSH**. Cuando algo sale a la web, **lo publica el operador** con HTTPS automático — vos no abrís puertos públicos.

1. Conectarte

Para trabajar en tu proyecto necesitás llegar al servidor del aula desde tu compu. Son dos pasos: **entrar a la red privada** y **abrir un túnel**.

Qué te da el profe: tu **usuario** (tu nombre, ej. `jorge`), tu **contraseña del aula** y una **clave de Tailscale** (empieza con `tskey-auth-`).

1.1 Tailscale: la red privada del aula

Tailscale arma una red privada entre tu compu y el servidor, como si estuvieran enchufados al mismo router aunque estés en tu casa. (**Por qué** la usamos y qué resuelve → [Red y accesos](#).) Se hace **una sola vez**. Instalalo desde <https://tailscale.com/download> y después:

Windows (en "Símbolo del sistema" / CMD, un comando por vez):

```
"C:\Program Files\Tailscale\tailscale.exe" logout
"C:\Program Files\Tailscale\tailscale.exe" up --auth-key=TU-CLAVE
```

Linux (en la Terminal, un comando por vez):

```
sudo tailscale logout
sudo tailscale up --auth-key=TU-CLAVE
```

El `logout` va primero por una razón: si alguna vez entraste a Tailscale con tu cuenta de Google, te creó **una red tuya aparte**, y desde ahí no ves el servidor. Para comprobar: `tailscale status` tiene que listar `homelab-01`.

1.2 El túnel SSH: traer tus servicios a tu compu

SSH (*Secure Shell*) es la forma segura de conectarte a otra computadora y escribirle comandos. Con la opción `-L` además abre un **túnel**: un pasillo privado que hace que un servicio del servidor aparezca en **tu** compu, en `localhost` (que quiere decir "esta misma computadora").

Tus servicios escuchan **solo adentro del servidor** (en `127.0.0.1`, que es `localhost` del servidor). Por eso nadie de afuera los ve, y vos llegás por el túnel. Cada equipo tiene **sus propios números de puerto** (un puerto es como el número de departamento de un servicio dentro del servidor; no se pueden repetir):

Equipo	Node-RED en el servidor	Tu página (nginx)	Comando del túnel
equipo-01	1880	—	<code>ssh -L 1880:localhost:1880 TU-USUARIO@100.110.123.76</code>
equipo-03	1882	8083	<code>ssh -L 1880:localhost:1882 -L 8080:localhost:8083 TU-USUARIO@100.110.123.76</code>
equipo-04	1884	8084	<code>ssh -L 1880:localhost:1884 -L 8080:localhost:8084 TU-USUARIO@100.110.123.76</code>
equipo-02 / 05	se asignan al armar su stack		pedíselo al profe

Cómo se lee `-L 1880:localhost:1884`: "el **1880 de mi compu** lleva al **1884 del servidor**". Así, en tu navegador siempre usás los mismos números:

- Node-RED → `http://localhost:1880`
- Tu página → `http://localhost:8080`

Al conectarte: la primera vez escribí `yes`; después tu **contraseña del aula** (no se ve mientras la tipeás, es normal). **Dejá esa ventana abierta**: si la cerrás, se corta el túnel.

Por qué así: el SSH de alumnos solo se acepta desde la red privada (nunca desde internet), y tus servicios no están publicados. El túnel es la forma de usarlos sin exponer nada. Ver [cap. 7 \(Seguridad\)](#).

2. Dónde vive cada cosa

Todo tu proyecto vive en **una sola carpeta** (`/srv/classroom/<tu-equipo>/`), con cuota de disco propia:

```

/srv/classroom/equipo-04/
├─ compose.yml          # LA definición de tu stack (lo editás vos)
├─ data/               # acá PERSISTEN los datos (cuentan contra tu cuota)
│   └─ nodered/        # flows y configuración de Node-RED
├─ web/               # tu página (si tu equipo tiene nginx)
│   └─ index.html
├─ nginx/
│   └─ default.conf    # config de nginx (sirve web/ y el puente a Node-RED)
└─ .shared-services.env # credenciales de base de datos y MQTT (SOLO LECTURA)

```

Por qué acá: la carpeta tiene permisos que dejan entrar **solo a tu equipo** (técnicamente, permisos `2770`). Cada número es un permiso: el `2` es *setgid*, que hace que todo archivo nuevo quede también del equipo; el primer `7` deja al dueño leer, escribir y entrar; el segundo `7`, lo mismo al grupo de tu equipo; y el `0` final dice que **nadie más** puede ni mirar). Además vive en un disco propio con **tope de 20 GB** (tu **cuota**). Nada afuera de esta carpeta cuenta contra tu cuota, y la política rechaza usar carpetas de afuera. **Guardá todo acá.**

- **compose.yml** — qué contenedores levantás y cómo. Es el archivo central.
- **data/...** — lo que quieras que sobreviva a un reinicio va acá. En el `compose.yml` se "conecta" una carpeta tuya con una carpeta del contenedor; eso se llama *bind mount* (ej. `./data/nodered:/data`: lo que Node-RED guarda en su `/data` queda en tu `data/nodered`).
- **.shared-services.env** — lo genera el operador; trae las credenciales de tu Postgres/Redis/MQTT. Lo **leés**, no lo editás:

```

PGHOST=postgres PGUSER=equipo_01 PGPASSWORD=... PGDATABASE=db_equipo_01
REDIS_URL=redis://redis:6379/2
MQTT_HOST=mqtt-aula MQTT_USER=equipo_01 MQTT_PASSWORD=...
MQTT_TOPIC_PREFIX=equipo-01/
MQTT_DEVICE_HOST=homelab-01.tail4eda13.ts.net MQTT_DEVICE_PORT=10000

```

3. Configuración básica (ejemplos verificados)

La política exige: imagen con **versión fija** (no `latest`), **límites** de CPU/RAM/procesos, **rotación de logs**, `restart`, puertos **solo en 127.0.0.1**, sin volúmenes con nombre (usá `bind mounts` de `./data`), y máximo 5 servicios.

3.1 `compose.yml` mínimo (pasa `labctl validate`)

```
---
services:
  web:
    image: nginxdemos/hello:plain-text # versión fija, nunca :latest
    restart: unless-stopped
    ports:
      - "127.0.0.1:8080:80" # SOLO loopback (nunca 0.0.0.0)
    volumes:
      - ./data/web:/data # persistencia dentro de tu cuota
    logging:
      driver: json-file
      options: { max-size: "10m", max-file: "3" }
    deploy:
      resources:
        limits: { cpus: "0.50", memory: 256M, pids: 200 }
```

3.2 Usar la base de datos compartida (recomendado)

No levantes tu propia Postgres. El aula te da una base propia, aislada y respaldada. Tu servicio la usa cargando `.shared-services.env`:

```
---
services:
  nodered:
    image: nodered/node-red:4.0
    restart: unless-stopped
    ports:
      - "127.0.0.1:1880:1880"
    env_file:
      - .shared-services.env # trae PGHOST/PGUSER/PGPASSWORD/PGDATABASE
    volumes:
      - ./data/nodered:/data
    logging:
      driver: json-file
      options: { max-size: "10m", max-file: "3" }
    deploy:
      resources:
        limits: { cpus: "0.50", memory: 256M, pids: 200 }
```

En Node-RED: instalá el nodo `node-red-contrib-postgresql` (*Manage Palette*) y en el config-node usá `${PGHOST}` `${PGUSER}` `${PGPASSWORD}` `${PGDATABASE}`. El host `postgres` resuelve dentro de tu red porque `labctl up` lo conecta.

Permisos de data/ : cada imagen corre con su propio uid (Node-RED 1000 , Mosquitto 1883). Si un contenedor queda en `Restarting` con `EACCES` sobre `/data` , la carpeta del

host tiene que pertenecer a ese uid → avisá al operador para el `chown` (ver [cap. 10, Caso 3](#)).

3.3 MQTT: usá el broker del aula (`mqtt-aula`)

MQTT es el idioma con el que las placas se mandan mensajes, y el **broker** es el programa que los reparte (lo explica, desde cero, el [capítulo 11](#)). El aula tiene **un broker para todos**: `mqtt-aula`. Es el que tenés que usar, por tres razones:

1. **Tu ESP32 llega a él desde cualquier red** (tu casa, el colegio), por internet y cifrado. Un broker dentro de tu proyecto solo se alcanza por túnel SSH, y una placa no puede abrir un túnel.
2. **Lo que publicás se guarda y aparece en Grafana** solo ([capítulo 12](#); el Grafana del aula se está poniendo en marcha).
3. **Es cerrado**: entrás con el usuario y la clave de tu equipo, y **solo podés usar topics que empiecen con el nombre de tu equipo** (`equipo-04/...`). Un mensaje con otro nombre se descarta sin aviso.

Desde	Host (dirección)	Puerto	¿Cifrado (TLS)?	Usuario / clave
Tu Node-RED (en el servidor)	<code>mqtt-aula</code>	1883	no hace falta (viaja adentro del servidor)	<code>MQTT_USER</code> / <code>MQTT_PASSWORD</code>
Tu ESP32 (cualquier red)	<code>homelab-01.tail4eda13.ts.net</code>	10000	sí	los mismos

Los datos están en tu `.shared-services.env`. Para verlos, entrá por SSH y corré (cambiá `04` por tu número):

```
grep MQTT_ /srv/classroom/equipo-04/.shared-services.env
```

Ojo: el **usuario** va con **guion bajo** (`equipo_04`) y los **topics** con **guion** (`equipo-04/...`). Es el error más común.

En Node-RED, el nodo `mqtt-broker` va a `mqtt-aula`, puerto `1883`, con usuario y clave. `labctl up` conecta `mqtt-aula` a la red de tu proyecto, así que el nombre `mqtt-aula` funciona solo.

¿Y un Mosquitto propio dentro de mi proyecto? Se puede (hay un ejemplo en [ejemplos/nodered-mqtt/](#)), pero solo sirve para pruebas entre tus propios servicios: tu placa no lo alcanza y no aparece en Grafana.

3.4 Comandos (`labctl`)

Los corrés con tu usuario, desde cualquier carpeta (el broker te jaula a la de tu equipo):

```

labctl validate  # revisa tu compose contra la política (hacelo SIEMPRE antes de up)
labctl up        # levanta tu stack
labctl ps        # estado de tus contenedores
labctl logs      # ver logs
labctl restart   # reiniciar tras un cambio de config
labctl usage     # cuánto disco/recursos usás
labctl down      # apagar tu stack

```

3.5 Tu dashboard de Node-RED y tu página

Con el túnel abierto ([paso 1.2](#)), en tu navegador:

Qué	Dirección	Para qué
Editor de Node-RED	<code>http://localhost:1880</code>	armar tus flujos (cajitas conectadas)
Dashboard de Node-RED	<code>http://localhost:1880/dashboard</code>	la pantalla con botones e indicadores: lo que se muestra
Tu página (si tu equipo tiene nginx)	<code>http://localhost:8080</code>	tu propia interfaz web

Los equipos que ya tienen un tablero armado (equipo-01, equipo-03 y equipo-04) lo encuentran en `/dashboard` → página **ESP32**. Para que el dashboard funcione, tu Node-RED tiene que tener el nodo `mqtt-broker` conectado a `mqtt-aula` ([paso 3.3](#)).

¿Dashboard de Node-RED o Grafana? El de Node-RED es para **controlar en vivo** (botones, estados); Grafana es para **ver la historia**. Cuál usar → [¿Node-RED, Grafana o ambos?](#).

Para la muestra: el dashboard se abre en **tu** compu, con el túnel. Si lo van a mostrar en otra compu, esa compu necesita Tailscale y el túnel. Checklist completo → [chequeo rápido antes de la muestra](#).

4. SSL / HTTPS — quién lo hace

Vos NO configurás SSL en tu proyecto. El certificado y el HTTPS los maneja **Caddy** (el proxy inverso del server), automáticamente, con certificados gratuitos que se renuevan solos.

Capa	Quién	Qué
Tu servicio	vos	escucha HTTP plano en un puerto loopback (ej. <code>127.0.0.1:8080</code>)
HTTPS público	Caddy (operador)	termina el TLS, pone el candado, renueva el certificado

Tu contenedor habla **HTTP** puertas adentro; el HTTPS lo agrega Caddy en el borde al publicarte. **No metas certificados ni TLS dentro de tu compose** — es redundante y la plataforma no lo usa. Este es el vhost que arma el operador (de referencia, no lo tocás):

```
equipo01.lucasland.duckdns.org {
    import secure_headers          # cabeceras de seguridad
    encode zstd gzip               # compresión
    reverse_proxy equipo-01-web-1:8080 # reenvía a TU contenedor 'web', puerto 8080
}
```

5. Salir a la web (producción)

Los alumnos **nunca** abren puertos públicos (la política lo bloquea). Si tu proyecto tiene que tener una dirección propia con HTTPS, lo **habilita el operador**. Vos preparás el servicio.

 **Hoy, "publicado" quiere decir "visible desde el tailnet", no desde internet.** El router de la casa del servidor no deja entrar conexiones de afuera (ver [Red y accesos](#)). Una dirección como `https://equipo01.lucasland.duckdns.org` funciona para quien tenga **Tailscale** conectado. Para mostrar algo a público general hace falta otra salida, que se decide aparte con el profe.

Qué preparás vos:

1. Un servicio en tu `compose.yml` que sirva **HTTP** en un puerto **loopback** (ej. `web` en `127.0.0.1:8080:80`).
2. Que esté **sano** (`labctl ps` lo muestra `Up / healthy`) y responda en ese puerto.
3. **No publiques cosas peligrosas.** El editor de Node-RED sin login = ejecución de código para cualquiera → no se publica. Publicá una app web real o un dashboard con autenticación.

Qué le pasás al operador: nombre del equipo, nombre del servicio en tu compose, puerto del contenedor y el hostname deseado.

Qué hace el operador (referencia — no lo hacés vos): agrega una entrada a `student_exposures` en `ansible/inventories/production/group_vars/all/classroom.yml` ...

```
student_exposures:
- team: equipo-01
  hostname: equipo01.lucasland.duckdns.org
  service: web
  port: 8080
  enabled: true
```

...y aplica `ansible-playbook site.yml --tags publish -K`. Caddy toma el cambio y tu servicio queda en `https://equipo01.lucasland.duckdns.org` (HTTPS automático, sobre el puerto

estándar 443), alcanzable desde el tailnet. El vhost responde aunque tu servicio esté apagado (verías un 502 hasta que hagas `labctl up`).

Ideas clave

- Trabajás **dentro de tu carpeta** (`/srv/classroom/<equipo>/`) con `labctl`; nunca Docker ni `sudo`.
- Puertos **solo en** `127.0.0.1`; los alcanzáis por **túnel SSH**.
- La **base de datos** va a la **Postgres compartida** (`.shared-services.env`), no a una db propia.
- El **HTTPS lo pone Caddy**, no tu contenedor. La **publicación la decide el operador**.

Errores comunes

`labctl validate` te dice **exactamente** qué rechaza. Ejemplos reales de un compose mal hecho:

```
service 'web': image 'nginx:latest' must be pinned to an explicit tag (not latest)
service 'web': missing CPU limit (deploy.resources.limits.cpus)
service 'web': missing logging (log rotation required)
service 'web': missing restart policy
service 'web': port '8080:80' must bind 127.0.0.1 explicitly (e.g. '127.0.0.1:8080:80')
named volumes are not allowed (datos); bind-mount a path under your project dir instead
```

Otros:

- **Contenedor en Restarting con EACCES en /data** → permisos del bind mount; el operador hace `chown` a la carpeta `data/` (uid del contenedor).
- **Tu app no encuentra postgres** → te faltó `env_file: .shared-services.env` o el stack no está `up`.
- **La URL pública da 502** → tu servicio no está arriba; `labctl up`.

Preguntas de repaso

1. ¿Por qué tus servicios escuchan en `127.0.0.1` y no en `0.0.0.0`?
2. ¿Dónde guardás datos para que sobrevivan a un `labctl down`?
3. ¿Quién pone el HTTPS de tu servicio publicado, y dónde escucha tu contenedor?

Ejercicios

1. Escribí un `compose.yml` con un servicio `web` que pase `labctl validate`.
2. Modificalo para que use la Postgres compartida (`env_file: .shared-services.env`).
3. Listá los 4 datos que le tenés que dar al operador para que publique tu servicio.

Ahora deberías poder

- **Conectarte:** Tailscale + túnel SSH con los puertos de **tu** equipo.
- Saber **dónde vive** cada archivo de tu proyecto y por qué.
- Conectar tu Node-RED y tu placa al **broker del aula**.
- Abrir tu **dashboard** y tu **página** para la muestra.

Seguí por acá:

- Si querés **conectar tu placa** y que aparezca en Grafana → [capítulo 12](#).
- Si algo **no te deja entrar** → [Diagnóstico: no puedo entrar](#).
- Si querés entender **por qué hay túneles y tantas contraseñas** → [Red y accesos](#).

Parte III — IoT aplicado

11. Arquitectura IoT del aula

 **Objetivo:** entender **cómo está pensado** el sistema con el que trabajan sus placas: de qué partes está hecho, cómo viaja un dato desde un sensor hasta una pantalla, y **por qué** se eligió cada pieza. Al terminar vas a poder dibujar tu propio proyecto con estas mismas partes.

 **Prerequisitos:** ninguno. Si querés el contexto del servidor, leé antes el [capítulo 1 \(Visión general\)](#).

 **Conceptos nuevos:** IoT, arquitectura de software, capa, sensor, actuador, microcontrolador, MQTT, broker, topic, mensaje, publicar/suscribir, mensaje retenido, último deseo (*Last Will*), TLS, contrato, serie temporal.

Empecemos por algo que ya usás: un grupo de WhatsApp

Pensá en el grupo de WhatsApp del curso:

- Cuando alguien **manda un mensaje al grupo**, no le escribe a cada persona una por una. Lo manda **al grupo**, y le llega a todos los que están adentro.
- Quien manda **no necesita saber** quién lo va a leer, ni si están conectados en ese momento.
- Hay un **servidor de WhatsApp** en el medio que recibe el mensaje y lo reparte.
- Podés **fijar** un mensaje arriba de todo, para que quien entre después lo vea primero.
- Cada grupo tiene **nombre**, y solo lees los grupos en los que estás.

El sistema de las placas del aula funciona exactamente así. Las placas, en vez de personas, mandan mensajes como "la temperatura es 23.5" o "la puerta está abierta" a "grupos" con nombre. Un servidor en el medio los reparte a quien esté escuchando: un tablero, una base de datos, otra placa. Todo lo que sigue en este capítulo es ponerle nombre técnico a esta idea y entender por qué está bien pensada.

¿Qué es IoT?

IoT son las siglas en inglés de *Internet of Things*: **Internet de las Cosas**. Es la idea de conectar a una red **objetos físicos** (no computadoras ni celulares) para que:

- **midan** algo del mundo real (temperatura, si una puerta está abierta, si hay alguien), o
- **hagan** algo en el mundo real (prender una luz, abrir una cerradura, cortar un enchufe).

Ejemplos que ya conocés: un portón que se abre desde el celular, una alarma que avisa al teléfono, un medidor de luz inteligente. Los proyectos de ustedes (el enchufe de Jorge, el

sensor de puerta de Jessi, el LED de Mijael) son IoT.

¿Qué es "arquitectura de software"?

Antes de construir una casa, un arquitecto decide **qué ambientes tiene, cómo se conectan y por qué** (la cocina cerca del comedor, el baño lejos del living). No pone ladrillos: dibuja los planos y justifica cada decisión.

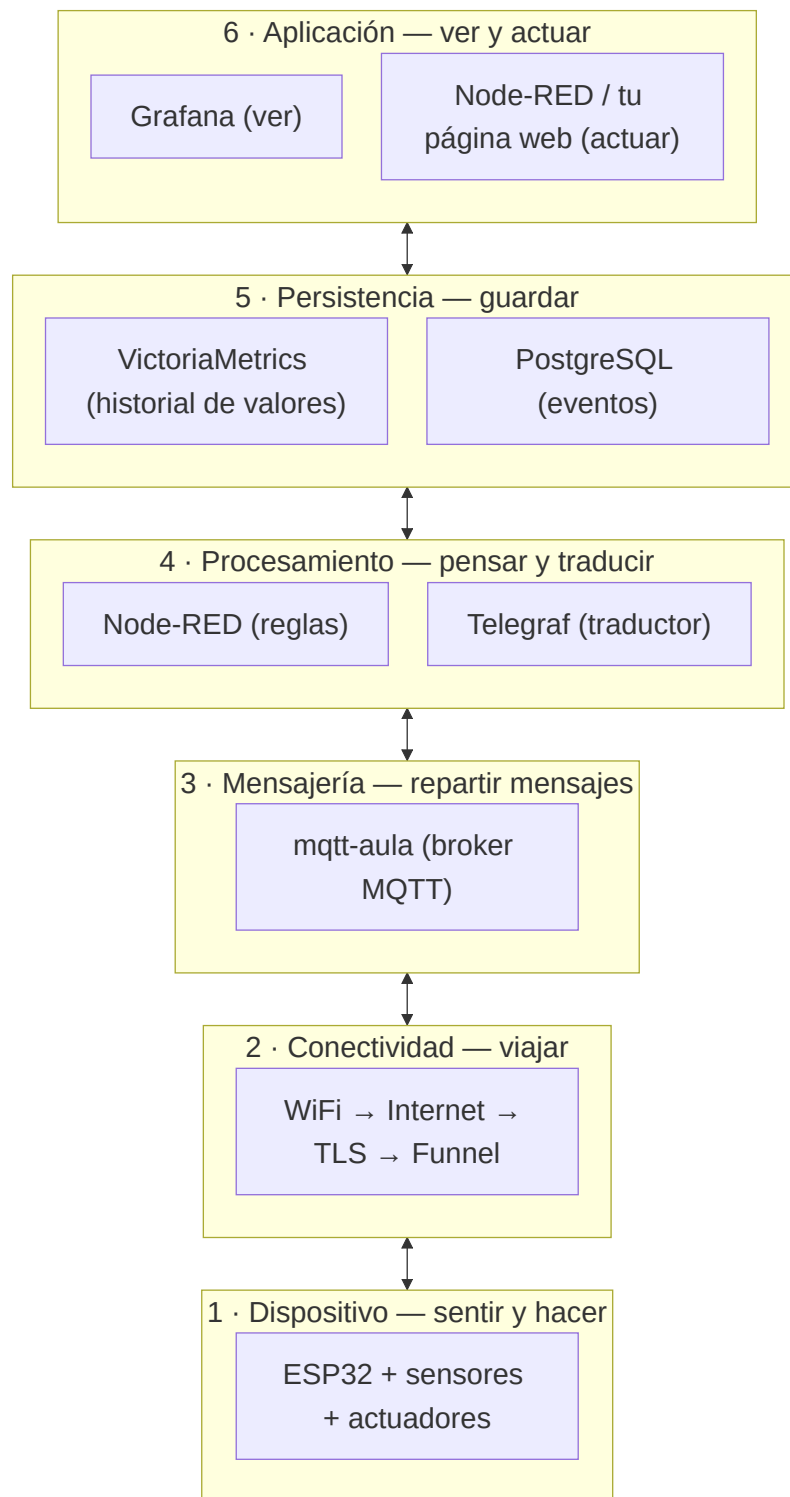
La **arquitectura de software** es lo mismo para un sistema: decidir **qué partes tiene, cómo se comunican y por qué se eligió cada una** frente a otras opciones. Un buen diseño hace que el sistema:

- se pueda **entender** (cada parte tiene un trabajo claro),
- se pueda **cambiar** sin romper todo (cambiar una parte no obliga a tocar las demás),
- **aguante fallas** (si algo se cae, lo demás sigue o se recupera solo).

En este capítulo vamos a mirar el sistema del aula con ojos de arquitecto.

Las capas: cada parte con un solo trabajo

Una forma clásica de ordenar un sistema IoT es en **capas**: como los pisos de un edificio, cada piso tiene su función y solo habla con el de arriba y el de abajo. Así, si cambiás un piso (por ejemplo, otra placa), los demás no se enteran.



Las vemos de abajo hacia arriba, siempre con un ejemplo de ustedes.

Capa 1 — Dispositivo: sentir y hacer

Es la parte que **toca el mundo físico**. Tiene tres piezas, y la analogía con el cuerpo ayuda:

Pieza	Como en el cuerpo	Qué es	Ejemplo del aula
Sensor	los sentidos	mide algo y lo convierte en un dato	el <i>reed switch</i> de Jessi (un interruptor que se cierra cuando le acercás un imán: sirve para saber si una puerta está cerrada)
Actuador	los músculos	hace algo cuando recibe una orden	el LED de Mijael, el relé del enchufe de Jorge (un relé es un interruptor que se acciona con una señal eléctrica chica y puede cortar un aparato grande)
Microcontrolador	un cerebro chico	una computadora mínima que lee sensores, maneja actuadores y se conecta a la red	la ESP32

La **ESP32** es una placa barata con WiFi incorporado. Sus patas de conexión se llaman **GPIO** (*General Purpose Input/Output*: entrada/salida de uso general): cada pata puede **leer** (¿hay corriente o no?) o **escribir** (dar corriente o no). Cuando el código dice `Pin(2, Pin.OUT)`, está diciendo "la pata número 2 la uso para dar corriente": ahí va el LED.

Al microcontrolador se le carga un programa llamado **firmware** (el software que vive adentro de un aparato). Ustedes lo escriben en **MicroPython** (una versión chica de Python para placas) o en **Arduino** (C++ con librerías para placas).

Capa 2 — Conectividad: cómo viaja el dato

El dato tiene que salir de la placa y llegar al servidor del aula. El viaje es:

1. **WiFi**: la placa se conecta a la red de la casa o del colegio (tiene que ser de **2.4 GHz**; la ESP32 no ve redes de 5 GHz).
2. **Internet**: el dato sale a internet, como cuando mandás un mensaje desde tu celular.
3. **TLS** (*Transport Layer Security*, "seguridad de la capa de transporte"): el dato viaja **cifrado**, como una carta en **sobre cerrado** en vez de una postal. Cualquiera en el camino ve que pasa una carta, pero no puede leerla. Es lo mismo que pone el candado  en el navegador.
4. **Tailscale Funnel**: el servidor del aula está en una casa, detrás de un router (el aparatito que da internet) que no deja entrar a nadie desde afuera. **Funnel** es un servicio de la empresa Tailscale que funciona como un **portero**: recibe al visitante en una dirección pública (`homelab-01.tail4eda13.ts.net` , puerto `10000`) y lo hace pasar al servidor por un pasillo seguro. Así no hay que abrir puertas del router.

Un **puerto** es como el número de departamento dentro de un edificio: la dirección lleva al edificio (el servidor) y el puerto al servicio exacto (el `10000` es el de los mensajes de las placas).

Para ustedes como personas (no para las placas) hay otro camino: **Tailscale**, una red privada que conecta sus computadoras con el servidor como si estuvieran en la misma casa. **Por qué dos caminos, qué es un túnel y por qué te piden tantas contraseñas** → [Red y accesos](#).

Capa 3 — Mensajería: el que reparte los mensajes

Acá vive el "servidor de WhatsApp" del principio. La tecnología se llama **MQTT** (*Message Queuing Telemetry Transport*): un idioma muy liviano, pensado para aparatos chicos con poca batería y redes malas, para mandar mensajes cortos.

Palabra MQTT	Qué es	En WhatsApp sería...
Broker	el programa del medio que recibe y reparte	el servidor de WhatsApp
Topic (tema)	el "nombre del grupo" al que se manda un mensaje	el nombre del grupo
Publicar (<i>publish</i>)	mandar un mensaje a un topic	escribir en el grupo
Suscribirse (<i>subscribe</i>)	anotarse para recibir lo de un topic	estar en el grupo
Payload (carga)	el contenido del mensaje (23.5 , ON)	el texto del mensaje
Mensaje retenido (<i>retained</i>)	el broker guarda el último y se lo da al que llega	un mensaje fijado
Último deseo (<i>Last Will</i>)	lo que el broker publica por vos si te desconectás de golpe	"si no contesto, avisen que me quedé sin batería"
QoS (<i>Quality of Service</i> , calidad de servicio)	cuánto esfuerzo pone en que llegue (0 = una vez, sin confirmar)	el doble tilde

El broker del aula se llama `mqtt-aula`. Es uno solo para todos los equipos, pero cada equipo tiene **su usuario y su clave**, y una **ACL** (*Access Control List*, lista de control de acceso: las reglas de "quién puede hacer qué") que lo deja escribir y leer **solo** en los topics que empiezan con su nombre. El equipo-04 puede usar `equipo-04/...`, y nada más.

Un topic se escribe como una ruta de carpetas, separada por `/`:

```
equipo-04 / enchufe / estado
  ↳ equipo    ↳ aparato    ↳ qué cosa
```

Capa 4 — Procesamiento: pensar y traducir

Los mensajes crudos hay que **usarlos**. En el aula hay dos piezas:

- **Node-RED**: una herramienta donde armás la lógica **conectando cajitas** en vez de escribir código ("cuando llega esto, hacé aquello"). Es la que usan para el interruptor

del dashboard y para la página de Mijael.

- **Telegraf:** un programa que hace de **traductor y escribano**. Escucha los mensajes de todos los equipos, los traduce a números (ON → 1, abierto → 1, "23.5" → 23.5) y los anota en la base de datos con la hora exacta.

Ojo con un detalle que confunde: **Telegraf escucha el broker directamente**. Los datos llegan a Grafana **sin pasar por Node-RED**. Node-RED es para **reglas y botones**, y es opcional para graficar. Cuándo usar cada uno → [¿Node-RED, Grafana o ambos?](#).

Capa 5 — Persistencia: guardar para después

Persistir quiere decir **guardar de forma que no se pierda** aunque se apague todo. Se usan bases de datos distintas según el tipo de dato, como en una casa se guarda la ropa en el placard y la comida en la heladera:

- **VictoriaMetrics** guarda **series temporales**: listas de valores con su hora, como una planilla de "hora | temperatura". Es lo ideal para sensores: "¿cómo cambió la temperatura en las últimas 3 horas?". Guarda **15 días**.
- **PostgreSQL** guarda **eventos y registros**, como un libro de actas: "a las 10:32 entró Jorge al pañol". Se usa donde importa quién hizo qué.

Capa 6 — Aplicación: lo que ve y usa la persona

- **Grafana** es un programa para **ver** datos en gráficos. Pensalo como el **tablero de un auto**: te muestra velocidad, nafta y temperatura, pero no maneja. Cada equipo tiene su propio tablero (*dashboard*). Ver [capítulo 13](#).
- **Node-RED o tu página web** son el **volante**: desde ahí se **mandan órdenes** (prender el LED, cortar el enchufe).

Lo que atraviesa todas las capas

Algunas cosas no son un piso del edificio sino **las instalaciones** que pasan por todos (como la electricidad o las cañerías):

- **Seguridad:** cada equipo con su identidad, el cifrado en el viaje, y cada programa con el **mínimo permiso** que necesita (Telegraf puede leer, pero no escribir).
- **Observabilidad:** poder saber qué está pasando adentro mirando desde afuera (los tableros, los registros o *logs*).
- **Infraestructura como código:** todo el servidor está descrito en archivos que se pueden volver a aplicar (ver [capítulo 4](#)). Si mañana se rompe, se reconstruye igual.

Seguí un dato de punta a punta: 24,7 °C

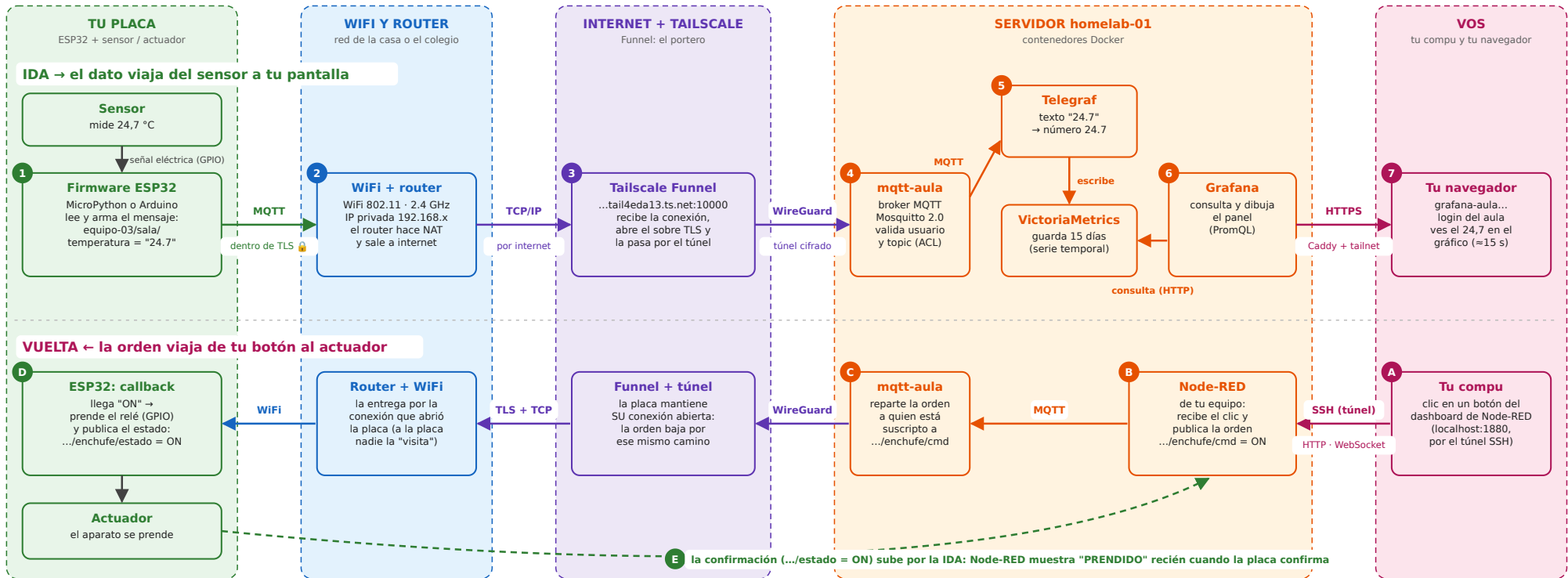
Las capas explican **qué** hay. Ahora seguimos **un dato concreto** por todas ellas, con los nombres reales del sistema. Supongamos que el equipo-03 tiene un sensor de temperatura en la sala y mide **24,7 °C**.

El ciclo completo, en una imagen

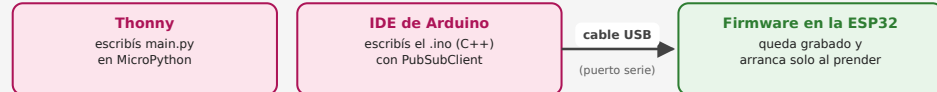
Ida (el dato, del sensor a tu pantalla) y vuelta (la orden, de tu botón al actuador), con **el protocolo de cada tramo** y las herramientas con las que trabajás:

El ciclo completo: ida (el dato) y vuelta (la orden)

Cada flecha dice QUÉ protocolo viaja por ese tramo. Los números y letras marcan el orden del recorrido.



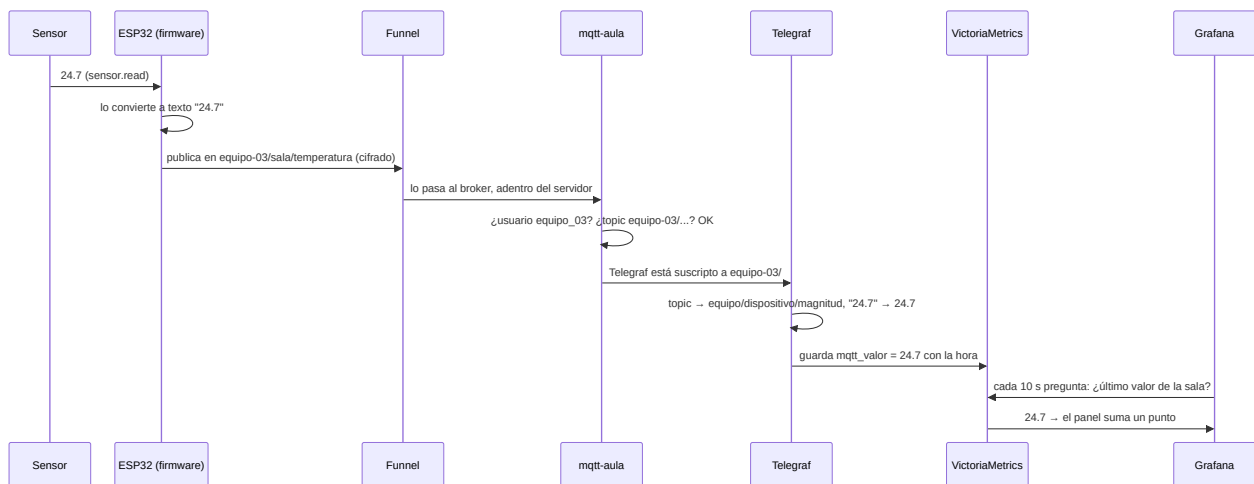
ANTES DE TODO: cómo llega tu código a la placa (y con qué trabajás vos)



Otras herramientas tuyas, y para qué:

- Tailscale: entrar a la red privada del aula • Terminal + SSH: entrar al servidor y abrir el túnel
- MQTT Explorer: escuchar tus mensajes en el broker • Navegador: Node-RED, tu página y Grafana
- Consola de Thonny / Monitor Serie: ver qué hace la placa (WiFi OK, MQTT OK, errores)

Paso a paso



#	¿Quién?	¿Qué hace con el dato?	¿En qué formato está?
1	Sensor	mide la temperatura	una señal eléctrica
2	Firmware de la ESP32	lo lee (<code>sensor.read()</code>) y lo pasa a texto	número <code>24.7</code> → texto <code>"24.7"</code> (con punto , no coma)
3	Firmware	lo publica en el topic <code>equipo-03/sala/temperatura</code>	mensaje MQTT: <i>topic + payload</i> <code>"24.7"</code>
4	WiFi → internet	lleva el mensaje cifrado hasta el portero	MQTT dentro de TLS (el sobre cerrado)
5	Funnel	recibe en <code>homelab-01.tail4eda13.ts.net:10000</code> , saca el sobre y lo entrega adentro del servidor	MQTT sin cifrar, pero ya dentro del servidor
6	mqtt-aula	comprueba que el usuario sea <code>equipo_03</code> y que el topic empiece con <code>equipo-03/</code> ; se lo reparte a todos los suscriptos	el mismo mensaje
7	Telegraf	separa el topic en partes (<code>equipo-03 / sala / temperatura</code>) y convierte <code>"24.7"</code> en el número 24.7	<code>mqtt_valor{equipo="equipo-03", dispositivo="sala", magnitud="temperatura"} = 24.7</code>
8	VictoriaMetrics	lo guarda con la hora exacta, durante 15 días	una fila más en la serie temporal
9	Grafana	cada 10 segundos le pregunta a VictoriaMetrics y dibuja	un punto en el gráfico

Tiempo total: el mensaje llega al broker en menos de un segundo; Telegraf escribe cada 5 segundos; Grafana se actualiza cada 10. En unos **15 segundos** el 24,7 está en tu panel.

El mismo camino, con un dato real del aula: cuando el enchufe de Jorge publica `ON` en `equipo-04/enchufe/estado`, recorre exactamente los pasos 3 a 9. En el paso 7, Telegraf convierte la palabra `ON` en el número **1** (y `OFF` en **0**) para poder graficarla.

¿Quién conoce a quién?

Una de las ideas más importantes de este diseño: **cada pieza conoce solo a su vecina.**

Pieza	Conoce a...	NO conoce a...
ESP32	la dirección del broker, su usuario y su topic	a Telegraf, a Grafana, a Node-RED
mqtt-aula	quién está suscripto a qué (en ese momento)	qué hace cada uno con los mensajes
Telegraf	el broker (para escuchar) y VictoriaMetrics (para escribir)	a las placas
VictoriaMetrics	nada: solo guarda y responde preguntas	de dónde vienen los datos
Grafana	VictoriaMetrics (su <i>datasource</i>)	a las placas y al broker
Node-RED	el broker	a Telegraf y a Grafana

Por eso se pudo **sumar Grafana sin tocar ninguna placa**: la placa sigue publicando igual que antes, y alguien nuevo se suscribió (decisión 1, abajo).

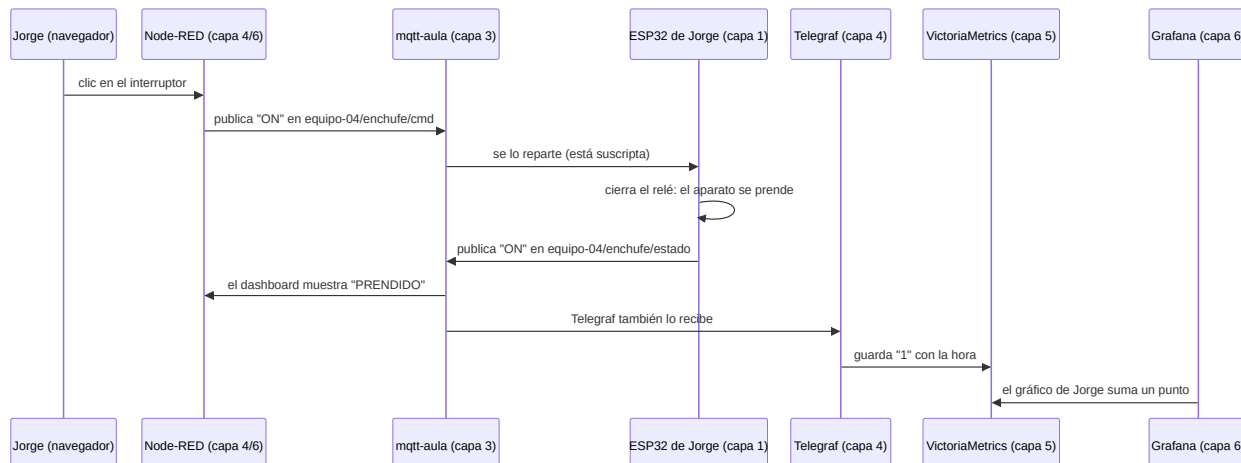
¿Qué pasa si se cae cada pieza?

Si se cae...	Pasa esto	Lo que sigue andando
el WiFi de la placa	no sale nada; la placa reintenta sola	todo lo demás
Funnel (el portero)	ninguna placa llega al broker	Grafana muestra lo ya guardado; Node-RED sigue abierto
mqtt-aula	nadie recibe ni manda mensajes	Grafana muestra el historial hasta ese momento
Node-RED	no hay botones ni reglas	los datos siguen llegando a Grafana (no pasan por Node-RED)
Telegraf	los datos llegan al broker pero no se guardan	Node-RED y las placas funcionan; Grafana muestra un hueco
VictoriaMetrics	no se guarda ni se puede consultar	las placas y Node-RED
Grafana	no se puede ver	los datos se siguen guardando: al volver, aparecen

Cuando no sabés cuál se cayó, el [diagnóstico](#) lo recorre paso a paso.

El viaje completo de un mensaje: Jorge prende su enchufe

Seguimos un clic, paso a paso, por todas las capas:



Fijate en algo importante: el dashboard muestra **"PRENDIDO"** recién cuando la placa **confirma**, no cuando Jorge hace clic. Esa es la primera de las decisiones de diseño que vemos ahora.

Las decisiones de diseño (y por qué)

Cada decisión sigue el mismo formato: **qué problema había, qué opciones existían, qué se eligió y qué pasaría si no.**

Decisión 1 — Publicar/suscribir en vez de que cada uno hable con cada uno

- **Problema:** hay placas, tableros, una base de datos y páginas web. Si cada uno tuviera que conocer la dirección de cada otro, sería una telaraña: sumar un tablero obligaría a reprogramar todas las placas.
- **Opciones:** que la placa le hable directo a cada programa, o poner un **intermediario** (el broker).
- **Elección:** intermediario. La placa publica **una vez** y no sabe ni le importa quién escucha. Sumar Grafana no requirió tocar ninguna placa.
- **Si no:** cada cambio en el servidor obligaría a reprogramar las placas.
- **Nombre técnico:** **desacople** (las partes no dependen unas de otras).

Decisión 2 — El topic es un contrato

- **Problema:** si cada equipo inventa sus topics y sus mensajes como quiere, nadie más puede entenderlos (ni Grafana ni otro equipo).
- **Opciones:** dejarlo libre, o acordar **reglas** de antemano.

- **Elección:** un **contrato**, que es un acuerdo escrito de cómo se habla: `equipo/dispositivo/magnitud`, y mensajes que son un número, una palabra de estado o un JSON (las [3 reglas del capítulo 12](#)). En software, a un contrato así se le llama **interfaz** o **API** (*Application Programming Interface*: la forma acordada en que dos programas se hablan).
- **Si no:** pasa lo que le pasó al reed de Jessi. Publicaba en `equipo-01/reed` (dos partes), y por eso no aparece en Grafana (ver [capítulo 14](#)).

Decisión 3 — Separar la orden del estado confirmado

- **Problema:** que mandes "prendé" no significa que se haya prendido. La placa puede estar desconectada, o el relé trabado.
- **Opciones:** mostrar lo que se pidió, o mostrar lo que la placa **confirma**.
- **Elección:** dos topics. `.../cmd` es la **orden**; `.../estado` es la **confirmación** que publica la placa después de hacerlo. El tablero muestra la confirmación.
- **Si no:** el tablero diría "PRENDIDO" con la placa apagada. En un enchufe o en una cerradura, mentirle al usuario es peligroso.

Decisión 4 — Ver y actuar, separados

- **Problema:** ¿desde dónde se prende el LED? ¿Desde Grafana?
- **Elección:** Grafana **solo muestra** (el tablero del auto); las órdenes salen de Node-RED o de tu página (el volante).
- **Por qué:** el que mira no necesita permiso para mandar. Así se le puede dar Grafana a mucha gente (incluso mostrarlo en público) sin que nadie pueda tocar un aparato.

Decisión 5 — La red falla: diseñar para reconectar

- **Problema:** el WiFi se corta, el servidor se reinicia, una entrada de internet se cae. **Va a pasar**, no es "si pasa".
- **Elección:** el firmware **reintenta solo** (cada 5 segundos), arranca en un estado seguro (relé apagado), y avisa con el **último deseo** si se desconecta de golpe (`equipo-NN/placa/conexion = offline`).
- **Si no:** cada corte necesitaría que alguien vaya a reiniciar la placa a mano.
- **Nombre técnico:** diseño **a prueba de fallos** (*fail-safe*). Casos reales de este mes en el [capítulo 10](#).

Decisión 6 — Cada equipo en su carril

- **Problema:** un solo broker para todos. ¿Qué impide que un equipo prenda el LED de otro, sin querer o a propósito?
- **Elección:** usuario y clave por equipo, más una ACL que lo encierra en `equipo-NN/#` (el `#` significa "todo lo que siga"). Un mensaje fuera de su carril el broker lo **descarta**.
- **Nombre técnico:** **multitenencia** (muchos "inquilinos" en un mismo sistema, cada uno en lo suyo).

Decisión 7 — Mínimo privilegio

- **Idea:** cada programa tiene **solo** el permiso que necesita para su trabajo.
- **En el aula:** Telegraf puede **leer** los topics de los equipos, pero no **escribir**. Si alguien lo engañara, no podría prender nada.

Decisión 8 — Ponerle límites al error

- **Problema:** una placa con un error de código puede publicar **miles de mensajes por segundo**, o inventar topics nuevos sin parar, y llenar el disco.
- **Elección: topes** en todo el camino. La base acepta un máximo de series nuevas por día, los nombres tienen un largo máximo y un JSON se lee hasta 20 valores.
- **Si no:** el error de un equipo tumbaría el sistema de todos.

Decisión 9 — Todo escrito, nada "a mano"

- **Elección:** el servidor, el broker, Grafana y hasta los permisos de cada equipo están descriptos en archivos (Ansible) dentro del repositorio.
- **Por qué:** sumar un equipo es agregar una línea y aplicar. Si el servidor se rompe, se reconstruye igual. Y cualquiera puede **leer** cómo está hecho.



Ideas clave

- IoT = objetos físicos que **miden** (sensores) o **hacen** (actuadores), conectados a una red.
- El sistema se ordena en **6 capas**: dispositivo, conectividad, mensajería, procesamiento, persistencia y aplicación. Cada una tiene un solo trabajo.
- **MQTT** funciona como un grupo de WhatsApp: se publica en un **topic** y le llega a quien esté suscripto, a través de un **broker**.
- El **topic es un contrato**: `equipo/dispositivo/magnitud`.
- **Orden** (`cmd`) y **estado confirmado** (`estado`) van separados.
- **Grafana mira, Node-RED actúa.**
- La red **va a fallar**: la placa tiene que reconectar sola.



Errores comunes

- Publicar sin respetar el contrato (`equipo-01/reed` , `casa/enchufe/cmd`): el mensaje se pierde o no aparece en Grafana.
- Mostrar en el tablero la **orden** en vez del **estado confirmado**.
- Querer manejar un actuador desde Grafana.
- Escribir firmware que, si se cae el WiFi, se queda trabado esperando para siempre.

? Preguntas de repaso

1. ¿Qué diferencia hay entre un sensor y un actuador? Da un ejemplo de cada uno de los proyectos del aula.
2. Explicá con la analogía de WhatsApp qué es un broker, un topic y un mensaje retenido.
3. ¿Por qué el dashboard de Jorge espera la confirmación de la placa antes de mostrar "PRENDIDO"?
4. ¿Qué pasaría si un equipo pudiera publicar en los topics de otro?
5. ¿Para qué sirve el "último deseo" (*Last Will*)?

Ejercicios

1. Dibujá **tu proyecto** con las 6 capas: qué pieza va en cada una.
2. Escribí los topics de tu proyecto respetando el contrato `equipo-NN/dispositivo/magnitud`. Marcá cuáles son **órdenes** y cuáles **estados**.
3. Elegí una de las 9 decisiones y escribí qué le pasaría a **tu** proyecto si no se hubiera tomado.

Ahora deberías entender

- **Qué problema resuelve** cada capa, y **dónde está** cada pieza del aula.
- El **recorrido de un dato**: sensor → firmware → WiFi → Funnel → broker → Telegraf → VictoriaMetrics → Grafana, con Node-RED al costado.
- **Quién conoce a quién**, y qué sigue andando cuando una pieza se cae.

Seguí por acá:

- Si querés entender **la parte invisible de la red** (Tailscale, túneles, por qué tantas contraseñas) → [Red y accesos](#).
- Si querés **conectar tu placa ya** → [capítulo 12](#).
- Si no sabés **si usar Node-RED, Grafana o los dos** → [¿Node-RED, Grafana o ambos?](#).
- Si tu dato **no aparece** → [Diagnóstico](#).

Red y accesos: cómo llegás a cada cosa (y por qué hay tantas contraseñas)

 **Objetivo:** entender la parte "invisible" que empieza después de que tu ESP32 se conecta al WiFi: **por dónde viajan** tu placa y tu compu hasta el servidor, **qué es Tailscale** y por qué lo usamos, **qué es un túnel**, y por qué te piden **distintas** autenticaciones en cada paso.

 **Prerequisitos:** [capítulo 11 \(Arquitectura IoT\)](#).

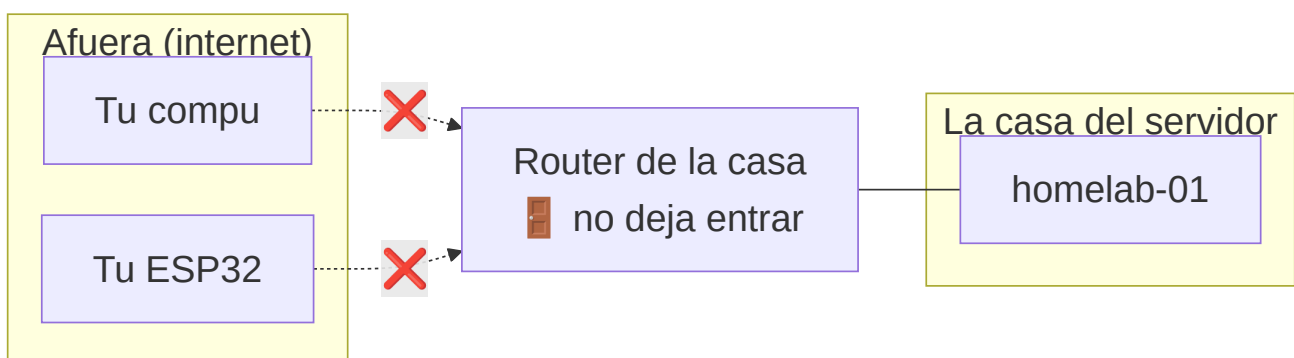
 **Conceptos nuevos** (cada uno con su definición en el glosario): [VPN](#) · [Tailscale / tailnet](#) · [Funnel](#) · [túnel](#) · [proxy inverso](#) · [autenticación](#) · [autorización](#) · [credencial](#).

Si tenés apuro: ¿solo necesitás conectarte ya? → [guía del equipo, paso 1](#). ¿No podés entrar a algo? → [Diagnóstico de accesos](#).

1. El problema: el servidor está en una casa

El servidor del aula es una computadora que está en una **casa**, conectada a un **router** común (el aparatito que da internet). Pensá en tu propia casa: desde afuera, nadie puede entrar a tu compu. El router funciona como la **puerta de calle**: deja **salir** todo, pero no deja **entrar** a nadie que no haya sido invitado.

Eso es bueno para la seguridad, pero nos deja un problema: ustedes, desde sus casas o desde el colegio, **necesitan llegar** a Node-RED, a Grafana o a la terminal del servidor. Y sus placas necesitan llegar al broker MQTT.



¿Por qué no "abrimos la puerta" y listo?

Se podría configurar el router para que deje entrar todo (eso se llama **redirección de puertos**). No lo hacemos por tres razones:

1. **Internet está lleno de robots** que prueban contraseñas en cualquier computadora que encuentran abierta, todo el día.
2. **Algunos servicios no tienen contraseña.** El editor de Node-RED, por ejemplo, permite ejecutar **cualquier cosa** en el servidor: abierto a internet, sería regalarle el servidor a cualquiera.
3. **No siempre se puede.** El router de esta casa (y muchos proveedores de internet) ni siquiera lo permite.

La decisión de arquitectura: no abrir puertas al público, y en cambio armar **caminos privados** para cada uno que necesita entrar. Son dos caminos, uno para **personas** y otro para **placas**.

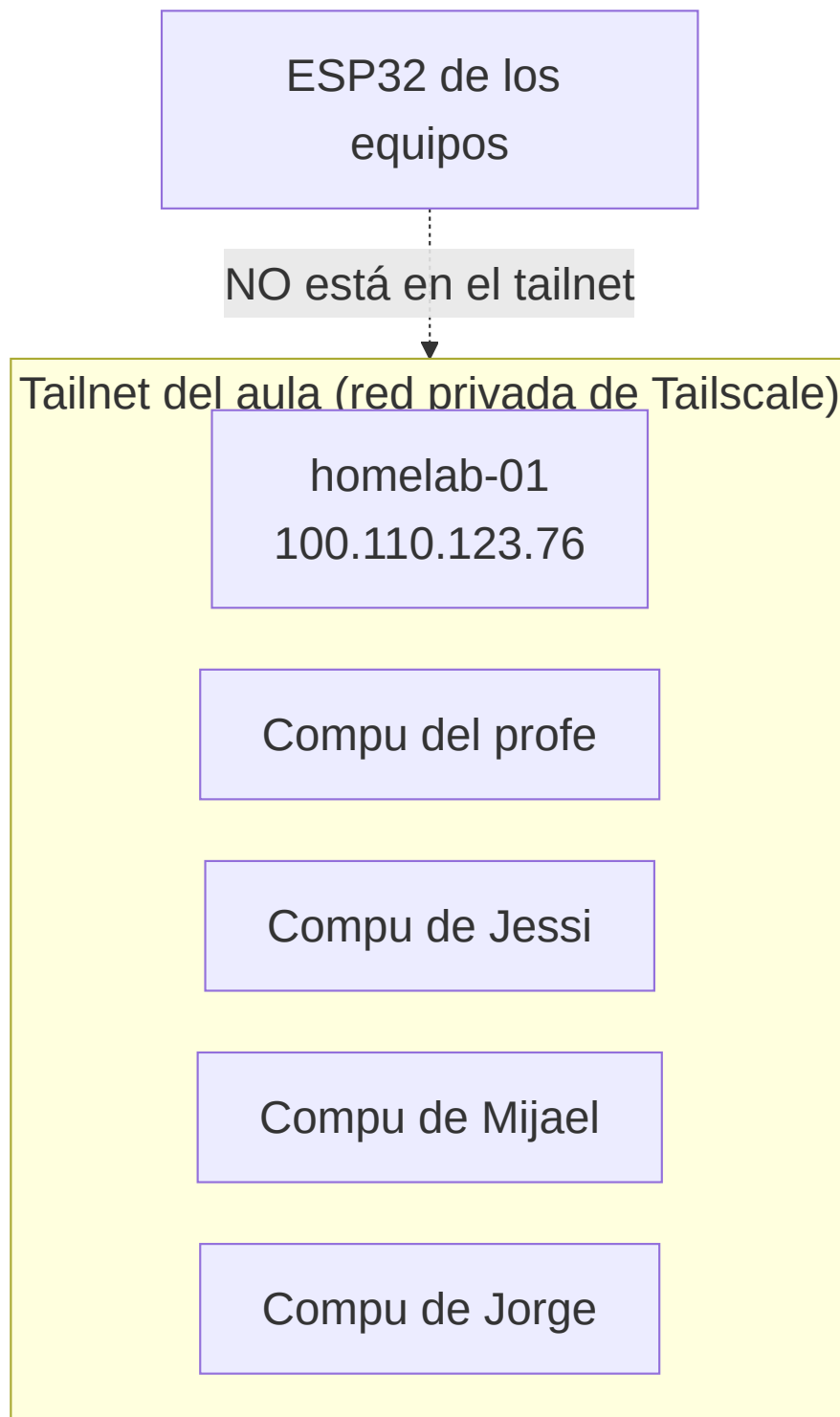
2. El camino de las personas: Tailscale, una red privada

La idea

Imaginá que el servidor, tu compu y las de tus compañeros estuvieran enchufados al **mismo router**, como en una sala de computación, aunque cada uno esté en su casa. Eso es una **red privada**: un "club" de máquinas que se ven entre ellas, y nadie de afuera las ve.

A una red privada que viaja **por adentro de internet**, cifrada, se le dice **VPN** (*Virtual Private Network*, red privada virtual). **Tailscale** es el programa que arma esa VPN de forma muy simple: lo instalás, te unís con una clave, y listo. A la red privada que arma Tailscale se le dice **tailnet**.

Qué máquinas participan



- **Están en el tailnet:** el servidor (homelab-01 , IP privada 100.110.123.76), la compu del profe y las compus de los alumnos que se sumaron con su clave.
- **No están:** las ESP32. Tailscale es un programa para computadoras y celulares; una placa tan chica no lo puede correr. Por eso las placas tienen **su propio camino** (sección 3).

Qué puede ver cada uno

Estar en el tailnet te deja **llegar** al servidor, pero llegar no es entrar:

- Ves al servidor en su IP privada (100.110.123.76).
- En el servidor solo responde lo que está habilitado para el tailnet: la **terminal** (SSH) y la **puerta web** (Caddy, que pide login).
- Los servicios **sin login propio** (tu Node-RED, tu página nginx) **no** responden al tailnet: escuchan **solo adentro del servidor** (127.0.0.1). Para llegar a ellos hace falta un túnel (sección 4).

Error típico real: si entraste a Tailscale con tu cuenta de Google antes de usar la clave del aula, Tailscale te armó **tu propio** tailnet vacío, sin el servidor. Por eso el primer comando es `tailscale logout` ([caso real](#) y [guía, paso 1.1](#)).

3. El camino de las placas: Funnel, un portero en internet

La ESP32 no puede estar en el tailnet, pero necesita llegar al broker. La solución es **Tailscale Funnel**: un servicio de Tailscale que pone un **portero** en internet, con una dirección pública, y le abre la puerta **solo a un servicio** del servidor:

```
homelab-01.tail4eda13.ts.net : 10000 → broker MQTT del aula (mqtt-aula)
```

Ese portero no deja pasar a cualquiera a cualquier lado. Del otro lado está el broker, que exige:

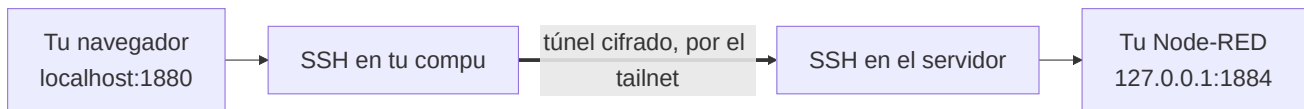
- **cifrado** (TLS: el sobre cerrado), y
- **usuario y clave del equipo**, y además
- solo deja usar los **topics de tu equipo** (equipo-NN/...).

Por eso es razonable dejar **solo el broker** en internet: tiene tres protecciones propias. El editor de Node-RED, que no tiene ninguna, **nunca** sale por acá.

4. El túnel: llegar a lo que no está publicado

Tu Node-RED escucha **solo adentro del servidor**. ¿Cómo lo abris en tu navegador? Con un **túnel**.

Un **túnel** es un **pasillo privado y cifrado** entre tu compu y el servidor: lo que entra por una punta sale por la otra. Lo arma **SSH** (la conexión segura a la terminal del servidor) con la opción `-L`:



`ssh -L 1880:localhost:1884 jorge@100.110.123.76` se lee:

"Abrí un pasillo: lo que mande al **1880 de mi compu** que salga en el **1884 del servidor**."

Por eso en el navegador escribís `localhost:1880` (que significa "mi propia compu") y ves **tu Node-RED, que está en el servidor**. Los números de cada equipo están en la [guía_paso 1.2](#).

¿Por qué Grafana no necesita túnel y Node-RED sí?

	Node-RED de tu equipo	Grafana del aula
¿Tiene login propio?	no : cualquiera que llega, lo maneja	sí : pasa por el login del aula
¿Dónde escucha?	solo adentro del servidor (127.0.0.1)	detrás de Caddy, la puerta web con login
¿Cómo llegás?	tailnet + túnel SSH (el SSH hace de login)	tailnet + navegador (<code>https://grafana-aula...</code>)

La regla de diseño: **lo que no tiene login, no se publica; se llega por un túnel que sí tiene login**.

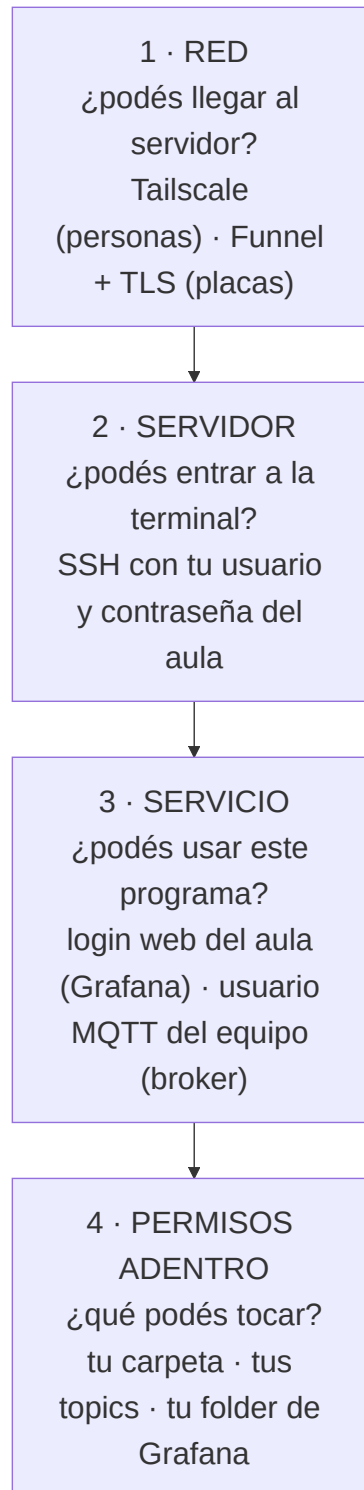
5. ¿Por qué me autentico tantas veces?

Porque cada puerta protege **una cosa distinta**, y una no reemplaza a la otra. Pasa lo mismo en un edificio: la llave de la puerta de calle no abre tu departamento, y la de tu departamento no abre el de al lado.

Dos palabras para no mezclar:

- **Autenticación**: probar **quién sos** (usuario y contraseña, una clave).
- **Autorización**: decidir **qué podés hacer** una vez que se sabe quién sos.

Las 4 capas



Capa	Pregunta	En este sistema	Qué protege	Si falla...
1 · Red	¿podés llegar?	Tailscale (compus) · Funnel + TLS (placas)	que nadie de internet llegue a los servicios privados	Connection timed out → diagnóstico
2 · Servidor	¿podés entrar a la terminal?	SSH con tu usuario del aula	la terminal, tus archivos, los túneles	Permission denied
3 · Servicio	¿podés usar este programa?	Grafana: login del aula (Authelia) · broker: usuario MQTT del equipo · Node-RED: protegido por el túnel SSH	cada programa por separado	Grafana te devuelve al login · la placa da error 5
4 · Permisos	¿qué podés tocar adentro?	Linux: solo tu carpeta · broker: solo equipo-NN/... · Grafana: solo tu folder	que un equipo no toque lo de otro	el mensaje se descarta sin aviso · no ves la carpeta

Tus credenciales, en una tabla

Este es el origen de la mayoría de los errores de este año (ver los [casos 5 y 6](#)):

Para...	Usuario	Clave	Quién la usa
Entrar al tailnet	—	la <i>auth key</i> de Tailscale (<code>tskey-auth-...</code>), una vez	tu compu
Entrar al servidor (SSH) y a Grafana	tu nombre (<code>jorge</code>)	tu contraseña del aula	vos
Que la placa entre al broker	el del equipo, con guion bajo (<code>equipo_04</code>)	<code>MQTT_PASSWORD</code> de tu <code>.shared-services.env</code>	tu ESP32 y tu Node-RED
Que la placa entre al WiFi	el nombre de la red	la clave del WiFi	tu ESP32

La confusión más común: poner la contraseña del aula en la placa, o la clave MQTT en el SSH. Son puertas distintas con llaves distintas.

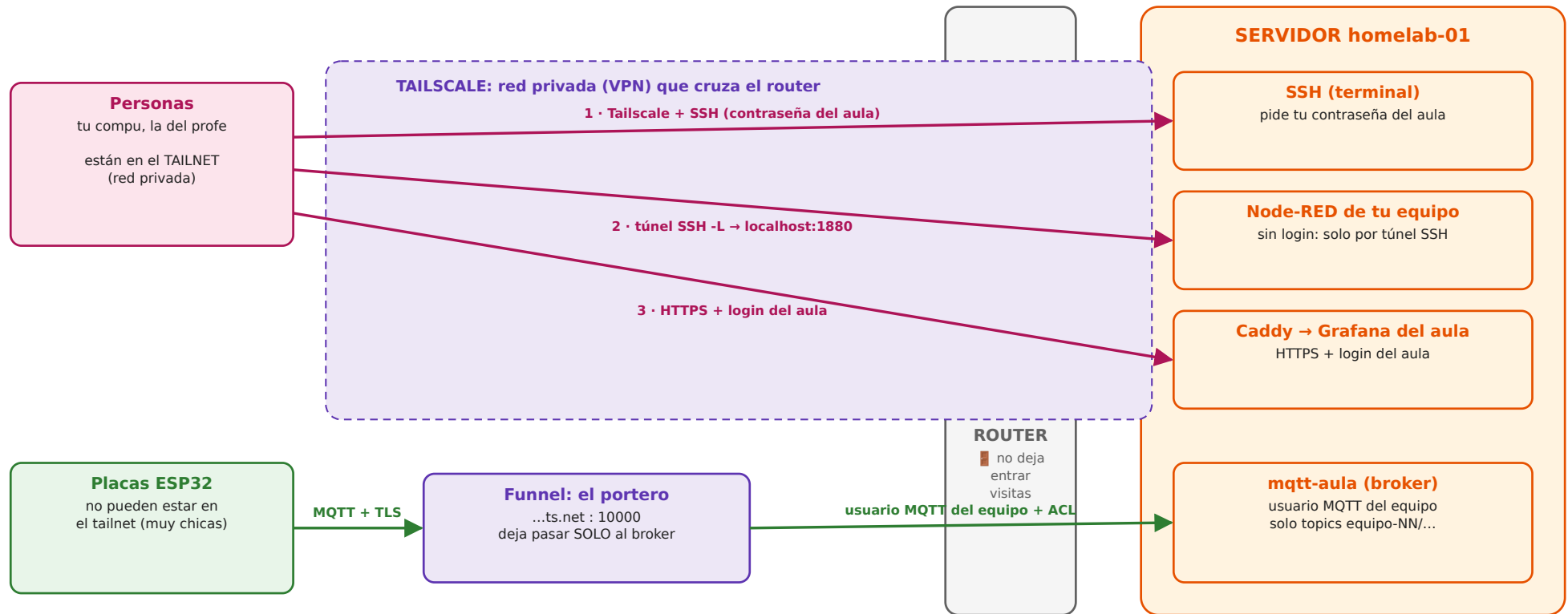
¿Por qué no una sola contraseña para todo?

Porque **cada capa protege algo distinto y la puede usar alguien distinto**. La clave MQTT la tienen que conocer **una placa** y un **Node-RED**: si fuera tu contraseña del aula, quien vea el código de tu placa podría entrar a tu terminal. Separarlas hace que perder una llave no abra todas las puertas (ver **defensa en profundidad** en el [capítulo 7](#)).

6. Los dos caminos, completos

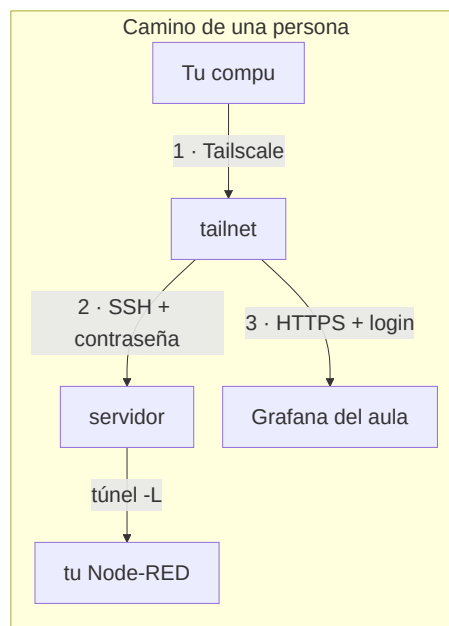
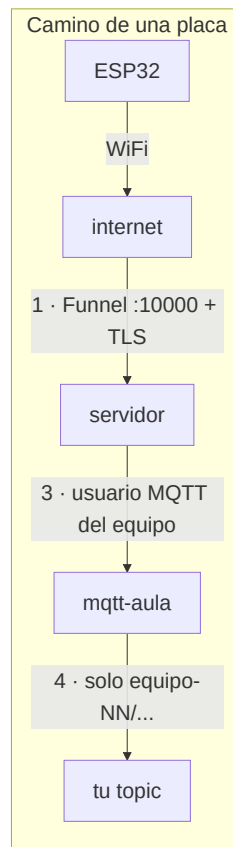
Dos caminos al servidor (y por qué son distintos)

El router de la casa no deja entrar a nadie de afuera. Cada uno llega por su propio camino privado.



Regla de diseño: lo que no tiene login no se publica; se llega por un túnel que sí tiene login.

El mismo dibujo, paso a paso:



Ahora deberías entender

- **Qué problema resolvemos:** el servidor está detrás de un router que no deja entrar, y no queremos abrirlo a internet.

- **Dónde está cada cosa:** las personas entran por **Tailscale**; las placas, por **Funnel**; lo que no tiene login, por un **túnel SSH**.
- **Qué protege cada autenticación:** red → servidor → servicio → permisos.

Seguí por acá:

- Si necesitás **conectarte ahora** → [guía del equipo, paso 1](#).
- Si querés **seguir un dato** desde la placa hasta Grafana → [capítulo 11, "Seguí un dato"](#).
- Si **llegás al servidor pero algo no te deja entrar** → [Diagnóstico: no puedo entrar](#).
- Si querés **las capas de seguridad del servidor** en detalle → [capítulo 7](#).

Para profundizar (operador): las reglas reales están en `ansible/roles/users_ssh/templates/sshd_config.j2` (bloque `Match Group classroom`), en el `Caddyfile` y en `configuration.yml.j2` de Authelia. El Funnel y el Split DNS se explican en [docs/deployment-guide.md](#).

12. Conectar tus sensores y actuadores a Grafana

🎯 **Objetivo:** que lo que mide o hace tu ESP32 aparezca **solo** en el Grafana del aula, en el dashboard de tu equipo, con historial de los últimos **15 días**.

🚧 **Estado (octubre 2026): el Grafana del aula se está poniendo en marcha.**

Todavía no está disponible: el profe avisa cuando lo esté. Mientras tanto ya podés **preparar tu placa** siguiendo las reglas de este capítulo: lo que publiques bien desde ahora va a aparecer solo cuando se active.

✂️ **Prerequisitos:** [capítulo 11 \(Arquitectura IoT\)](#), para entender las piezas. Los datos de tu equipo están en la [guía del equipo](#).

Palabras de este capítulo (todas explicadas en el [capítulo 11](#) y en el [glosario](#)):

- **MQTT:** el idioma con el que las placas se mandan mensajes cortos.
- **Broker:** el programa que recibe los mensajes y los reparte (`mqtt-aula`).
- **Topic:** el "nombre del grupo" al que mandás un mensaje, escrito como una ruta: `equipo-04/enchufe/estado` .
- **Publicar:** mandar un mensaje a un topic.
- **Payload:** el contenido del mensaje (`23.5` , `ON`).
- **TLS:** el cifrado del viaje; como mandar una carta en sobre cerrado.
- **Grafana:** el programa de tableros donde se **ven** los datos.

Cómo funciona (la idea en un dibujo)

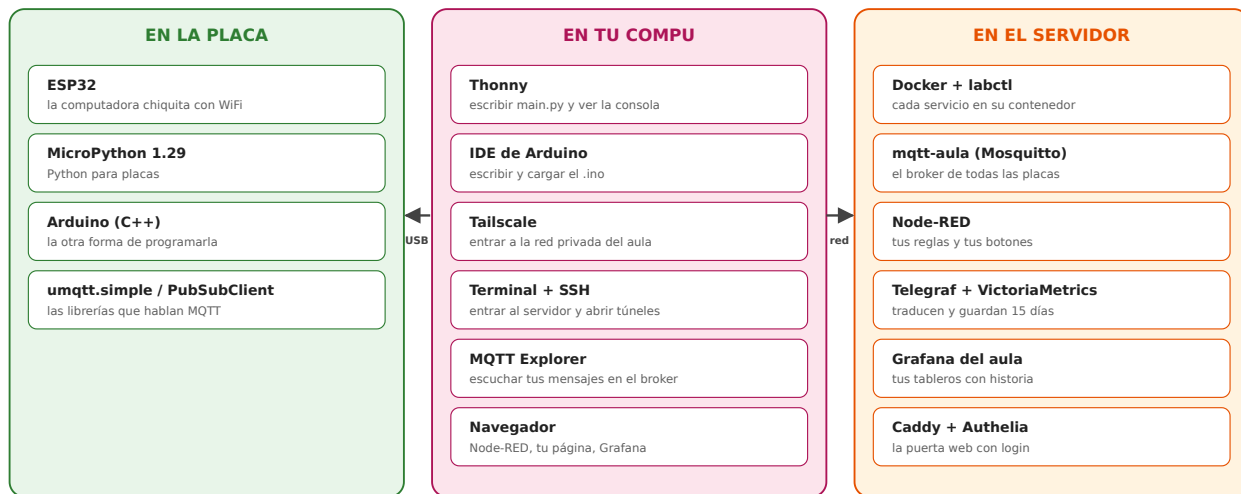
```

tu ESP32 —publica—> mqtt-aula —> se guarda (15 días) —> Grafana del aula
                    (broker)                                ^ dashboard de TU equipo
  
```

No tenés que instalar nada en el servidor ni tocar Grafana para que aparezca: **si publicás respetando las 3 reglas de abajo, tus datos se guardan y se ven solos**. Grafana es para **ver**; para **mandar órdenes** (prender un LED) seguís usando Node-RED o tu página.

Las herramientas que vas a usar

Tus herramientas: qué es cada una y dónde trabaja



Las 3 reglas

Regla 1 — El topic tiene 3 partes (o más)

equipo-NN / dispositivo / magnitud

Parte	Qué es	Ejemplos
equipo-NN	tu equipo, con guion	equipo-04
dispositivo	qué placa o aparato	enchufe, puerta, sala, invernadero
magnitud	qué medís o qué estado es	temperatura, humedad, estado, reed

Ejemplos que **sí** funcionan:

```
equipo-04/enchufe/estado
equipo-03/sala/temperatura
equipo-01/puerta/reed
equipo-02/invernadero/suelo/humedad    (más de 3 partes: se juntan como
"suelo_humedad")
```

Ejemplos que **no** se guardan:

```
equipo-04/temperatura    (faltan partes: ¿de qué dispositivo?)
casa/sala/temperatura    (no empieza con tu equipo: el broker lo descarta)
equipo_04/sala/temperatura (guion bajo: ese es el USUARIO, no el topic)
```

Por qué así: Grafana arma los gráficos agrupando por dispositivo y magnitud. Si el topic no los trae, no hay cómo ordenarlos.

Regla 2 — El mensaje es un número, una palabra de estado o un JSON

Un número (con punto decimal, no coma):

```
23.5
-4
1013
```

Una palabra de estado (mayúsculas o minúsculas da igual). Se guardan como 1 o 0 para poder graficarlas:

Se guarda como 1	Se guarda como 0
ON on true	OFF off false
encendido prendido activo	apagado inactivo
abierto	cerrado
online	offline
alto si sí detectado	bajo no libre

Un JSON plano con varios valores a la vez. **JSON** (*JavaScript Object Notation*) es una forma de escribir datos con **nombre: valor** entre llaves, que entienden casi todos los programas. "Plano" quiere decir sin cajas adentro de cajas. Es útil para sensores que miden dos cosas a la vez, como el **DHT** (un sensor barato de temperatura y humedad):

```
{"temp": 22.1, "hum": 61}
```

Publicado en `equipo-03/sala/clima`, aparece como dos magnitudes: `clima_temp` y `clima_hum`.

Lo que NO se guarda: texto libre (`"hola"`, `"temperatura alta"`), números con coma (`23,5`) o con unidades pegadas (`23.5C`). Si necesitás mandar texto, mandalo a Node-RED: Grafana grafica números.

Regla 3 — Publicá seguido y con medida

- **Sensores:** cada **5 a 60 segundos** está bien. Más rápido que 1 por segundo no sirve para ver y llena la base.
- **Actuadores:** publicá el **estado confirmado** cada vez que cambia (como `equipo-04/enchufe/estado`).
- **Nunca** publiques dentro de un `while True` sin `sleep`: son miles de mensajes por segundo y el sistema te frena.

Datos de conexión

Están en el servidor, en el archivo de tu equipo (solo lectura). Para leerlo tenés que entrar al servidor por **SSH** (la conexión segura por terminal; ver [guía del equipo, paso 1.2](#)):

```
/srv/classroom/equipo-NN/.shared-services.env
```

Para verlos, **entrá por SSH al servidor** y corré (cambiá **NN** por tu número):

```
grep MQTT_ /srv/classroom/equipo-NN/.shared-services.env
```

Dato	Valor
Servidor (host)	homelab-01.tail4eda13.ts.net
Puerto	10000 (con TLS)
Usuario	equipo_NN (con guion bajo)
Clave	la de MQTT_PASSWORD=

Bonus profesional: que Grafana sepa si tu placa está conectada

Usá el "**último deseo**" (*Last Will*) de MQTT, que es como dejarle dicho a un amigo "si no te contesto, avisale a los demás que me quedé sin batería": al conectarte le decís al broker "*si me desconecto de golpe, publicá offline por mí*". Y apenas conectás, publicás **online**. Así Grafana muestra si la placa está viva aunque no esté mandando datos.

Topic: `equipo-NN/<dispositivo>/conexion` con **online** / **offline** (retenido).

Está incluido en los dos ejemplos de abajo.

Ejemplo A — MicroPython (sensor + actuador)

Mide un valor cada 10 segundos (acá simulado con el sensor de temperatura interno; cambialo por el tuyo), y maneja un LED que se ordena desde Node-RED.

Archivo `main.py` en la ESP32:

```

from machine import Pin
import time, network, ssl, esp32
from umqtt.simple import MQTTClient

WIFI_SSID = "NOMBRE_WIFI"
WIFI_PASS = "CLAVE_WIFI"

EQUIPO = "equipo-NN"
DISPOSITIVO = "placa1"
MQTT_HOST = "homelab-01.tail4eda13.ts.net"
MQTT_PORT = 10000
MQTT_USER = "equipo_NN"
MQTT_PASS = "CLAVE_MQTT"

T_CONEXION = EQUIPO + "/" + DISPOSITIVO + "/conexion"
T_TEMP = EQUIPO + "/" + DISPOSITIVO + "/temperatura"
T_LED_CMD = EQUIPO + "/" + DISPOSITIVO + "/led/cmd"
T_LED_ESTADO = EQUIPO + "/" + DISPOSITIVO + "/led/estado"

led = Pin(2, Pin.OUT)
wlan = network.WLAN(network.STA_IF)

def conectar_wifi():
    wlan.active(False)
    time.sleep(0.5)
    wlan.active(True)
    wlan.disconnect()
    wlan.connect(WIFI_SSID, WIFI_PASS)
    for _ in range(40):
        if wlan.isconnected():
            print("Wi-Fi OK", wlan.ifconfig()[0])
            return
        time.sleep(0.5)
    raise OSError("No conecta al Wi-Fi")

def al_recibir(topic, msg):
    led.value(1 if msg in (b"ON", b"1") else 0)
    cliente.publish(T_LED_ESTADO, "ON" if led.value() else "OFF", retain=True)

def conectar_mqtt():
    ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
    ctx.verify_mode = ssl.CERT_NONE
    c = MQTTClient(EQUIPO + "-" + DISPOSITIVO, MQTT_HOST, port=MQTT_PORT,
                  user=MQTT_USER, password=MQTT_PASS, keepalive=60, ssl=ctx)
    c.set_last_will(T_CONEXION, "offline", retain=True)
    c.set_callback(al_recibir)
    c.connect()
    c.publish(T_CONEXION, "online", retain=True)
    c.subscribe(T_LED_CMD)
    print("MQTT OK")
    return c

```

```

while True:
    try:
        if not wlan.isconnected():
            conectar_wifi()
        cliente = conectar_mqtt()
        ultimo = 0
        while True:
            cliente.check_msg()
            if time.ticks_diff(time.ticks_ms(), ultimo) > 10000:
                ultimo = time.ticks_ms()
                temp_c = (esp32.raw_temperature() - 32) / 1.8
                cliente.publish(T_TEMP, "{:.1f}".format(temp_c))
            time.sleep_ms(50)
    except Exception as e:
        print("Error, reintento en 5 s:", e)
        time.sleep(5)

```

Primera vez: la placa necesita la librería **umqtt.simple** (el código que sabe hablar MQTT). Se instala con **mip**, el instalador de paquetes de MicroPython. En la consola de **Thonny** (el programa con el que cargás código a la placa), con la placa ya conectada al WiFi, escribí:

```
import mip; mip.install("umqtt.simple")
```

Ejemplo B — Arduino (sensor + actuador)

Necesita la librería **PubSubClient** (el código que sabe hablar MQTT en Arduino). En el IDE de Arduino (el programa donde escribís y cargás el código): **Herramientas → Administrar bibliotecas** → buscar `PubSubClient` → Instalar.

Dos piezas del código que conviene entender:

- `WiFiClientSecure + setInsecure()`: la conexión va **cifrada** (TLS), pero sin comprobar el certificado del servidor (el "documento de identidad" del servidor). Para el aula alcanza; en un producto real se verificaría.
- `mqtt.connect(id, usuario, clave, topicDeseo, 0, true, "offline")`: entra al broker con usuario y clave, y deja registrado el último deseo.

```

#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>

const char* WIFI_SSID = "NOMBRE_WIFI";
const char* WIFI_PASS = "CLAVE_WIFI";

const char* MQTT_HOST = "homelab-01.tail4eda13.ts.net";
const int   MQTT_PORT = 10000;
const char* MQTT_USER = "equipo_NN";
const char* MQTT_PASS = "CLAVE_MQTT";
const char* CLIENT_ID = "equipo-NN-placa1";

const char* T_CONEXION  = "equipo-NN/placa1/conexion";
const char* T_TEMP      = "equipo-NN/placa1/temperatura";
const char* T_LED_CMD   = "equipo-NN/placa1/led/cmd";
const char* T_LED_ESTADO = "equipo-NN/placa1/led/estado";

const int LED_PIN = 5;

WiFiClientSecure red;
PubSubClient mqtt(red);
unsigned long ultimo = 0;

void alRecibir(char* topic, byte* payload, unsigned int largo) {
    String msg;
    for (unsigned int i = 0; i < largo; i++) msg += (char)payload[i];
    bool on = (msg == "ON" || msg == "1");
    digitalWrite(LED_PIN, on ? HIGH : LOW);
    mqtt.publish(T_LED_ESTADO, on ? "ON" : "OFF", true);
}

void conectar() {
    while (WiFi.status() != WL_CONNECTED) { delay(500); }
    while (!mqtt.connected()) {
        // usuario, clave y "último deseo": si se corta, el broker publica offline
        if (mqtt.connect(CLIENT_ID, MQTT_USER, MQTT_PASS, T_CONEXION, 0, true, "offline")) {
            mqtt.publish(T_CONEXION, "online", true);
            mqtt.subscribe(T_LED_CMD);
            Serial.println("MQTT OK");
        } else {
            Serial.printf("MQTT falló (estado %d), reintento en 5 s\n", mqtt.state());
            delay(5000);
        }
    }
}

void setup() {
    Serial.begin(115200);
    pinMode(LED_PIN, OUTPUT);
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    red.setInsecure();
    mqtt.setServer(MQTT_HOST, MQTT_PORT);
    mqtt.setCallback(alRecibir);
}

```

```

mqtt.setKeepAlive(60);
}

void loop() {
  if (!mqtt.connected()) conectar();
  mqtt.loop();
  if (millis() - ultimo > 10000) {
    ultimo = millis();
    float temp = temperatureRead();          // cambialo por tu sensor
    mqtt.publish(T_TEMP, String(temp, 1).c_str());
  }
}

```

Comprobar que llega (en dos pasos)

1. ¿Llegó al broker? → MQTT Explorer

Antes de mirar Grafana, confirmá que tu mensaje **entró al servidor**. Con **MQTT Explorer** te conectás al broker desde tu compu, con la clave de tu equipo, y ves tus topics en vivo. Cómo configurarlo: [la herramienta clave del diagnóstico](#).

Si tu topic aparece ahí con el valor correcto, **la placa y la red están bien**: cualquier problema está del broker para adelante.

2. ¿Se guardó? → Grafana

🚧 Mientras el Grafana del aula se pone en marcha, el paso 1 es la comprobación que sí podés hacer.

1. Entrá a **<https://grafana-aula.lucasland.duckdns.org>** con **Tailscale** conectado (la red privada del aula) y tu usuario y contraseña del aula. Cómo usarlo: [capítulo 13](#).
2. Menú → **Dashboards** → carpeta **equipo-NN** → **equipo-NN — sensores y actuadores**.
3. En la tabla **Estado actual** tiene que aparecer tu dispositivo y tu magnitud, con el **último valor** y **hace cuántos segundos** llegó.

Llega en unos **15 segundos** desde que la placa publica ([por qué ese tiempo](#)).

Si no aparece

Los errores más comunes, en una línea (el recorrido completo, paso a paso, está en el [Diagnóstico](#)):

Síntoma	Causa más probable	Ir a...
MQTT Explorer no ve tu topic	el topic no empieza con <code>equipo-NN/</code> (guion) o la placa no llegó	Regla 1 · diagnóstico, pasos 3 a 6
MQTT Explorer lo ve, Grafana no	el mensaje no es número ni palabra de estado, o tiene menos de 3 partes	Regla 2
La placa da error <code>5</code> o <code>4</code>	usuario o clave MQTT mal	tabla de credenciales
La placa da error <code>-2</code> o <code>-4</code>	no llega, o falta el cifrado	diagnóstico, paso 3
"Hace" crece en la tabla y no baja	la placa dejó de publicar	mirá Thonny / el Monitor Serie
Picos raros en los gráficos	publicás muy rápido o mezclás unidades	Regla 3

Ahora deberías entender

- Que **el topic es un contrato**: `equipo-NN/dispositivo/magnitud`, y el mensaje un número, una palabra de estado o un JSON.
- Que tu placa **solo conoce al broker**: lo que pasa después (guardar, mostrar) lo hacen otros ([quién conoce a quién](#)).
- Cómo comprobar que llegó: primero **MQTT Explorer**, después **Grafana**.

Seguí por acá:

- Si querés **armar tus gráficos** → [capítulo 13](#).
- Si necesitás **calcular algo o reaccionar** a tus datos → [¿Node-RED, Grafana o ambos?](#).
- Si **no aparece** → [Diagnóstico](#).

¿Node-RED, Grafana o ambos?

 **Objetivo:** decidir, para **tu** proyecto, qué parte hace Node-RED, qué parte hace Grafana y cómo se combinan. No son dos programas distintos "para hacer gráficos": tienen **trabajos distintos**.

 **Prerequisitos:** [capítulo 11 \(Arquitectura IoT\)](#) y [conectar tu placa \(cap. 12\)](#).

 **Conceptos nuevos:** [flujo \(flow\)](#), evento, transformación, dato derivado, republicar. Repasá también [broker](#), [topic](#), [dashboard](#) y [datasource](#).

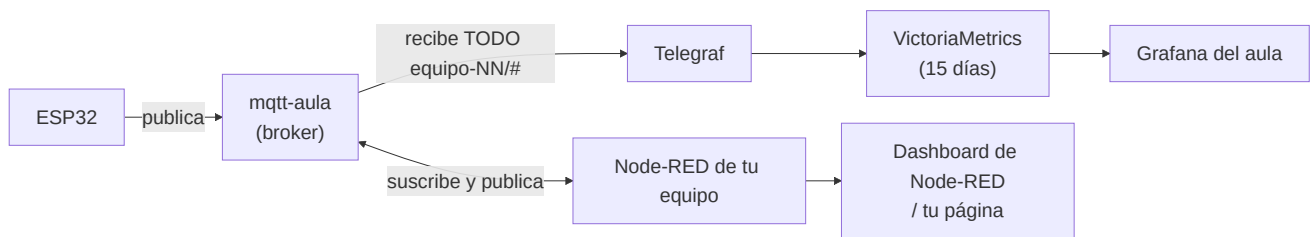
La idea en una frase

Node-RED piensa y actúa. Grafana mira y recuerda.

Volviendo a la analogía del auto ([cap. 11](#)):

- **Node-RED** es el **conductor**: recibe lo que pasa ("se abrió la puerta"), decide ("si está abierta hace 5 minutos...") y **hace** algo ("...prendé la luz roja").
- **Grafana** es el **tablero con historial**: muestra cómo estuvo todo en las últimas horas o días, pero **no maneja**.

Cómo están conectados HOY en el aula



Fijate en dos cosas importantes:

1. **Los datos llegan a Grafana sin pasar por Node-RED.** Telegraf escucha el broker directamente. Si tu placa publica bien, aparece en Grafana aunque no tengas ni un nodo en Node-RED.
2. **Lo que Node-RED publique en un topic de tu equipo también llega a Grafana,** por el mismo camino. Esa es la llave del caso C.

Caso A — Alcanza con Grafana

Cuándo: querés **ver** valores y **cómo cambian en el tiempo**, sin decidir ni hacer nada.



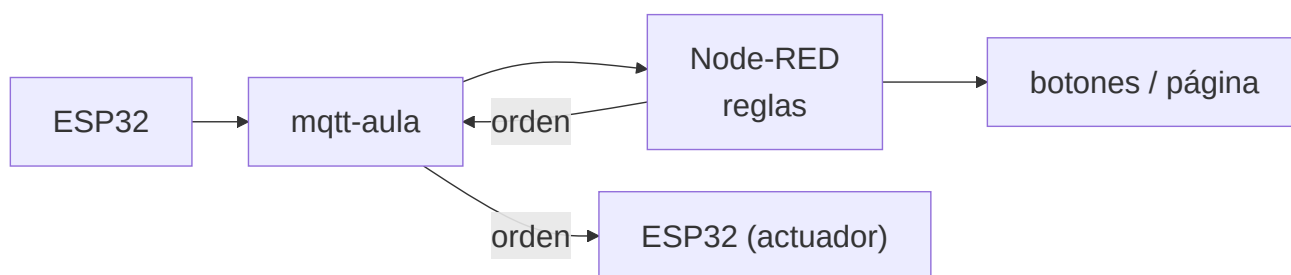
Ejemplos del aula:

- La puerta de Jessi: ¿cuántas veces se abrió hoy y a qué hora? (Con su topic pasado a 3 partes, ver [cap. 14.](#))
- Una temperatura de la sala: la curva de las últimas 24 horas, el máximo, el promedio.
- El enchufe de Jorge: cuánto tiempo estuvo prendido el aparato.

Qué hacés: publicar respetando las [3 reglas](#) y armar el panel en tu dashboard ([cap. 13](#)). Nada más.

Caso B — Hace falta Node-RED

Cuándo: algo tiene que **pasar** cuando llega un mensaje: decidir, transformar, combinar o responder.



Ejemplos del aula:

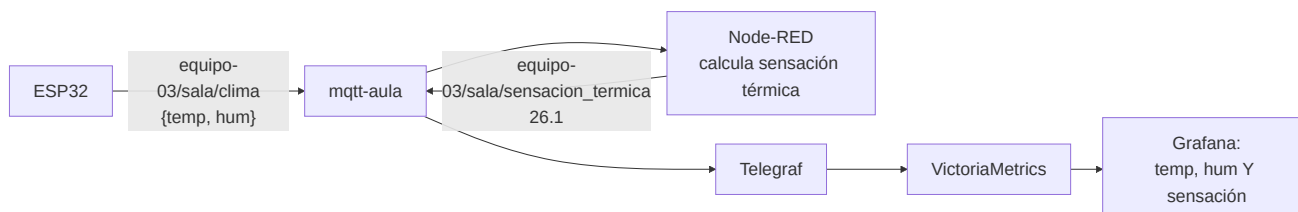
- **Recibir una orden de una persona y mandarla a la placa:** el interruptor del enchufe de Jorge y los botones de la página de Mijael (Node-RED recibe el clic y publica `ON`).
- **Reaccionar a una condición:** "si la puerta de Jessi queda abierta más de 5 minutos, publicá `ON` en el LED de alarma".
- **Transformar:** la placa manda grados Fahrenheit y vos querés Celsius; o manda un número crudo del sensor y hay que convertirlo.
- **Combinar:** "prendé el ventilador si la temperatura pasa de 28 y hay alguien en la sala".
- **Avisar:** mandar un mensaje cuando algo pasa.

Por qué Node-RED y no Grafana: Grafana no recibe órdenes ni ejecuta reglas sobre cada mensaje. Su trabajo es consultar lo guardado y mostrarlo.

Caso C — Los dos: Node-RED procesa, Grafana muestra

Cuándo: querés **ver en el tiempo** un valor que **no manda la placa**, sino que hay que **calcular**.

La forma de hacerlo en este sistema es **republicar**: Node-RED calcula el valor nuevo y lo publica en **otro topic de tu equipo**. Como Telegraf escucha todo `equipo-NN/#`, ese valor calculado se guarda y aparece en Grafana como cualquier otro.



Ejemplos:

- **Dato derivado:** sensación térmica a partir de temperatura y humedad.
- **Estado calculado:** "minutos que la puerta lleva abierta", publicado cada minuto en `equipo-01/puerta/minutos_abierta`.
- **Promedio de varios sensores:** tres placas miden la temperatura y Node-RED publica el promedio en `equipo-NN/sala/temp_promedio`.

En Node-RED: nodo `mqtt in` (escucha) → nodo `function` (calcula) → nodo `mqtt out` (publica en el topic nuevo, con el **mismo** usuario MQTT del equipo). El topic nuevo tiene que respetar el [contrato](#).

¿Y si guardo en mi base de datos (PostgreSQL)? Tu Node-RED puede guardar en la base de tu equipo, y sirve para registros (quién, cuándo). Pero hoy el **Grafana del aula no lee esas bases**: para que algo se vea en Grafana, tiene que pasar por MQTT.

La tabla para decidir

Necesito...	Node-RED	Grafana	Ambos	Por qué
ver la temperatura de las últimas horas		✓		historial: lo guarda Telegraf, lo muestra Grafana
un botón que prenda el LED	✓			es una orden : Grafana no manda órdenes
ver cuándo estuvo prendido el enchufe		✓		el estado confirmado ya llega por MQTT
apagar algo si pasa una condición	✓			es una regla que reacciona a cada mensaje
graficar un valor calculado			✓	Node-RED calcula y republica; Grafana lo muestra
convertir unidades antes de guardar			✓	o mejor: que la placa ya mande la unidad correcta
una pantalla con botones para la muestra	✓			dashboard de Node-RED (ver la guía)
mostrar en la muestra cómo evolucionó todo		✓		Grafana del aula (cuando esté activo)
avisar si la placa se desconectó	✓	✓		Node-RED reacciona al <code>offline</code> ; Grafana muestra hace cuánto

No es una regla absoluta. Node-RED también tiene gráficos en su dashboard, y para una demo rápida en vivo pueden alcanzar. La diferencia es que Node-RED **no guarda historial** (si recargás la página, empieza de cero) y Grafana sí.

Para la muestra

Lo más completo es **mostrar las dos pantallas**, porque cuentan dos partes de la historia de tu sistema:

Pantalla	Qué cuenta	Cómo se abre
Dashboard de Node-RED (o tu página)	"puedo controlar mi sistema en vivo"	túnel SSH → <code>http://localhost:1880/dashboard</code>
Grafana del aula	"mi sistema registra lo que pasa y lo puedo analizar"	<code>https://grafana-aula.lucasland.duckdns.org</code> (cuando esté activo)

Ahora deberías entender

- **Node-RED** recibe eventos, decide, transforma y **actúa**. **Grafana** consulta lo guardado y **muestra**.
- En el aula, los datos llegan a Grafana **directo desde el broker**, sin pasar por Node-RED.
- Para graficar algo calculado, Node-RED lo **republica** en un topic de tu equipo.

Seguí por acá:

- Si querés **armar tu dashboard de Grafana** → [capítulo 13](#).
- Si querés **abrir tu Node-RED** → [guía del equipo, paso 1.2](#).
- Si querés ver **cómo lo resolvieron otros equipos** → [capítulo 14](#).
- Si **algo no aparece** → [Diagnóstico](#).

13. Usar Grafana (manual rápido)

 **Objetivo:** entrar al Grafana del aula, entender tu dashboard y **modificarlo:** agregar gráficos, cambiarlos y guardarlos.

 **Estado (octubre 2026): el Grafana del aula se está poniendo en marcha.** Todavía no está disponible: el profe avisa cuando lo esté. Mientras tanto ya podés **preparar tu placa** siguiendo las reglas de este capítulo: lo que publiques bien desde ahora va a aparecer solo cuando se active.

 **Prerequisitos:** Tailscale conectado y tus datos llegando ([capítulo 12](#)).

Primero, la idea: el tablero de un auto

El tablero de un auto te muestra velocidad, nafta y temperatura del motor de un vistazo, con agujas y luces de colores. No maneja el auto: **te deja ver** cómo anda. **Grafana** es eso para tus placas.

Palabras de este capítulo:

- **Dashboard (tablero):** una pantalla con varios gráficos.
- **Panel:** cada cuadro del tablero (un gráfico, un número grande, una tabla).
- **Consulta (query):** la pregunta que el panel le hace a la base de datos ("dame la temperatura de la sala de las últimas 3 horas").
- **Datasource (fuente de datos):** la base de datos a la que se le pregunta. La tuya se llama **MQTT — equipo-NN** y solo tiene datos de tu equipo.
- **PromQL:** el idioma en que se escriben esas consultas. No hace falta aprenderlo entero: abajo hay un **recetario** para copiar.
- **Serie temporal:** una lista de valores con su hora, como una planilla "hora | valor".

1. Entrar

1. Con **Tailscale conectado**, abrí en el navegador:

```
https://grafana-aula.lucasland.duckdns.org
```

2. Te pide **usuario y contraseña del aula** (los mismos de SSH). Entrás directo a Grafana, no hay otro login.

Si el navegador dice que no encuentra la página: Tailscale no está conectado.

Qué podés ver: solo la carpeta de **tu equipo**. Las de los otros equipos no aparecen.

2. Encontrar tu dashboard

Menú (≡ arriba a la izquierda) → **Dashboards** → carpeta **equipo-NN** → **equipo-NN — sensores y actuadores**.

Un **dashboard** es una pantalla con varios **paneles** (cada cuadro es un panel).

3. Leer el dashboard

Panel	Qué te dice
Dispositivos reportando	cuántas placas mandaron algo en los últimos 5 minutos. Verde = hay datos
Último mensaje hace	cuánto pasó desde el último dato. Rojo = hace más de 15 minutos que no llega nada
Estado actual	tabla con el último valor de cada dispositivo y magnitud
Actuadores y estados	barras de color en el tiempo: verde = ON/abierto, gris = OFF/cerrado
Todas las magnitudes	gráfico de líneas con la historia de cada valor
Mensajes por minuto	cuánto publica cada placa (sirve para detectar bucles)

Mover el tiempo (arriba a la derecha):

- El reloj 🕒 elige el rango: *Last 15 minutes*, *Last 24 hours*, etc.
- Hacé **clic y arrastrá** sobre un gráfico para hacer zoom en ese tramo.
- La flecha circular ↻ actualiza. El dashboard ya se actualiza solo cada 10 s.

Pasá el mouse sobre un gráfico: ves el valor exacto en ese momento.

4. Editar un panel

1. Pasá el mouse sobre el panel → menú ⋮ (arriba a la derecha del panel) → **Edit**.
2. Abajo está la **consulta** (*query*). Arriba, cómo se ve.
3. A la derecha, las **opciones** de visualización:
 - **Title:** el nombre del panel.
 - **Standard options** → **Unit:** la unidad (°C = *Temperature* → *Celsius*, % = *Misc* → *Percent (0-100)*).

- **Thresholds:** colores según el valor (ej. rojo arriba de 30).
- 4. **Back to dashboard** (arriba) para volver.
- 5. **Save dashboard** (arriba a la derecha) para guardar. **Si no guardás, se pierde.**

5. Agregar un gráfico nuevo

1. Arriba → **Add** → **Visualization**.
2. En **Data source** elegí **MQTT — equipo-NN** (el de tu equipo).
3. En la consulta, cambiá a modo **Code** y escribí qué querés ver. Usá el **recetario** de abajo.
4. Elegí el tipo de gráfico a la derecha (arriba de las opciones):

Tipo	Para qué
Time series	valores que cambian en el tiempo (temperatura, humedad)
Stat	un número grande (temperatura actual)
Gauge	un reloj de aguja (nivel de tanque, %)
State timeline	encendido/apagado en el tiempo (relé, puerta)
Table	lista de valores

5. Ponele título, **Back to dashboard** y **Save dashboard**.

6. Recetario de consultas

Tus datos se llaman siempre `mqtt_valor` y tienen dos **etiquetas** (datos que acompañan a cada valor para poder filtrarlo, como las columnas de una planilla): `dispositivo` y `magnitud`, que son las partes 2 y 3 de tu topic.

Cómo se lee una consulta: `mqtt_valor{dispositivo="sala"}` quiere decir "los valores **cuyo** dispositivo sea `sala`". Las llaves `{}` son el filtro. `avg_over_time(...[1h])` quiere decir "el **promedio** (*average*) de la última hora".

Quiero ver...	Consulta
todo lo de mi equipo	<code>mqtt_valor</code>
una magnitud de un dispositivo	<code>mqtt_valor{dispositivo="sala", magnitud="temperatura"}</code>
todo lo de un dispositivo	<code>mqtt_valor{dispositivo="sala"}</code>
el promedio de la última hora	<code>avg_over_time(mqtt_valor{magnitud="temperatura"}[1h])</code>
el máximo del día	<code>max_over_time(mqtt_valor{magnitud="temperatura"}[24h])</code>
cuántas veces se abrió la puerta hoy	<code>changes(mqtt_valor{magnitud="reed"}[24h]) / 2</code>
hace cuánto llegó el último dato (s)	<code>time() - tlast_over_time(mqtt_valor{dispositivo="sala"}[15d])</code>
si la placa está conectada (1/0)	<code>mqtt_valor{magnitud="conexion"}</code>

Para que la leyenda diga algo útil en vez de todas las etiquetas: en **Options** → **Legend** poné `{{dispositivo}} · {{magnitud}}`.

7. Buenas prácticas

- **Antes de cambiar mucho, hacé una copia:** ⚙️ (Settings) → **Save as** → ponele otro nombre. Si rompés algo, tenés el original.
- **Guardá seguido.** Grafana guarda versiones: ⚙️ → **Versions** te deja volver a una anterior.
- **Un panel = una idea.** Mejor varios paneles chicos que uno con 20 líneas.
- **Poné unidades.** "23.5" no dice nada; "23.5 °C" sí.

8. Preguntas frecuentes

No veo mi carpeta. Puede ser que todavía no entraste nunca a Grafana o que te cambiaron de equipo. Avisale al profe.

El panel dice "No data". Revisá que la placa esté publicando (tabla **Estado actual**) y que la consulta tenga bien escritos `dispositivo` y `magnitud` (son en minúsculas, con `_` en vez de espacios o guiones).

Me equivoqué y borré un panel. Si no guardaste: recargá la página sin guardar. Si ya guardaste: ⚙️ → **Versions** → elegí la anterior → **Restore**.

¿Puedo prender el LED desde Grafana? No: Grafana es para **ver**. Las órdenes se mandan desde Node-RED o tu página.

Ahora deberías entender

- Qué es un **dashboard**, un **panel**, una **consulta** y un **datasource**.
- Que Grafana **consulta lo guardado** (no recibe los mensajes directo): por eso muestra historia, y por eso no prende LEDs.

Seguí por acá:

- Si un panel dice "**No data**" → [Diagnóstico, paso 9](#).
- Si querés graficar un valor **calculado** (no lo manda la placa) → [Caso C](#).
- Si querés entender **de dónde salen** los datos que ves → [Seguí un dato](#).

14. Los proyectos de los equipos (casos de diseño)

 **Objetivo:** mirar los proyectos reales del aula con los ojos del [capítulo 11](#): qué pieza va en cada capa, qué decisiones se tomaron, **qué falló y qué aprendimos**. Son proyectos hechos por ustedes, con errores reales: por eso enseñan más que un ejemplo inventado.

 **Prerequisitos:** [capítulo 11 \(Arquitectura IoT\)](#).

 **Conceptos nuevos:** caso de diseño, evolución de un contrato, firmware modular, dashboard propio.

Los proyectos son de sus autores: **Mijael** (equipo-03), **Jessi** (equipo-01) y **Jorge** (equipo-04), con sus compañeros de equipo. Se cuentan tal como pasaron, errores incluidos.

Vista rápida

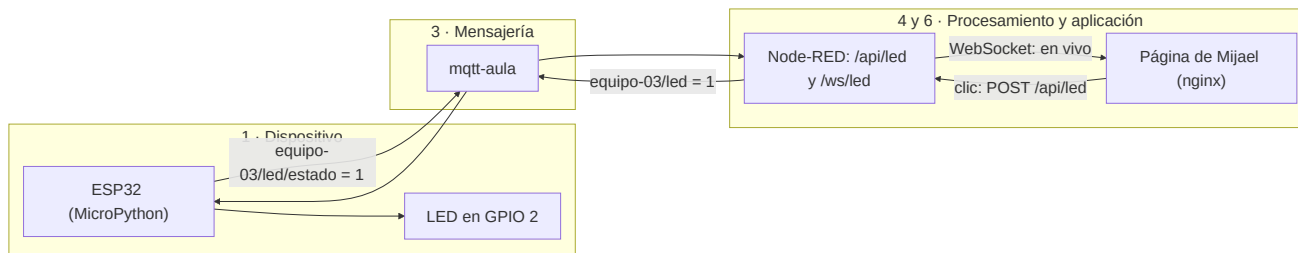
	Mijael · equipo-03	Jessi · equipo-01	Jorge · equipo-04
Qué hace	prende y apaga un LED desde la web	avisa si una puerta está abierta o cerrada	enchufe inteligente: prende y corta un aparato
Tipo	actuador	sensor	actuador
Lenguaje	MicroPython	MicroPython	Arduino (C++)
Pieza física	LED en GPIO 2	<i>reed switch</i> en GPIO 27 + LED azul de la placa como testigo	relé (simulado con un LED) en GPIO 5 + LED de "vivo" en GPIO 4
Mensajes	1 / 0	abierto / cerrado	ON / OFF
Interfaz	dashboard de Node-RED + página propia (nginx)	dashboard de Node-RED	dashboard de Node-RED + página propia (nginx)

Caso 1 — El LED de Mijael (equipo-03)

Qué hace

Desde el navegador, un botón prende o apaga un LED que está en la placa. La página muestra el estado **que confirma la placa**, en vivo.

Las capas



Palabras nuevas del dibujo:

- **nginx** (se pronuncia "engine-x"): un **servidor web**, el programa que le entrega la página (los archivos HTML) al navegador.
- `/api/led`: una dirección dentro de Node-RED que recibe la orden de la página. **API** es la puerta acordada para que dos programas se hablen.
- **WebSocket**: una conexión que **queda abierta** entre la página y el servidor, para que los cambios lleguen solos sin recargar. Es como una llamada telefónica abierta en vez de mandar cartas.

Decisiones que se tomaron

- **Orden y confirmación separadas** (decisión 3 del capítulo 11): la página no pinta el foco cuando hacés clic, sino cuando la placa avisa por `equipo-03/led/estado`.
- **La página habla con Node-RED, no con el broker**: el navegador no tiene que saber nada de MQTT ni tener la clave. Node-RED hace de **intermediario**.

Qué falló y qué aprendimos

Qué pasó	Por qué	Qué aprendimos
<code>OSError: Wifi Internal State Error</code> al reiniciar	un reinicio "suave" dejó el WiFi a medio conectar	apagar y prender el WiFi antes de conectar (caso 4)
<code>Error ... 5</code> en cada intento	en <code>MQTT_PASS</code> estaba la clave de entrar al servidor , no la del broker	son dos claves distintas (caso 5)
<code>MBEDTLS_ERR_SSL_CONN_EOF</code>	una de las dos "puertas" de internet de Funnel estaba caída	el problema no siempre es tuyo: medir antes de cambiar código (caso 7)

En Grafana

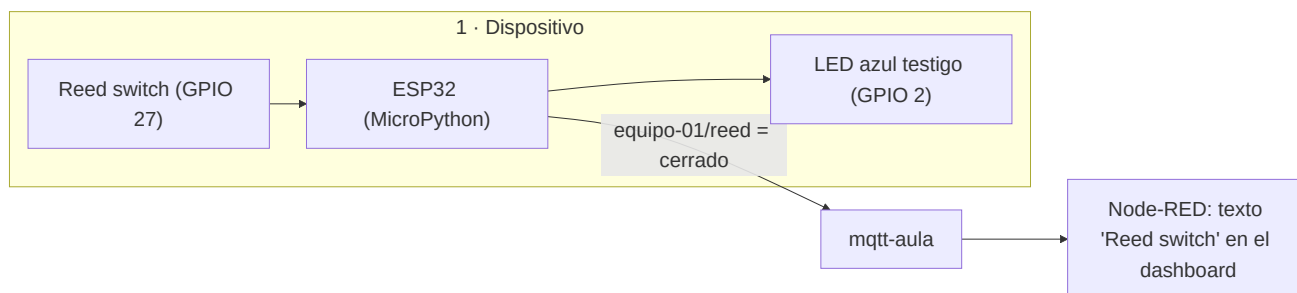
`equipo-03/led/estado` cumple el contrato (3 partes) → aparece como dispositivo **led**, magnitud **estado**. En cambio la orden `equipo-03/led` tiene **2 partes** y no se guarda. Para la página no importa, pero si Mijael quiere graficar también las órdenes, conviene pasarla a `equipo-03/led/cmd`.

Caso 2 — La puerta de Jessi (equipo-01)

Qué hace

Un **reed switch** (un interruptor que se cierra con un imán) detecta si una puerta está cerrada. El LED azul de la placa hace de **testigo**: prendido = cerrada. El estado viaja al dashboard.

Las capas



Decisiones que se tomaron

- **Pull-up interno:** el pin se "tira" hacia arriba por dentro de la placa, así no hace falta una resistencia externa. Cerrado (imán cerca) = `0`; abierto = `1`.
- **Antirrebote:** un contacto mecánico, al cerrarse, "rebota" varias veces en milésimas de segundo. El código **lee dos veces con 30 ms** de diferencia y solo cree si coinciden.
- **Publicar solo cuando cambia, y retenido:** no llena el broker de mensajes iguales, y quien abre el dashboard tarde igual ve el último estado.
- **Testigo local:** el LED azul funciona **aunque no haya WiFi**. Si la red se cae, la placa sigue cumpliendo su función básica (decisión 5: diseño a prueba de fallos).

Qué falló y qué aprendimos

Qué pasó	Por qué	Qué aprendimos
Error, reintento en 5 s: 5 sin parar	clave MQTT equivocada (la del otro equipo)	cada equipo tiene la suya (caso 5)
& was unexpected at this time en Windows	el comando era para PowerShell y se corrió en CMD	cada terminal tiene su sintaxis

En Grafana: un contrato que tiene que evolucionar

El topic `equipo-01/reed` tiene **2 partes**: cumple con el broker pero **no** con el contrato de Grafana (`equipo/dispositivo/magnitud`). Por eso **todavía no aparece** en el dashboard de Grafana. El cambio es de una línea:

```
T_REED = b"equipo-01/puerta/reed"
```

y en Node-RED, cambiar el topic del nodo que lo escucha. Esto se llama **evolución de un contrato**: cuando las reglas cambian, hay que actualizar a **los dos lados** (el que publica y el que escucha) al mismo tiempo.

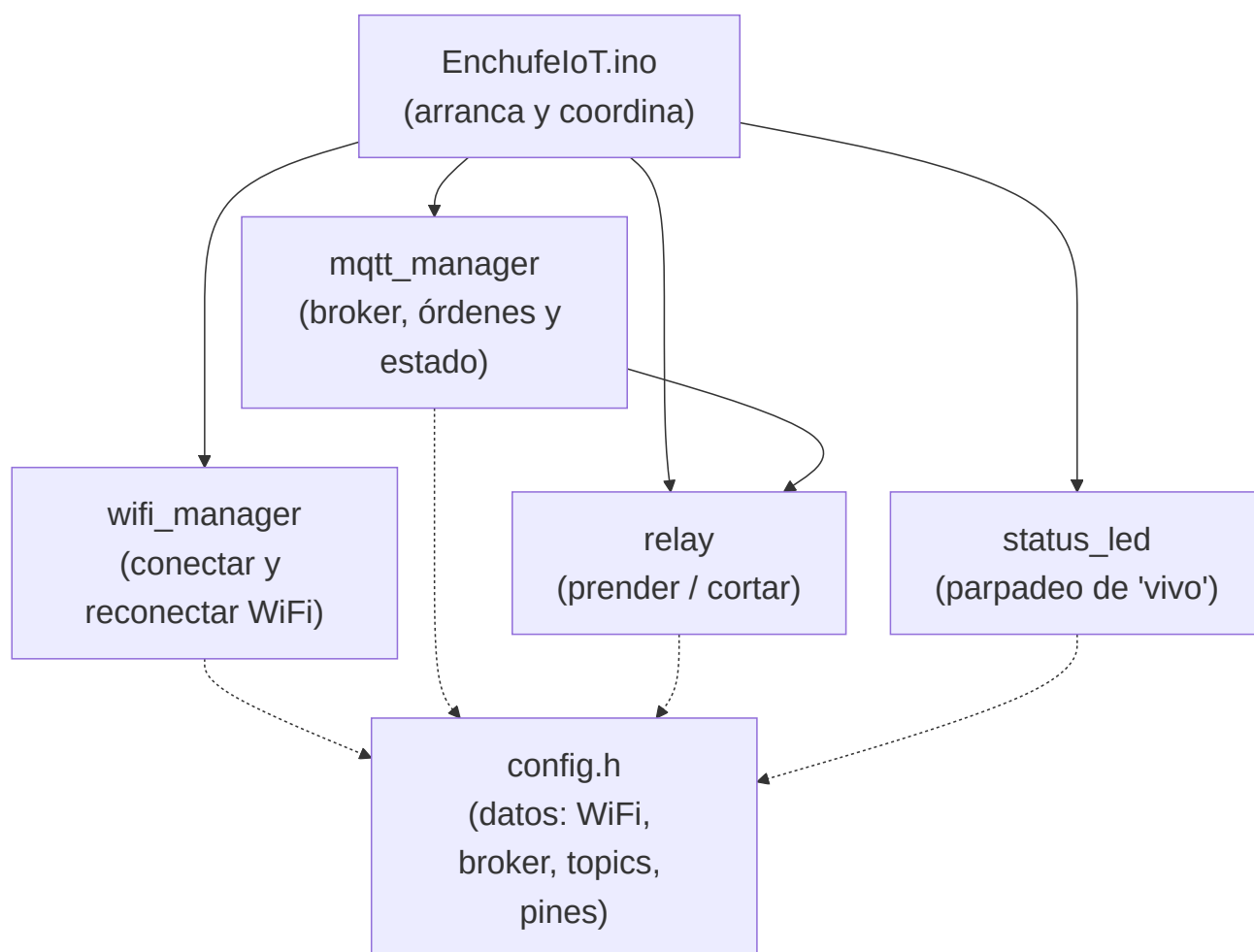
Caso 3 — El enchufe de Jorge (equipo-04)

Qué hace

Un enchufe inteligente: desde la web se prende o se corta un aparato. Un relé (acá simulado con un LED) hace el corte, y otro LED parpadea para mostrar que el sistema "está vivo".

Un firmware bien organizado

Jorge separó el programa en **módulos**, cada uno con un solo trabajo. Es la misma idea de las capas, pero adentro de la placa:



Por qué está bueno: para cambiar de broker se tocó **solo** `config.h` y `mqtt_manager`. El relé, el LED y el WiFi no se enteraron. Eso es **desacople** adentro del firmware.

Qué falló y qué aprendimos

Qué pasó	Por qué	Qué aprendimos
Error 4 y en el servidor no aparecía ningún intento	config.h seguía apuntando a otro broker (192.168.18.17:1883), sin cifrado ni clave	si el servidor no ve tus intentos, tu placa está hablando con otro lado (caso 6)
El dashboard mandaba 1 / 0 y la placa esperaba ON / OFF	las dos puntas no habían acordado el formato de los mensajes	el contrato se acuerda antes (caso 8)

En Grafana

equipo-04/enchufe/estado = ON → se guarda como **1**; OFF → **0**. Aparece en el panel **Actuadores y estados** como barras verdes y grises en el tiempo: se ve exactamente cuándo estuvo prendido el aparato.

Lo que tienen en común (y lo que aprendimos como grupo)



Ejercicios

1. Elegí **uno** de los tres proyectos y proponé una mejora usando una de las 9 decisiones del capítulo 11.

2. Jessi tiene que pasar de `equipo-01/reed` a `equipo-01/puerta/reed`. Listá **todo** lo que hay que cambiar, en qué archivo y en qué máquina.
 3. Dibujá el viaje de un mensaje de **tu** proyecto como el diagrama del caso 1.
-

Ahora deberías poder

- Ubicar cada pieza de **tu** proyecto en las capas del capítulo 11.
- Explicar qué decisiones tomó tu equipo y qué aprendieron de sus errores.

Seguí por acá:

- Si tu proyecto necesita **reglas o botones** además de gráficos → [¿Node-RED, Grafana o ambos?](#).
- Si querés ver **tu proyecto en su ciclo de vida** (del código a la muestra) → [capítulo 15](#).
- Si algo de tu proyecto **dejó de andar** → [Diagnóstico](#).

Parte IV — Operar y evolucionar

Diagnóstico: ¿dónde se cortó el recorrido?

 **Objetivo:** que puedas encontrar **vos mismo**, paso a paso, en qué punto del camino se pierde tu dato (o tu orden), incluso en medio de la muestra.

 **Antes de empezar:** el recorrido completo está explicado en [el capítulo 11, "Seguí un dato"](#). Acá lo recorremos **al revés**: buscando dónde se rompió.

Palabras que vas a usar: [broker](#) · [topic](#) · [payload](#) · [MQTT Explorer](#) · [túnel](#) · [datasource](#).

Elegí tu problema:

-  [Mi ESP32 mide, pero no veo los datos](#)
-  [Aprieto el botón y mi actuador no responde](#)
-  [No puedo entrar a algo \(servidor, Node-RED, Grafana\)](#)
-  [Chequeo rápido antes de la muestra](#)

La herramienta clave: escuchar el broker desde tu compu

Casi todo el diagnóstico se resuelve con una pregunta: **¿el mensaje llegó al broker?** Si llegó, el problema está "después" (Node-RED, Grafana). Si no llegó, está "antes" (la placa, el WiFi, la clave).

Para responderla sin pedirle nada al profe, usá **MQTT Explorer**: un programa gratuito (Windows, Linux, Mac) que se conecta al broker **como si fuera otra placa de tu equipo** y te muestra todo lo que pasa en tus topics. Descargalo de <https://mqtt-explorer.com> y configuralo así:

Campo	Valor
Protocolo	mqtt:// con Encryption (tls) activado
Host	homelab-01.tail4eda13.ts.net
Port	10000
Username	equipo_NN (con guion bajo)
Password	tu MQTT_PASSWORD (de <code>.shared-services.env</code>)
<i>Advanced</i> → Topic	borrá los que vienen y poné <code>equipo-NN/#</code> (con guion)

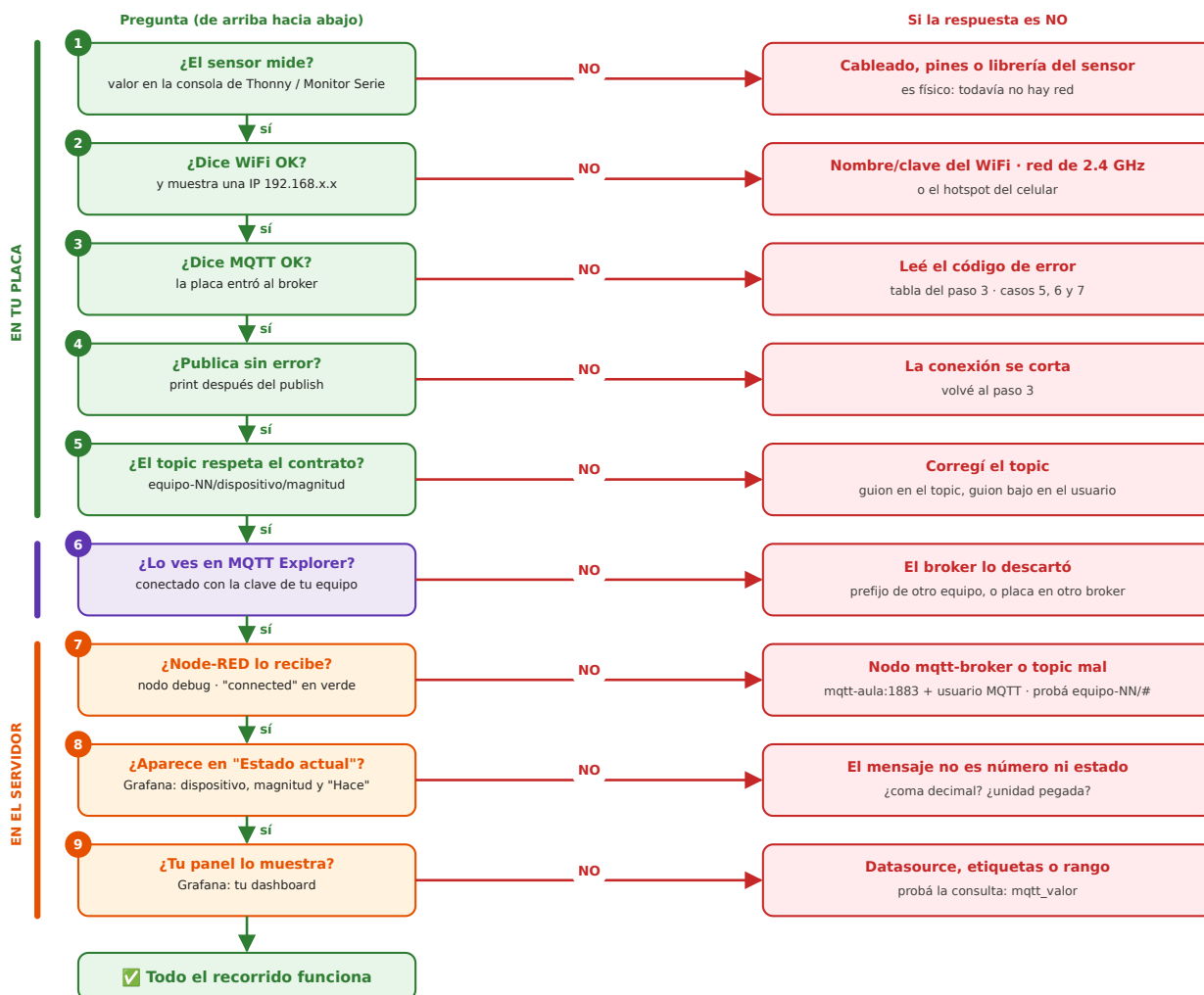
Conectá. Cada mensaje de tu equipo aparece en el árbol, con su valor y la hora.

No necesitás Tailscale para esto: MQTT Explorer entra por el mismo camino que tu placa (Funnel). Y como usa la clave de tu equipo, solo ve **tus** topics.

Mi ESP32 mide, pero no veo los datos

No veo mi sensor: ¿dónde se cortó?

Bajá por la izquierda mientras la respuesta sea SÍ. Al primer NO, ese es tu problema: andá a la derecha.



Paso 1 — ¿El sensor mide?

- **Qué compruebo:** que la placa **lee** el sensor, antes de cualquier red.
- **Cómo:** agregá un `print(valor)` (MicroPython) o `Serial.println(valor)` (Arduino) justo después de leer, y mirá la consola de Thonny o el Monitor Serie (115200 baudios).
- **Qué debería pasar:** un número razonable (24.7 , no 0 , -127 ni nan).
- **Si falla:** es un problema **físico o de la librería**: cables, pin equivocado, alimentación (3.3 V o 5 V), librería del sensor. Todavía no tiene nada que ver con el servidor.

Paso 2 — ¿La ESP32 tiene WiFi?

- **Qué compruebo:** que la placa se conectó a la red y tiene una dirección IP.
- **Cómo:** en la consola tiene que aparecer `WiFi OK` y una IP (`192.168.x.x`).
- **Qué debería pasar:** aparece en pocos segundos.
- **Si falla:**
 - se queda en `Conectando...` → nombre o clave del WiFi mal, o la red es de **5 GHz** (la ESP32 solo ve **2.4 GHz**);
 - `Wifi Internal State Error` → [caso 4](#);
 - red del colegio que pide usuario en una página → la ESP32 no la puede usar: usá el hotspot del celular en 2.4 GHz.

Paso 3 — ¿Llega al broker?

- **Qué compruebo:** que la placa **sale a internet**, llega al portero (Funnel) y el broker **la acepta**.
- **Cómo:** la consola tiene que decir `MQTT OK` . Si no, el **código de error** dice qué pasó:

Lo que ves	Qué significa	Ir a...
<code>5 / not authorised / 4</code> (Arduino)	llegó , pero usuario o clave MQTT mal	caso 5 · tabla de credenciales
<code>-2</code> (Arduino) / <code>ECONNREFUSED</code> / no resuelve	no llega: host, puerto o internet	revisá <code>homelab-01.tail4eda13.ts.net</code> y puerto <code>10000</code>
<code>-4</code> (Arduino)	conectó sin cifrado a un puerto que lo exige	usá <code>WiFiClientSecure</code> + <code>setInsecure()</code> (cap. 12)
<code>MBEDTLS_ERR_SSL_CONN_EOF</code>	cortaron la conexión mientras se armaba el cifrado	caso 7
error y el profe no ve ningún intento en el broker	tu placa habla con otro broker	caso 6 : revisá <code>MQTT_BROKER</code>

- **Qué debería pasar:** `MQTT OK` una vez, y que no se repita cada pocos segundos (si se repite, se está desconectando y reconectando).

Paso 4 — ¿Publica?

- **Qué compruebo:** que el código **llega** a la línea del `publish` y no tira error.
- **Cómo:** poné un `print("publiqué", valor)` justo después del `publish`.
- **Qué debería pasar:** aparece cada vez que mide (por ejemplo, cada 10 segundos).
- **Si falla:** si aparece `Error, reintento...` después de publicar, la conexión se corta: volvé al paso 3. Si nunca llega al `print`, el programa se quedó antes (¿un `while` sin salida?).

Paso 5 — ¿El topic es correcto?

- **Qué compruebo:** que el topic respeta el **contrato** del aula.
- **Cómo:** comparará tu topic con estas reglas:

✓ Bien	✗ Mal	Por qué
equipo-04/sala/temperatura	equipo_04/sala/temperatura	el topic va con guion ; el guion bajo es el usuario
equipo-04/sala/temperatura	equipo-04/temperatura	faltan partes: ¿de qué dispositivo?
equipo-04/enchufe/estado	casa/enchufe/estado	no empieza con tu equipo: el broker lo tira

- **Si falla:** corregí el topic en el código **y** en Node-RED si lo escucha ([reglas completas](#)).

Paso 6 — ¿El servidor lo recibe?

- **Qué compruebo:** que el mensaje **entró al broker** del servidor.
- **Cómo:** abrí **MQTT Explorer** ([arriba](#)) y buscá tu topic en el árbol.
- **Qué debería pasar:** aparece tu topic con el valor que mandó la placa, y se actualiza cuando mide de nuevo.
- **Si falla** (la placa dice que publicó pero MQTT Explorer no lo ve):
 - el topic **no empieza con tu equipo** → el broker lo descarta **sin avisar**;
 - la placa usa el usuario de **otro** equipo (mira otro carril);
 - la placa se conectó a **otro broker** ([caso 6](#)).

Paso 7 — ¿Node-RED lo recibe?

Solo si tu proyecto usa Node-RED para este dato. Para que llegue a Grafana no hace falta ([por qué](#)).

- **Qué compruebo:** que tu Node-RED está conectado al broker y escucha el topic.
- **Cómo:** abrí tu Node-RED por el túnel ([guía, paso 1.2](#)). Abajo del nodo `mqtt in` tiene que decir **"connected"** en verde. Conectale un nodo `debug` y mirá la pestaña de debug (el ícono del bichito, a la derecha).
- **Qué debería pasar:** cada mensaje aparece en el panel de debug.
- **Si falla:**
 - dice **"disconnected"** → el nodo `mqtt-broker` tiene que apuntar a `mqtt-aula`, puerto `1883`, con el usuario y la clave MQTT del equipo;
 - está "connected" pero no llega nada → el topic del nodo no coincide (probá `equipo-NN/#` para escuchar todo lo de tu equipo).

Paso 8 — ¿Llega al almacenamiento?

🔧 El Grafana del aula se está poniendo en marcha. Mientras no esté activo, el recorrido verificable termina en el paso 6 o 7.

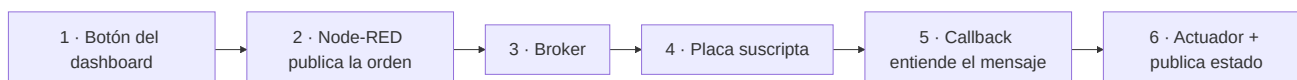
- **Qué compruebo:** que el valor se **guardó** (Telegraf lo tradujo y lo escribió).
- **Cómo:** en Grafana, tu dashboard → tabla **Estado actual**.
- **Qué debería pasar:** aparece tu **dispositivo** y tu **magnitud** con el último valor, y la columna **Hace** en pocos segundos.
- **Si falla** (MQTT Explorer lo ve pero Grafana no):
 - el mensaje **no es un número ni una palabra de estado**: 23,5 (coma), 23.5C (unidad pegada) o un texto libre se descartan ([regla 2](#));
 - el topic tiene **menos de 3 partes** (se descarta).

Paso 9 — ¿Grafana consulta lo correcto?

- **Qué compruebo:** que **tu panel** pregunta por tu dato.
- **Cómo:** edita el panel (menú : → *Edit*) y revisa tres cosas: 1. **Data source:** MQTT – equipo-NN (el de **tu** equipo); 2. **Consulta:** `mqtt_valor{dispositivo="sala", magnitud="temperatura"}`, con los nombres **en minúsculas** y con `_` donde había espacios o guiones; 3. **Rango de tiempo** (arriba a la derecha): que incluya el momento en que publicaste.
- **Si falla:** proba la consulta más simple, `mqtt_valor`: si así aparece, el problema está en el filtro ([recetario](#)).

Aprieto el botón y mi actuador no responde

El camino de una **orden** es al revés: de la pantalla a la placa ([el enchufe de Jorge, paso a paso](#)).

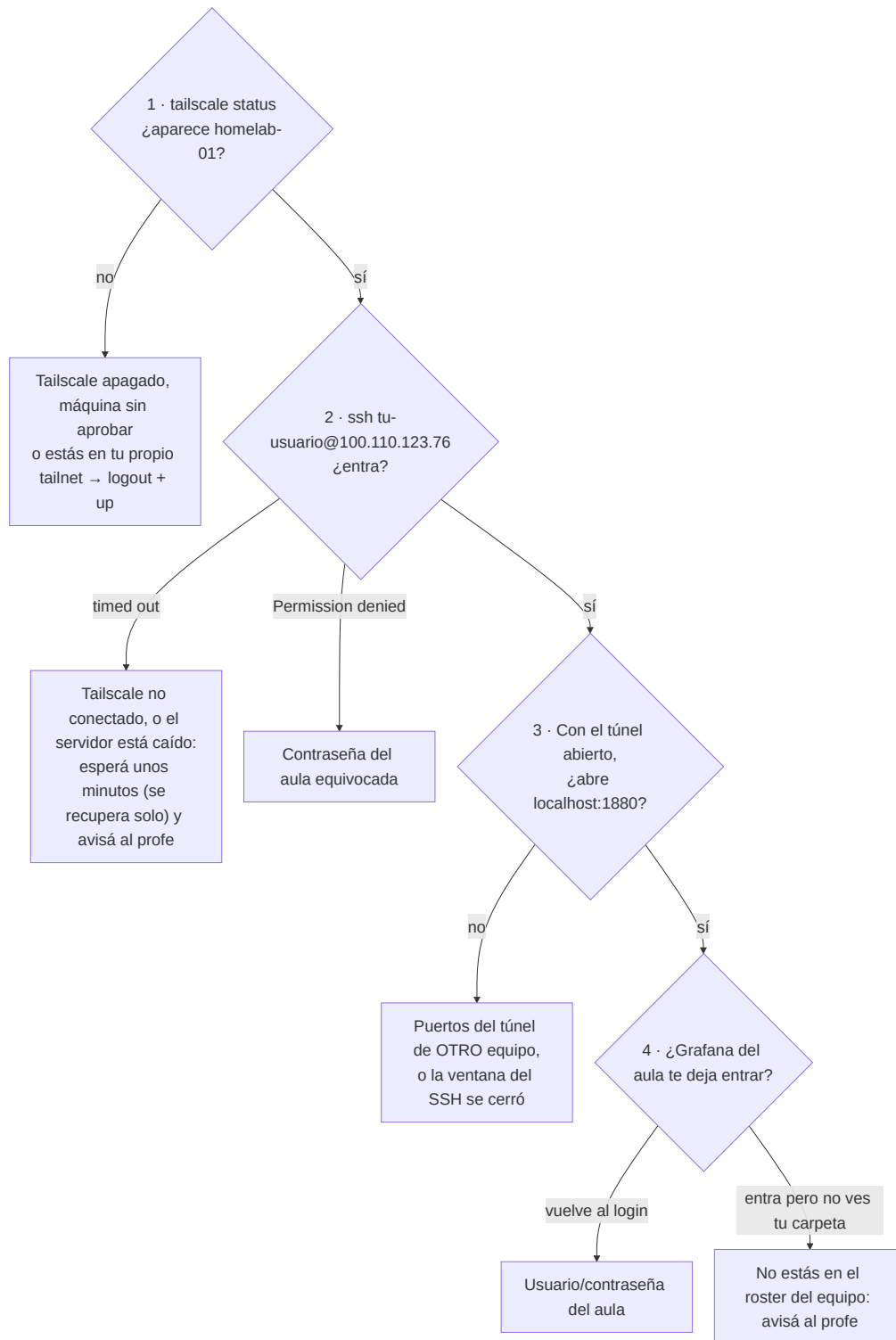


Paso	Qué compruebo	Cómo	Si falla
1-2	¿Node-RED manda la orden?	MQTT Explorer: ¿aparece <code>equipo-NN/.../cmd</code> al apretar?	nodo <code>mqtt out</code> sin conectar o con otro topic
3-4	¿La placa está escuchando ese topic?	en el código, el <code>subscribe</code> usa exactamente el mismo topic	corregir el topic (mayúsculas, guiones)
5	¿La placa entiende el mensaje?	<code>print</code> del mensaje recibido en el callback	el formato no coincide: <code>ON</code> contra <code>1</code> (caso 8)
6	¿El actuador se mueve y confirma?	el LED/relé cambia; aparece <code>.../estado</code> en MQTT Explorer	pin equivocado; no publica el estado confirmado

Si el dashboard muestra **"PRENDIDO"** pero la placa está apagada, el dashboard está mostrando la **orden** y no la **confirmación**: ver la [decisión 3 del capítulo 11](#).

No puedo entrar a algo

Las cuatro puertas están explicadas en [Red y accesos](#). Se revisan **en orden**: si falla una, las de abajo no tienen sentido todavía.



Mensaje	Qué puerta	Qué hacer
Connection timed out	1 · red	tailscale status ; si no ves homelab-01 , guía paso 1.1
Permission denied	2 · servidor	es tu contraseña del aula , no la de Google ni la MQTT
bind: Address already in use	túnel	ya usás ese puerto en tu compu: cambiá el número de la izquierda (-L 1890:...)
El navegador "no puede conectar" a localhost	túnel	la ventana del SSH se cerró: abrila de nuevo y dejala abierta
Grafana vuelve al login	3 · servicio	usuario y contraseña del aula
& was unexpected at this time	(tu terminal)	el comando era para PowerShell y estás en CMD: sin el &

Chequeo rápido antes de la muestra

Cinco minutos antes, en este orden:

- ☐ La placa está enchufada y en la consola dice **WiFi OK** y **MQTT OK**.
- ☐ En **MQTT Explorer** ves tus topics actualizándose.
- ☐ El **túnel SSH** está abierto en una ventana que no vas a cerrar.
- ☐ Tu **dashboard de Node-RED** (<http://localhost:1880/dashboard>) o tu página (<http://localhost:8080>) responde, y el botón mueve el actuador.
- ☐ (Si está activo) tu tablero de **Grafana** muestra el último valor hace pocos segundos.
- ☐ Tenés a mano: la **red WiFi** que usa la placa (¿es la misma del lugar de la muestra? ¿es de 2.4 GHz?) y el **hotspot del celular** como plan B.

Plan B si se cae la red del lugar: el hotspot del celular (2.4 GHz) para la placa y para tu compu. Como todo pasa por internet (Funnel y Tailscale), no importa en qué red estés: si hay internet, funciona.

Ahora deberías poder

- Ubicar en qué **tramo** se cortó tu dato: placa, WiFi, broker, topic, Node-RED, almacenamiento o panel.
- Usar **MQTT Explorer** para saber si el problema está antes o después del broker.
- Distinguir un problema de **red**, de **credenciales** o de **contrato**.

Seguí por acá:

- Para entender **por qué** el recorrido es así → [capítulo 11](#).

- Para ver **historias reales** de estos errores → [capítulo 10](#).
- Si el problema es **quién te deja entrar** → [Red y accesos](#).

10. Casos prácticos

 **Objetivo:** aprender a **diagnosticar y resolver** problemas reales, con recorridos paso a paso. Es el capítulo "de taller".

¿Tenés un problema ahora? Este capítulo cuenta **historias** de errores reales. Para buscar **tu** problema paso a paso, andá al **Diagnóstico**, que te trae a estos casos cuando corresponde.

 **Prerequisitos:** todos los anteriores (se usan como referencia).

 **Capítulo en crecimiento.** Tiene once casos reales: cinco del servidor y seis de las placas de los equipos. Se suman más a medida que pasan.

Método general de diagnóstico

1. **¿Qué síntoma veo exactamente?** (mensaje de error, código HTTP, "se cuelga").
2. **¿Dónde?** (navegador, terminal, un servicio puntual).
3. **Reproducir** de forma mínima.
4. **Aislar la capa:** ¿es red, DNS, permisos, contenedor, config?
5. **Confirmar la causa** con un comando, no con una corazonada.
6. **Arreglar y demostrar** que quedó resuelto.

Caso 1 — "Grafana se queda pensando" (DNS / `/etc/hosts`)

Síntoma: el navegador gira infinito al abrir un servicio.

Diagnóstico: el `/etc/hosts` apuntaba a la **IP de LAN** (`192.168.100.48`), inalcanzable desde otra red; y el subdominio `auth.` resolvía a la **IP pública** (sin port-forward). El navegador redirige al login y no puede resolverlo.

Solución: apuntar **todos** los subdominios (incluido `auth.`) a la **IP del tailnet** en `/etc/hosts` (o, mejor, configurar Split DNS en Tailscale para no tocar `/etc/hosts`).

Lección: cuando "se cuelga" (timeout, no "error"), sospechá de **red/DNS** antes que de la app.

Caso 2 — Un cambio de config no tiene efecto (el inode del bind mount)

Síntoma: editás el Caddyfile y aplicás, pero Caddy sigue con la config vieja.

Diagnóstico: Ansible reemplaza el archivo de forma atómica → **nuevo inode**. El contenedor tenía montado el archivo por el **inode viejo**, así que un `caddy reload` leía el viejo.

Solución: **reiniciar el contenedor** (re-resuelve el bind mount). Se corrigió en el rol para que reinicie Caddy solo cuando su config cambia.

Lección: los bind mounts de **archivos** (no carpetas) pueden quedar pegados al inode viejo; ante dudas, **reiniciá el contenedor**.

Caso 3 — Node-RED en bucle de reinicio (EACCES en un bind mount)

Síntoma: después de `labctl up`, `labctl ps` muestra el contenedor de Node-RED como `Restarting` una y otra vez (mientras Mosquitto queda `Up`).

Diagnóstico: mirar los logs del contenedor:

```
docker logs equipo-01-nodered-1 --tail 20
# Error: EACCES: permission denied, copyfile ... -> '/data/settings.js'
```

Node-RED corre **como el uid 1000** adentro del contenedor, pero la carpeta que se le montó en `/data` (el bind mount `./data/nodered`) se sembró como `root`. El uid 1000 no es dueño ni del grupo → no puede escribir → se cae al arrancar.

Solución: hacer la carpeta escribible por el uid del contenedor y reiniciar:

```
sudo chown -R 1000:1000 /srv/classroom/equipo-01/data/nodered
sudo -u <alumno> labctl restart
```

Cada imagen corre con **su uid**: Node-RED usa `1000`, Mosquitto `1883`. Al sembrar el proyecto, ajustá el dueño de cada carpeta de datos a ese uid (ver el ejemplo [ejemplos/nodered-mqtt/](#)).

Lección: `EACCES` / "permission denied" ≈ **permisos**. En un bind mount, la carpeta del host tiene que ser escribible por el **uid con el que corre el contenedor**, no por el del alumno.

Casos de la ESP32 y la red (septiembre-octubre 2026)

Los casos que siguen **pasaron en clase**, con las placas de los equipos. Para cada uno: qué se vio, cómo se encontró la causa (casi siempre **mirando del otro lado**: los registros del

servidor) y qué nos llevamos.

Herramienta clave de todos estos casos: el **registro** (*log*) del broker. Es un cuaderno donde el broker anota cada conexión: quién entró, con qué usuario y si lo rechazó. Leerlo es como preguntarle al portero "¿vino alguien?". Lo lee el operador en el servidor con `docker logs mqtt-aula`.

Caso 4 — Wifi Internal State Error después de reiniciar

Síntoma (placa de Mijael, en Thonny):

```
E (912940) wifi:sta is connecting, cannot set config
OSError: Wifi Internal State Error
```

Diagnóstico: apareció después de un **soft reboot** (reinicio "suave": Ctrl+D o el botón Run de Thonny, que reinicia el programa pero **no** apaga el chip). El WiFi había quedado **a mitad de conectarse** del intento anterior, y el código volvió a pedirle `connect()`. El chip contestó "estoy ocupado conectando, no puedo cambiar la configuración".

Solución: antes de conectar, **apagar y prender** el WiFi para empezar de cero:

```
wlan.active(False)
time.sleep(0.5)
wlan.active(True)
wlan.disconnect()
wlan.connect(WIFI_SSID, WIFI_PASS)
```

Lección: no asumas que el aparato arranca "limpio". Llévalo vos a un estado conocido antes de usarlo.

Caso 5 — El error 5 0 not authorised: claves mezcladas

Síntoma: en la placa, Error, reintento en 5 s: 5 cada pocos segundos. En el registro del broker:

```
New connection from 172.21.0.1 on port 1883.
Client esp32-equipo-03 disconnected, not authorised.
```

Diagnóstico: la placa **sí llegaba** al servidor (la conexión aparece en el registro), pero el broker la **rechazaba**. El 5 es el código de MQTT para "no autorizado". En `MQTT_PASS` estaba la contraseña de **entrar al servidor por SSH**, no la del **broker**. Con Jessi pasó parecido: tenía la clave de otro equipo.

Las claves de un alumno, para no confundirlas:

Para qué	Usuario	Clave
Entrar al servidor (SSH) y a Grafana	tu nombre (<code>mijael</code>)	la del aula
Que la placa entre al broker	el del equipo con guion bajo (<code>equipo_03</code>)	<code>MQTT_PASSWORD</code> de <code>.shared-services.env</code>

Lección: si el servidor **ve** tu intento y lo rechaza, el problema es **quién sos** (usuario/clave), no la red.

Caso 6 — El error 4 y en el servidor no aparece nada

Síntoma: el enchufe de Jorge mostraba `Error 4 - Reintentando en 5 segundos`. Pero en el registro del broker **no había ningún intento** de su placa.

Diagnóstico: si la placa estuviera hablando con nuestro broker, el registro lo mostraría (aunque fuera para rechazarla, como en el caso 5). Que no aparezca nada quiere decir que **está hablando con otro lado**. Al leer **todo** el código del proyecto (no solo el archivo que nos pasaron), `config.h` seguía apuntando a otro broker de su red de casa:

```
constexpr char MQTT_BROKER[] = "192.168.18.17"; // otro broker
constexpr int MQTT_PORT = 1883;                // sin cifrado
```

Y `mqtt_manager.cpp` se conectaba sin cifrado (`WiFiClient`) y sin usuario.

Solución: apuntar al broker del aula (`homelab-01.tail4eda13.ts.net`, puerto `10000`), usar el cliente cifrado (`WiFiClientSecure` + `setInsecure()`) y mandar usuario y clave. Ver [capítulo 14](#).

Lección: "no aparece nada del otro lado" es una pista enorme: el mensaje no está llegando al lugar que creés. Y siempre leé **todo** el proyecto, no solo el archivo que parece importante.

Caso 7 — `MBEDTLS_ERR_SSL_CONN_EOF` : una puerta de entrada caída

Síntoma: la placa de Mijael, que andaba, empezó a fallar con:

```
Error, reintentado en 5 s: (-29312, 'MBEDTLS_ERR_SSL_CONN_EOF')
```

Qué significa: `mbedtls` es la librería que hace el cifrado (TLS) en la ESP32. `CONN_EOF` (*End Of File*, fin del archivo) quiere decir "el otro lado cortó la conexión mientras se armaba el cifrado".

Diagnóstico: el nombre `homelab-01.tail4eda13.ts.net` apunta a **dos** servidores de entrada de Tailscale Funnel (dos "porteros"). Probando cada uno por separado, uno andaba y **el otro**

cortaba siempre, con el mismo error que la placa. A la placa le había tocado el que fallaba, y lo recordaba.

Puerta de entrada	Resultado
209.177.145.97	✓ conecta
209.177.145.192	✗ corta (unexpected eof)

Solución: era una falla del servicio externo (Tailscale); al rato se normalizó. Mientras tanto, **desenchufar y enchufar** la placa la hace preguntar de nuevo y puede tocarle la puerta buena.

Lección: antes de cambiar tu código, **medí cada pieza del camino**. A veces el problema no es tuyo, y saberlo ahorra horas.

Caso 8 — `ON` y `OFF` contra `1` y `0`: el contrato sin acordar

Síntoma: el enchufe de Jorge conectaba bien, pero al mover el interruptor del dashboard **no pasaba nada**.

Diagnóstico: el dashboard mandaba `1` y `0`; el código de Jorge esperaba `ON` y `OFF`. Los dos tenían razón según su propia idea, pero **nunca habían acordado** cómo se iban a hablar.

Solución: se adaptó el dashboard a `ON / OFF` (cambiar un lado es más simple que cambiar los dos). Grafana entiende los dos formatos.

Lección: el formato de los mensajes es un **contrato** (ver [capítulo 11, decisión 2](#)). Se acuerda **antes** de programar y se escribe, para que las dos puntas lo respeten.

Caso 9 — El servidor se mudó de red y el firewall seguía mirando la vieja

Síntoma: desde la red de la casa, `ssh` al servidor quedaba colgado hasta dar `Connection timed out`. Por Tailscale, en cambio, entraba bien.

Diagnóstico: el servidor cambió de red varias veces (`192.168.100.x` → `192.168.0.x` → `192.168.8.x`, al cambiar de router o de WiFi). El **firewall** (el "patovica" que decide qué conexiones dejar entrar, en el servidor se llama UFW) solo dejaba entrar por SSH desde la red **vieja**. Para él, la red nueva era "un desconocido".

Solución: actualizar en el inventario de Ansible la red de la casa (`lan_cidr`) y volver a aplicar el firewall. Mientras tanto, se entra por Tailscale, que no depende de la red de la casa.

Lección: cuando algo **se cuelga** (no da error, solo espera) sospechá de la **red** o del **firewall** antes que de la aplicación. Y tené siempre **una segunda puerta** (acá, Tailscale) para no quedarte afuera.

Caso 10 — El servidor se congeló, y el diagnóstico automático se equivocó

Síntoma: el sábado 3 de octubre, a eso de las 12:50, el servidor dejó de responder por **todos** los caminos: Tailscale, Funnel (las placas no llegaban al broker) y la IP pública. Volvió recién a las 14:40, cuando alguien lo prendió.

Lo que dijo el informe automático (el vigía de la notebook del profe, que arma un informe solo cuando el servidor vuelve):

"Apagado ordenado: **sí**" · "El sistema tuvo un **error grave del kernel**"

Lo que mostraban las pistas de verdad: el diario del sistema terminaba así:

```
12:48:30 systemd[851038]: Reached target shutdown.target - Shutdown.
12:48:30 systemd[1]: Removed slice user-1001.slice - User Slice of UID 1001.
(... y nada más hasta las 14:40)
```

Esa línea de "Shutdown" la escribió `systemd[851038]`: el gestor de **una sesión de usuario** (la conexión SSH del profe cerrándose), no `systemd[1]`, que es el del **sistema**. El "error del kernel" eran mensajes normales de **cualquier** arranque ("NMI watchdog: Enabled", "drm panic"). La conclusión real: el diario **se corta en seco**, sin apagado y sin cortes de red registrados antes. La máquina entera **se congeló o se quedó sin luz**.

Por qué ningún programa lo arregló: en una máquina congelada no corre nada, ni siquiera un guardián. Hace falta algo **debajo** del sistema operativo.

Solución (rol `vigia_red`, ver <docs/vigia-red.md>):

1. **Watchdog de hardware:** la placa madre tiene un chip (iTCO) que reinicia el equipo si el sistema deja de "darle señal" durante 30 segundos. Ubuntu lo traía apagado; ahora está armado.
2. **Más evidencia:** el diario se escribe cada 30 s (antes, cada 5 min: en un congelamiento se perdían esos minutos), y un **latido** por minuto dice a qué hora murió la máquina.
3. **El diagnóstico corregido:** solo cuenta como apagado ordenado lo que dice `systemd[1]`, y solo cuenta errores del kernel reales.

Lección: una herramienta automática **también se equivoca**. Antes de creerle a una conclusión, mirá las **pistas crudas** que la sostienen. Y si algo puede congelarse, la protección tiene que estar **por debajo** de lo que se congela.

Caso 11 — Se reinicia, pero no vuelve a la red

Síntoma: después de un corte de luz el servidor arranca (el BIOS lo prende solo), pero **no aparece** en Tailscale: a veces vuelve a los minutos, a veces a la media hora, y una vez **tres horas y media** después.

Diagnóstico: los registros de NetworkManager (el programa que maneja la red) de los arranques lentos muestran esto:

```
14:03:10 association took too long ... ssid-not-found
14:04:37 association took too long ... ssid-not-found
        (y después... nada durante un buen rato)
```

`ssid-not-found` quiere decir "**no encuentro la red**". ¿Por qué no la encontraba, si la red existe? Porque después de un corte de luz se prenden **los dos a la vez**: el servidor y el router. El servidor está listo en un minuto; el WiFi del router tarda más. NetworkManager prueba, no ve la red, prueba de nuevo... y **después de 4 intentos fallidos se rinde por un rato** (es su configuración de fábrica). Cuando el router ya está listo, nadie está intentando.

Solución:

1. Cada red guardada **reintenta para siempre** (`autoconnect-retries 0`).
2. El guardián de red, al minuto 2 sin router, **busca redes de nuevo y conecta** a la mejor guardada.

Lección: un sistema no falla solo "por dentro": también falla por **cómo se relaciona con otros**. Acá nadie estaba roto: el problema era el **orden** en que arrancan dos aparatos. Diseñar para eso es pensar en **arquitectura**, no solo en código ([cap. 11, decisión 5](#)).

Casos a documentar (próximamente)

- Un alumno **rompe** su Compose (la política lo rechaza: cómo leer el error).
- **El disco se llena** (cuota del equipo, `labctl usage`, dashboard Capacity).
- **Prometheus deja de responder** (dónde mirar, cómo reiniciar).
- **Grafana muestra un contenedor unhealthy** (healthchecks, logs).
- **DuckDNS deja de actualizar** (timer, token, logs).
- **La máquina pierde Internet** (qué sigue andando: consola, Tailscale directo).



Ideas clave

- Diagnosticar es **aislar la capa** y **confirmar** con un comando.
- "Se cuelga" $\approx$ red/DNS; "permission denied" $\approx$ permisos; "connection refused" $\approx$ el servicio no está escuchando.

Ejercicios

1. Ante "connection refused" a un servicio, listá 3 comandos para diagnosticar.
2. Tu app no resuelve `postgres` : ¿qué revisás primero?

15. Ciclo de vida y madurez del sistema

 **Objetivo:** entender que un sistema **no se "termina"**: nace, crece, se usa, se arregla y cambia. Vas a ver **en qué etapa** está cada parte del sistema del aula, **con qué tecnologías** está hecho, y **qué tan madura** es cada una.

 **Prerequisitos:** [capítulo 1 \(Visión general\)](#) y [capítulo 11 \(Arquitectura IoT\)](#).

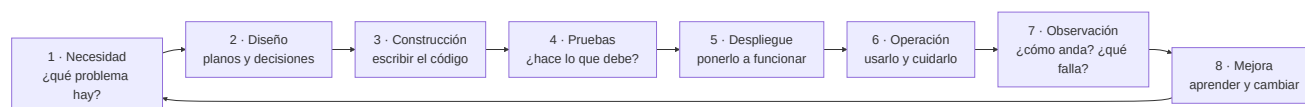
 **Conceptos nuevos:** ciclo de vida, versión, stack, madurez, deuda técnica, hoja de ruta.

Empecemos por algo cercano: una bici, o una huerta

Nadie arma una bici una vez y se olvida. La usás, se pincha una goma, le cambiás la cadena, le agregás una luz, un día le ponés cambios. Una **huerta** igual: se planifica, se siembra, se riega, se mira cómo crece, se corrige lo que sale mal y la temporada siguiente se planta mejor.

Un sistema de software vive igual. A ese recorrido se le llama **ciclo de vida**.

El ciclo de vida de un sistema



Es un **círculo**, no una línea: lo que se aprende al **operar** (etapa 6) y al **observar** (etapa 7) se convierte en una necesidad nueva (etapa 1). Ejemplos del aula:

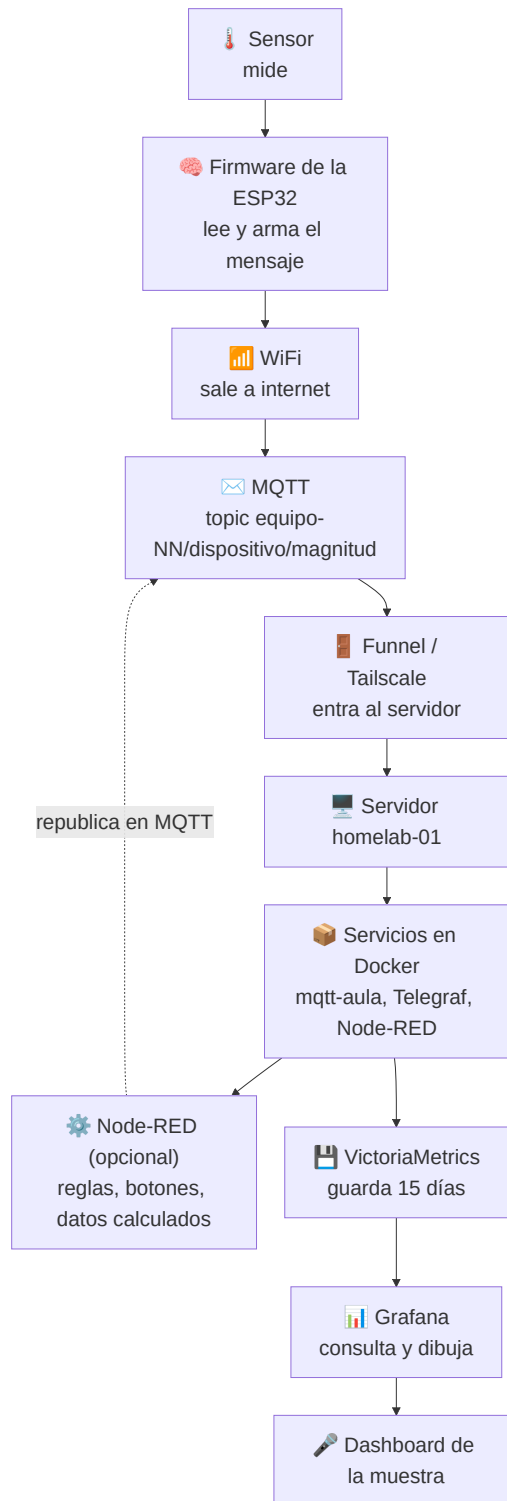
Etapa	Ejemplo real
1 · Necesidad	"Las ESP32 no pueden abrir un túnel SSH: no llegan al broker de su equipo."
2 · Diseño	Un broker compartido del aula, con un usuario por equipo y reglas (ACL), publicado por Funnel.
3 · Construcción	Rol <code>shared_services</code> + <code>mqtt-aula</code> en el repositorio.
4 · Pruebas	Tests automáticos: "el broker no acepta conexiones sin clave", "un equipo no puede escribir en otro".
5 · Despliegue	Se levantó en el servidor y se conectó la placa de Mijael.
6 · Operación	Jessi y Jorge conectan las suyas.
7 · Observación	El registro del broker muestra un <code>not authorised</code> repetido → claves mezcladas.
8 · Mejora	Se escribe el caso en el manual y se agrega Grafana para ver los datos. → vuelve a empezar

Nombre técnico: en la industria se le dice **SDLC** (*Software Development Life Cycle*, ciclo de vida del desarrollo de software). Cuando las etapas de construir y operar se juntan y se repiten muy seguido, se le dice **DevOps** (*Development + Operations*: desarrollo y operación como un solo trabajo).

El ciclo de vida de tu proyecto

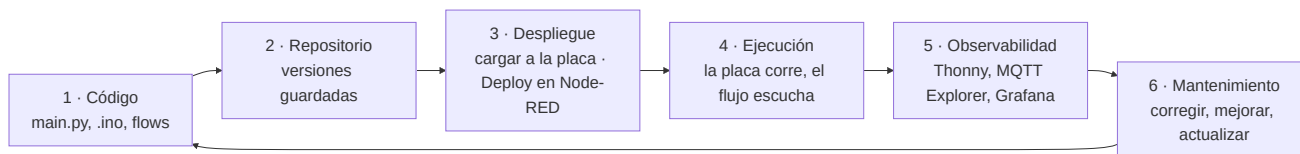
Tu proyecto también tiene dos ciclos de vida, y al terminar el año los vas a poder explicar los dos.

El ciclo de vida del dato: de lo que mide el sensor a la pantalla



Cada flecha está explicada: el recorrido completo en el [capítulo 11](#), la red en [Red y accesos](#), el contrato en el [capítulo 12](#) y la parte de Node-RED en [¿Node-RED, Grafana o ambos?](#).

El ciclo de vida del software: de tu código a algo que funciona y se cuida



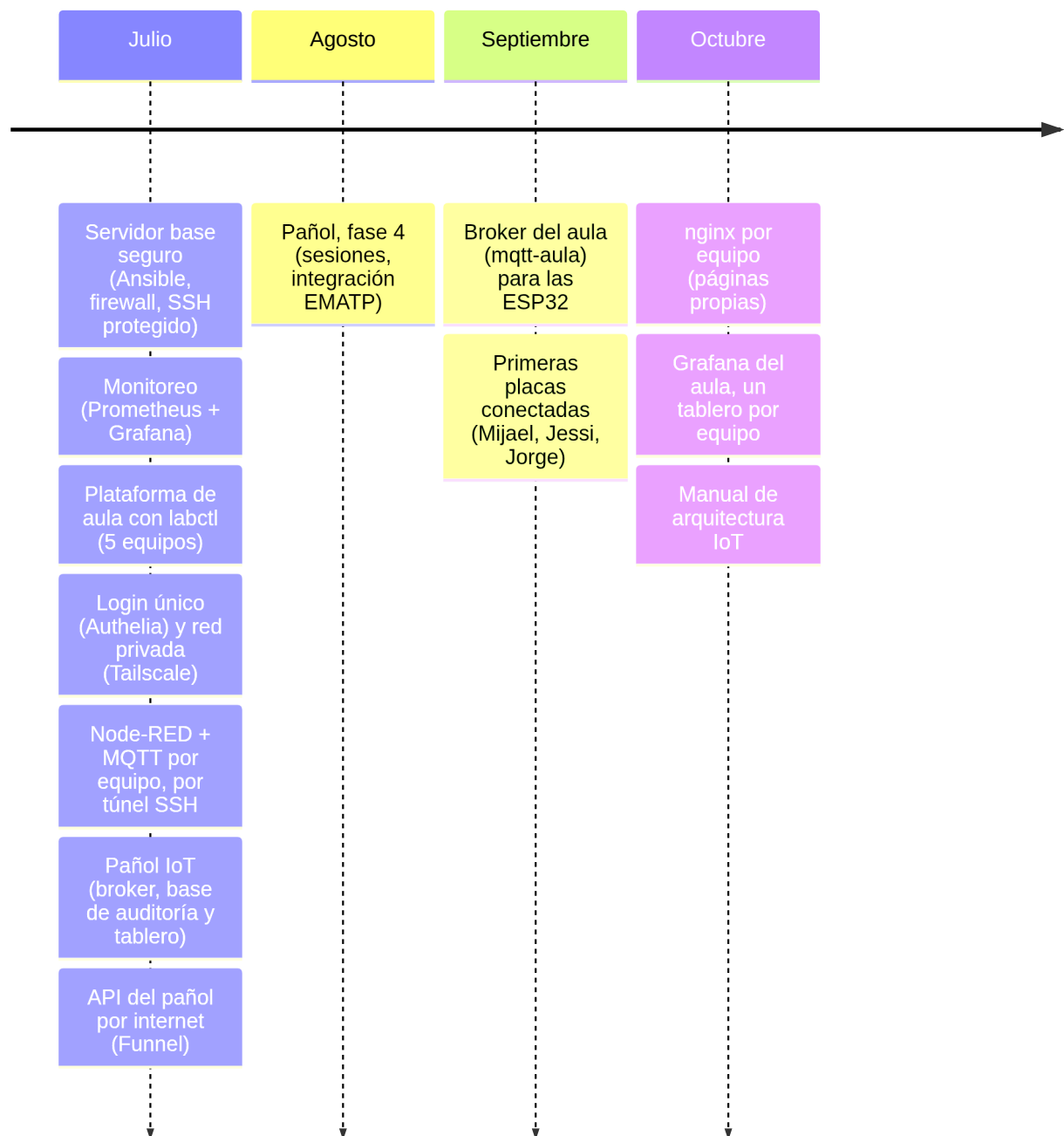
Etapa	En tu proyecto	Si la salteás...
1 · Código	el <code>main.py</code> o el <code>.ino</code> de la placa; los flujos de Node-RED; tu página	—
2 · Repositorio	guardar cada versión que anda (en git, o como mínimo copias con fecha)	el día que algo se rompe, no tenés a qué volver
3 · Despliegue	cargar el firmware a la placa (Thonny / IDE de Arduino); el botón Deploy de Node-RED; <code>labctl up</code>	"en mi compu andaba": lo que corre no es lo que escribiste
4 · Ejecución	la placa conectada y publicando; tu stack <code>Up</code>	—
5 · Observabilidad	Thonny / Monitor Serie, MQTT Explorer , tu dashboard, Grafana, <code>labctl logs</code>	te enterás de que no anda en el medio de la muestra
6 · Mantenimiento	ajustar el topic al contrato, cambiar una clave, mejorar un flujo	el proyecto envejece y deja de andar solo

El salto que queremos que des este año: dejar de pensar "**hice andar un sensor**" y poder decir "**construí un pequeño sistema IoT y entiendo el ciclo de vida del dato y del software**".

La historia de este sistema (hasta hoy)

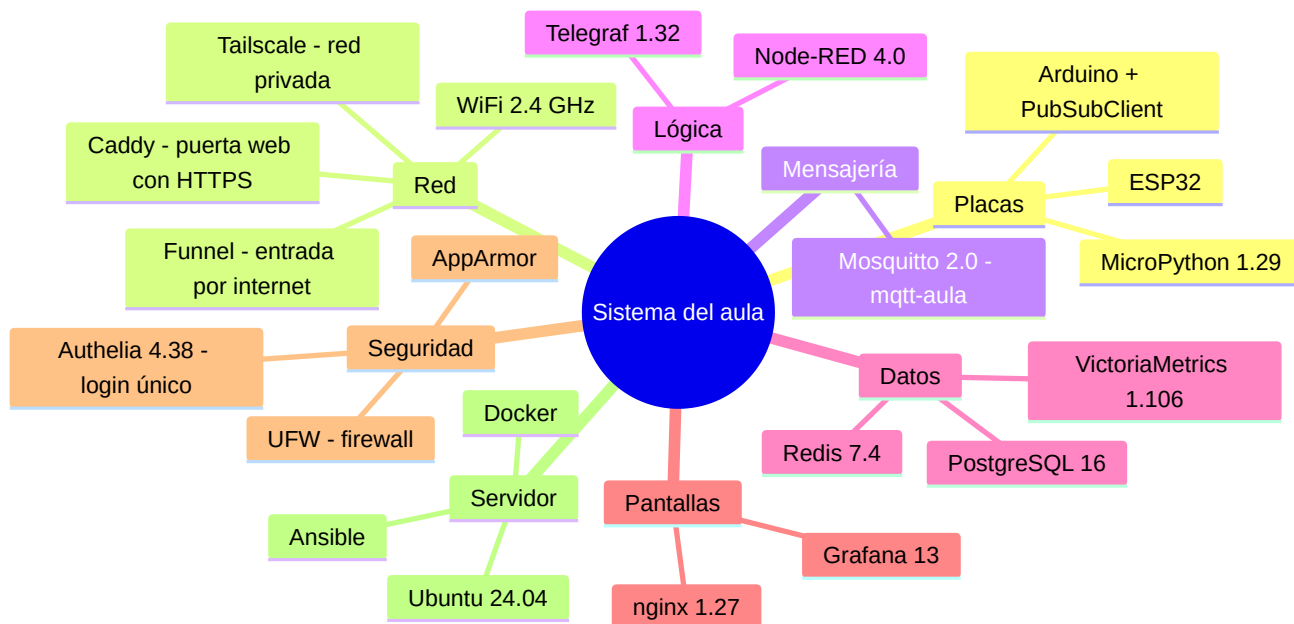
Cada punto es un cambio real, sacado del historial del repositorio (git guarda **qué** cambió, **cuándo** y **por qué**).

El sistema del aula, paso a paso (2026)



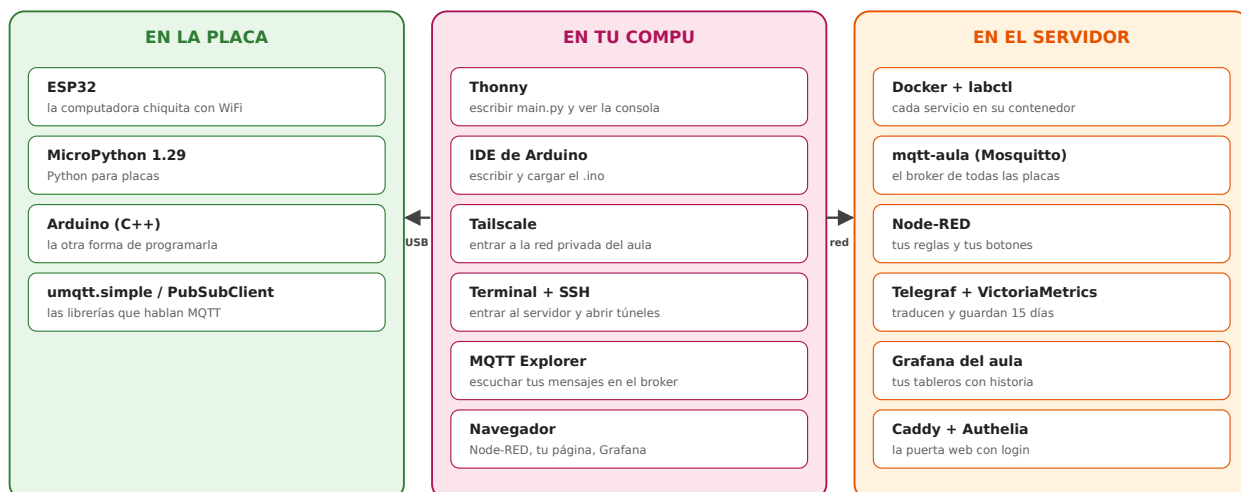
Los stacks: con qué está hecho

Un **stack** (pila) es el **conjunto de tecnologías** que usa un sistema, una arriba de la otra, como los pisos de un edificio. Este es el del aula, con la **versión** de cada pieza. La versión es el número que identifica una edición exacta, como "Android 14": la fijamos para que el sistema no cambie solo de un día para el otro.



Y vistas desde el lado de quien las usa:

Tus herramientas: qué es cada una y dónde trabaja



Capa (cap. 11)	Pieza	Versión	Para qué, en una frase
Dispositivo	ESP32	—	la placa con WiFi
	MicroPython	1.29	Python para placas (Mijael, Jessi)
	Arduino + PubSubClient	—	C++ para placas y su librería MQTT (Jorge)
Conectividad	Tailscale / Funnel	1.102	red privada / entrada desde internet
	Caddy	(compilación propia)	puerta web: pone el HTTPS y pide el login
Mensajería	Mosquitto (mqtt - aula)	2.0.20	el broker que reparte los mensajes
Procesamiento	Node-RED	4.0	lógica con cajitas, botones
	Telegraf	1.32	traduce mensajes a números y los guarda
Persistencia	VictoriaMetrics	1.106.1	historial de valores (15 días)
	PostgreSQL	16.4	base de datos de eventos y de los equipos
Aplicación	Grafana	13.0.2	tableros para ver
	nginx	1.27	sirve las páginas propias de los equipos
Transversal	Authelia	4.38	un solo login para todo
	Ubuntu Server	24.04 LTS	el sistema operativo
	Docker / Compose	—	cada servicio en su "caja" (contenedor)
	Ansible	—	describe todo el servidor en archivos

LTS (*Long Term Support*, soporte de largo plazo): una versión que recibe arreglos de seguridad durante años. Para un servidor se elige siempre una LTS.

¿Qué tan maduro está cada parte?

Madurez es qué tan **confiable y cuidada** está una parte del sistema. Usamos una escala simple, como los niveles de un videojuego:

Nivel	Nombre	Cómo se reconoce	En una bici sería...
0	Idea	está pensado, nada más	"quiero una bici"
1	Prototipo	anda en la mesa de alguien	la armé y doy una vuelta a la manzana
2	Funciona	anda en el servidor, pero se hizo a mano	la uso todos los días, la arreglo como puedo
3	Repetible	está escrito en el repositorio : se puede reconstruir igual	tengo el manual y las piezas: la armo igual otra vez
4	Confiable	además tiene pruebas, monitoreo, alertas y copias de seguridad	service, luces, candado y seguro
5	Mejora sola	se mide su uso y se mejora con esos datos	sé cuántos km hago y cambio piezas antes de que fallen

Cómo está hoy (octubre 2026)

Parte del sistema	Nivel		Qué le falta para subir
Servidor y seguridad (Ansible, firewall, SSH)	3		el firewall quedó con la red vieja; faltan alertas
Plataforma de aula (labctl , reglas de Compose)	3		pruebas sobre el servidor real, alertas de cuota
Monitoreo del servidor (Prometheus + Grafana)	3		alertas; fijar versiones (hoy algunas son latest)
Broker del aula (mqtt-aula)	3		alerta si se cae; copia de seguridad de su config
Pañol IoT	3		recuperar el acceso por la red local
Grafana del aula (este manual)	2→3		está escrito en el repositorio; falta desplegarlo y probarlo
Proyectos de los equipos (Node-RED, páginas)	2		se armaron a mano: pasarlos al repositorio
Firmware de las placas	1-2		guardarlo en git, versiones, contrato de topics
Robustez ante caídas (watchdog, guardián de red)	3		armado y probado (oct 2026); falta el cable de red, el BIOS "encender al volver la luz" y alertas
Red del servidor (WiFi USB)	2		adaptador de 15 años (RTL8187B): cable de red o un adaptador nuevo
Copias de seguridad	1		están diseñadas pero no corren : falta el disco
Documentación	2-3		capítulos 4 a 8 incompletos

El mapa: qué tan importante vs. qué tan maduro

Lo urgente es lo que está **arriba a la izquierda**: muy importante y poco maduro.

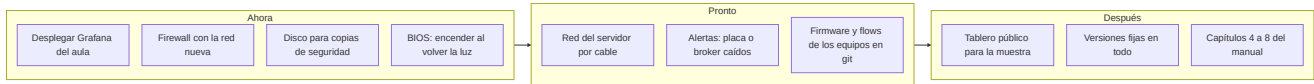
Importancia vs. madurez (octubre 2026)



Deuda técnica: cuando algo se hace "rápido y a mano" para salir del paso (como los flows de Node-RED de los equipos), queda una **deuda**: algún día hay que volver a hacerlo bien, y mientras tanto genera "intereses" (más trabajo, más riesgo). No es malo tener deuda; es malo **no anotarla**. Esta tabla es la forma de anotarla.

Hoja de ruta: cómo sube de nivel

Una **hoja de ruta** es el plan de qué se hace primero. Se ordena por el mapa de arriba: primero lo urgente.



Ideas clave

- Un sistema **no se termina**: vive un **ciclo** (necesidad → diseño → construcción → pruebas → despliegue → operación → observación → mejora).
- El **stack** es el conjunto de tecnologías; cada pieza tiene una **versión** fija para que nada cambie solo.
- **Madurez** = qué tan confiable y cuidada está una parte. Pasar de "anda" (2) a "está escrito y se reconstruye" (3) es el salto más importante.
- La **deuda técnica** no es mala si está **anotada** y tiene plan.

? Preguntas de repaso

1. ¿Por qué el ciclo de vida se dibuja como un círculo y no como una línea?
2. ¿Qué diferencia hay entre el nivel 2 y el nivel 3 de madurez?
3. ¿Por qué las copias de seguridad están en la zona "URGENTE"?
4. ¿Qué es la deuda técnica? Da un ejemplo del aula.

Ejercicios

1. Ubicá **tu proyecto** en el ciclo de vida: ¿en qué etapa está hoy?
2. Calificá tu proyecto con la escala de madurez (0 a 5) y escribí **una** cosa concreta que lo subiría un nivel.
3. Armá el **stack** de tu proyecto como la tabla de arriba: pieza, versión y para qué sirve.

Glosario

Cada entrada tiene una **definición simple**, una **técnica** y **dónde aparece** en este proyecto (en los términos centrales, **qué significa en el aula**). Ordenado alfabéticamente.

La definición **simple** está pensada para leerla primero, sin saber nada de sistemas. La **técnica** es la que vas a encontrar en internet o en un libro.

ACL (Access Control List)

- **Simple:** una lista de reglas de "quién puede hacer qué".
- **Técnica:** conjunto de reglas de autorización asociadas a un recurso.
- **Dónde:** las reglas SSH de Tailscale (modo `check` vs `accept`); las reglas de acceso por grupo de Authelia.

ACME

- **Simple:** el protocolo para pedir certificados HTTPS automáticamente.
- **Técnica:** *Automated Certificate Management Environment*; lo usan Let's Encrypt y ZeroSSL.
- **Dónde:** Caddy lo usa (con el desafío DNS-01 de DuckDNS) para emitir/renovar los certificados solo.

Actuador

- **Simple:** la parte que **hace** algo en el mundo físico cuando recibe una orden (los "músculos").
- **Técnica:** dispositivo que convierte una señal eléctrica en una acción: luz, movimiento, corte de corriente.
- **Dónde:** el LED de Mijael, el relé del enchufe de Jorge. Ver [cap. 11](#).

AIDE

- **Simple:** un "detector de cambios" en archivos del sistema.
- **Técnica:** *Advanced Intrusion Detection Environment*; guarda huellas de los archivos y avisa si cambian.
- **Dónde:** rol `audit`, corre por un timer.

Ansible

- **Simple:** una herramienta para configurar servidores escribiendo archivos.
- **Técnica:** motor de automatización/IaC sin agente, declarativo e idempotente.
- **Dónde:** todo el plano del host. Ver [cap. 4](#).

API (Application Programming Interface)

- **Simple:** la "puerta acordada" por la que un programa le pide cosas a otro.
- **Técnica:** interfaz de programación: conjunto de operaciones, formatos y reglas que un componente expone a otros.
- **Dónde:** `POST /api/led` en el Node-RED de los equipos; el contrato de topics también funciona como API.

AppArmor

- **Simple:** un "chaleco" que limita qué puede hacer cada programa.
- **Técnica:** módulo de seguridad del kernel (MAC) basado en perfiles por programa.
- **Dónde:** rol `apparmor`, en modo enforce.

Autenticación

- **Simple:** probar **quién sos** (usuario y contraseña, una clave).
- **Técnica:** verificación de identidad de un usuario o sistema.
- **En el aula:** hay cuatro puertas, cada una con su llave: Tailscale, SSH, el login web y el usuario MQTT. Ver [las 4 capas](#).

Authelia

- **Simple:** el portal de login único (SSO) del laboratorio.
- **Técnica:** servidor de autenticación/forward-auth con reglas por grupo.
- **Dónde:** stack `auth`, integrado a Caddy con `forward_auth`. Ver [cap. 7](#).

Autorización

- **Simple:** decidir **qué podés hacer** una vez que se sabe quién sos.
- **Técnica:** control de permisos sobre recursos para una identidad ya autenticada.
- **En el aula:** solo tu carpeta, solo tus topics `equipo-NN/...`, solo tu folder de Grafana. Ver [Red y accesos](#).

Backup

- **Simple:** una copia de seguridad de los datos.
- **Técnica:** respaldo versionado y cifrado (aquí, con Borg/borgmatic).
- **Dónde:** rol `backups`; requiere un disco montado en `/mnt/backup`.

Bind mount

- **Simple:** montar una carpeta del server dentro de un contenedor.
- **Técnica:** mapeo directo de una ruta del host a una ruta del contenedor.
- **Dónde:** los alumnos guardan datos así (obligatorio por la política).

Broker (MQTT)

- **Simple:** el "cartero" que recibe los mensajes y se los reparte a quien los quiere (como el servidor de WhatsApp).
- **Técnica:** servidor intermediario del modelo publicar/suscribir: recibe publicaciones y las entrega a los suscriptores de cada topic.
- **En el aula:** `mqtt-aula`. Tu ESP32 y tu Node-RED se conectan a él; él reparte cada mensaje a quien esté suscripto (Telegraf, tu Node-RED, MQTT Explorer). Ver [Seguí un dato](#).

Caddy

- **Simple:** el portero web: recibe todo y reparte, con HTTPS automático.
- **Técnica:** servidor web / proxy inverso con gestión automática de TLS.
- **Dónde:** stack `proxy`; única entrada web. Ver [cap. 5](#).

cAdvisor

- **Simple:** mide cuánto consume cada contenedor.
- **Técnica:** *Container Advisor*; exporta métricas por contenedor a Prometheus.
- **Dónde:** stack `monitoring`.

cgroup

- **Simple:** un mecanismo para limitar recursos de un grupo de procesos.
- **Técnica:** *control group* del kernel (v2); limita CPU/RAM/PIDs.
- **Dónde:** límites de compose + slices de systemd por equipo.

CI (Integración Continua)

- **Simple:** robots que revisan tu código cada vez que subís cambios.
- **Técnica:** pipeline automatizado (lint, tests) en cada push/PR.
- **Dónde:** GitHub Actions (`.github/workflows/ci.yml`). Ver [cap. 8](#).

Ciclo de vida

- **Simple:** el recorrido de un sistema: se necesita, se diseña, se construye, se usa, se mira y se mejora, una y otra vez.
- **Técnica:** SDLC (*Software Development Life Cycle*): etapas iterativas de desarrollo y operación de un sistema.
- **Dónde:** [cap. 15](#).

Commit

- **Simple:** una "foto" de los cambios de un proyecto, con un mensaje que explica por qué se hicieron.
- **Técnica:** registro atómico en la historia de Git (cambios + autor + fecha + mensaje).

- **Dónde:** cada cambio de este repositorio. Ver [cap. 8](#).

Compose (Docker Compose)

- **Simple:** describir contenedores en un archivo y prenderlos juntos.
- **Técnica:** herramienta para definir apps multi-contenedor en YAML.
- **Dónde:** todo el plano de servicios (`compose/`).

Contenedor

- **Simple:** una imagen en ejecución, aislada.
- **Técnica:** proceso(s) aislados con namespaces + cgroups.
- **En el aula:** cada servicio del servidor corre en uno: `mqtt-aula` , `grafana-aula` , y el Node-RED de tu equipo (`equipo-04-nodered-1`). Los levantas con `labctl up` . Ver [cap. 3](#).

Contrato (de mensajes)

- **Simple:** el acuerdo escrito de "cómo nos vamos a hablar": qué nombre tiene cada mensaje y qué formato lleva.
- **Técnica:** especificación de la interfaz entre productores y consumidores (estructura de topics, formato y semántica del payload).
- **Dónde:** `equipo-NN/dispositivo/magnitud` . Ver [cap. 12](#).

CPU (Central Processing Unit)

- **Simple:** el "cocinero" de la computadora: el que hace las cuentas y sigue las instrucciones.
- **Técnica:** unidad central de procesamiento; cada núcleo ejecuta instrucciones en paralelo.
- **Dónde:** el servidor tiene 4 núcleos. Ver [cap. 2](#).

Credencial

- **Simple:** lo que usás para probar quién sos: un usuario con su contraseña, una clave, un token.
- **Técnica:** dato secreto que autentica a un usuario o sistema.
- **En el aula:** tenés cuatro distintas (tailnet, aula, MQTT del equipo, WiFi). Ver [la tabla](#).

Daemon

- **Simple:** un servicio que corre de fondo, permanente.
- **Técnica:** proceso en segundo plano (suele terminar en `d`).
- **Dónde:** `dockerd` , `sshd` , `tailscaled` , `labctld` , `dnsmasq` .

Dashboard

- **Simple:** un tablero con gráficos.
- **Técnica:** conjunto de paneles de visualización (en Grafana).

- **En el aula:** hay dos tipos: el de **Node-RED** (`localhost:1880/dashboard` , para controlar en vivo) y el de **Grafana** (para ver la historia). Ver [¿Node-RED, Grafana o ambos?](#).

Datasource (fuente de datos)

- **Simple:** la base de datos a la que un panel de Grafana le hace preguntas.
- **Técnica:** conexión configurada en Grafana hacia un origen de datos.
- **En el aula:** la tuya es **MQTT — equipo-NN**: consulta VictoriaMetrics y solo ve los datos de tu equipo. Ver [cap. 13](#).

DDNS / DuckDNS

- **Simple:** mantener un dominio apuntando a tu casa aunque cambie tu IP.
- **Técnica:** DNS dinámico; DuckDNS es un proveedor gratuito.
- **Dónde:** rol `ddns` , con un timer que actualiza la IP.

Defensa en profundidad

- **Simple:** varias medidas de seguridad independientes, para que romper una no alcance (como en un boliche: puerta, patovica, pulsera, cámaras).
- **Técnica:** estrategia de seguridad en capas con controles redundantes e independientes.
- **Dónde:** las 9 capas del [cap. 7](#).

Deuda técnica

- **Simple:** lo que se hizo rápido "para salir del paso" y algún día hay que rehacer bien.
- **Técnica:** costo futuro acumulado por elegir una solución rápida en lugar de la adecuada; genera "intereses" en forma de trabajo y riesgo.
- **Dónde:** los flows de Node-RED armados a mano. Ver [cap. 15](#).

DevSecOps

- **Simple:** desarrollar, asegurar y operar, todo integrado y automatizado.
- **Técnica:** cultura/práctica que integra seguridad en el ciclo Dev+Ops.
- **Dónde:** el pipeline (CI, lint, tests, gitleaks). Ver [cap. 8](#).

DNS

- **Simple:** la guía telefónica que traduce nombres a IPs.
- **Técnica:** *Domain Name System*.
- **En el aula:** `homelab-01.tail4eda13.ts.net` (el nombre que usa tu placa) y `grafana-aula.lucasland.duckdns.org` (el que usás vos) se traducen a direcciones IP. Dentro del tailnet, el segundo apunta a la IP privada del servidor (Split DNS).

Docker

- **Simple:** empaquetar y correr apps en contenedores.

- **Técnica:** plataforma de contenedores (engine + CLI + formatos).
- **Dónde:** el corazón del plano de servicios y de aula. Ver [cap. 3](#).

Drift (deriva)

- **Simple:** cuando el servidor real ya no es lo que dicen los archivos, porque alguien cambió algo a mano.
- **Técnica:** divergencia entre el estado declarado (IaC) y el estado real de la infraestructura.
- **Dónde:** el broker del aula existió unos días solo en el servidor. Ver [cap. 4](#).

Endpoint

- **Simple:** una "puerta" con dirección a la que un programa le manda pedidos.
- **Técnica:** URL o dirección de un servicio que acepta peticiones.
- **En el aula:** `POST /api/led` en el Node-RED de los equipos con página propia; para el broker, `homelab-01.tail4eda13.ts.net:10000`.

ESP32

- **Simple:** una placa chiquita y barata, con WiFi, que se programa para leer sensores y manejar actuadores.
- **Técnica:** microcontrolador de Espressif con WiFi/Bluetooth, doble núcleo, programable en C++ (Arduino) o MicroPython.
- **Dónde:** las placas de todos los equipos.

Exporter

- **Simple:** un programita que "traduce" métricas a un formato que Prometheus entiende.
- **Técnica:** endpoint HTTP que expone métricas en formato Prometheus.
- **Dónde:** `node-exporter` (host), `cAdvisor` (contenedores).

Fail2ban

- **Simple:** el que echa por un rato a quien prueba contraseñas una y otra vez.
- **Técnica:** servicio que analiza logs y bloquea IPs con demasiados intentos fallidos.
- **Dónde:** 4 intentos en 10 min → bloqueo de 1 h. Ver [cap. 7](#).

Filesystem

- **Simple:** cómo se organizan los archivos en el disco.
- **Técnica:** estructura jerárquica desde `/`.
- **Dónde:** `/opt/homelab/stacks`, `/srv/classroom`, loopbacks de cuota.

Firmware

- **Simple:** el programa que vive **adentro** de un aparato (la placa).
- **Técnica:** software grabado en la memoria no volátil de un dispositivo embebido.

- **Dónde:** el `main.py` de MicroPython o el `.ino` de Arduino.

Flujo (flow)

- **Simple:** en Node-RED, un dibujo de cajitas conectadas que dice "cuando pasa esto, hacé aquello".
- **Técnica:** grafo de nodos que procesa mensajes en Node-RED.
- **En el aula:** el flujo del interruptor del LED: `ui-switch` → `mqtt out`. Ver [¿Node-RED, Grafana o ambos?](#).

Funnel (Tailscale Funnel)

- **Simple:** un "portero" en internet que recibe a las placas y las hace pasar al servidor sin abrir puertas del router.
- **Técnica:** servicio de Tailscale que publica un puerto local en internet (443, 8443 o 10000) mediante una conexión saliente, con TLS.
- **Dónde:** `homelab-01.tail4eda13.ts.net:10000` → `mqtt-aula`; `:8443` → API del pañol.

Git

- **Simple:** el programa que guarda la historia de un proyecto: cada cambio, quién, cuándo y por qué.
- **Técnica:** sistema de control de versiones distribuido.
- **Dónde:** todo el repositorio. Ver [cap. 8](#).

GPIO (General Purpose Input/Output)

- **Simple:** las patitas de la placa: cada una puede **leer** si hay corriente o **dar** corriente.
- **Técnica:** pines digitales de entrada/salida de propósito general de un microcontrolador.
- **Dónde:** `Pin(2, Pin.OUT)` (LED), `Pin(27, Pin.IN, Pin.PULL_UP)` (reed).

Grafana

- **Simple:** la app que muestra los gráficos de las métricas.
- **Técnica:** plataforma de visualización de series temporales.
- **Dónde:** stack `monitoring`, detrás de Authelia.

Handler (Ansible)

- **Simple:** una tarea que corre solo si otra avisó que algo cambió ("si cambió la config, reiniciá").
- **Técnica:** tarea notificada que se ejecuta al final si hubo cambios.
- **Dónde:** reiniciar Caddy si cambió el Caddyfile. Ver [cap. 4](#).

Hardening (endurecimiento)

- **Simple:** sacar o ajustar todo lo que un sistema trae "por si acaso" y no hace falta.

- **Técnica:** reducción de la superficie de ataque mediante configuración segura (p. ej., guía CIS).
- **Dónde:** rol `hardening`. Ver [cap. 7](#).

HTTPS / TLS

- **Simple:** la web con candado (cifrada).
- **Técnica:** HTTP sobre TLS; requiere un certificado válido.
- **Dónde:** lo maneja Caddy automáticamente.

IaC (Infraestructura como Código)

- **Simple:** definir la infraestructura en archivos de texto versionables.
- **Técnica:** gestionar servidores/servicios de forma declarativa y reproducible.
- **Dónde:** todo el repo (Ansible + Compose).

Idempotencia

- **Simple:** aplicar algo dos veces da el mismo resultado que una.
- **Técnica:** propiedad que garantiza convergencia sin efectos acumulativos.
- **Dónde:** todos los playbooks de Ansible.

Imagen

- **Simple:** la plantilla de una app (molde).
- **Técnica:** artefacto de solo lectura en capas.
- **Dónde:** cada `image:` de los compose.

inode

- **Simple:** la ficha interna de un archivo en el disco.
- **Técnica:** estructura del filesystem con metadatos y punteros a los datos.
- **Dónde:** el bug del Caddyfile (bind mount al inode viejo). Ver [cap. 10](#).

Inventario (Ansible)

- **Simple:** la lista de a qué servidores se aplica la configuración y con qué datos propios.
- **Técnica:** definición de hosts y variables por grupo/entorno.
- **Dónde:** `inventories/production` y `staging`. Ver [cap. 4](#).

IoT (Internet of Things)

- **Simple:** Internet de las Cosas: objetos físicos conectados a una red que miden o hacen algo.
- **Técnica:** red de dispositivos físicos con sensores/actuadores y conectividad que intercambian datos con otros sistemas.
- **Dónde:** todos los proyectos de placas del aula. Ver [cap. 11](#).

JSON

- **Simple:** una forma de escribir datos ordenados con nombre y valor, que entienden casi todos los programas: `{"temp": 22.1}`.
- **Técnica:** *JavaScript Object Notation*: formato de texto para datos estructurados (objetos, listas, números, textos).
- **Dónde:** mensajes con varios valores a la vez (cap. 12, regla 2).

labctl / labctld

- **Simple:** la única herramienta con la que los alumnos manejan sus contenedores.
- **Técnica:** cliente + daemon broker (Python) que valida y ejecuta compose.
- **Dónde:** roles `labctl`. Ver [cap. 9](#).

LAN

- **Simple:** tu red local (casa, aula).
- **Técnica:** *Local Area Network*, con IPs privadas.
- **Dónde:** `lan_cidr: 192.168.100.0/24`.

Last Will (último deseo)

- **Simple:** lo que el broker publica **por vos** si tu placa se desconecta de golpe: "avisen que me quedé sin batería".
- **Técnica:** mensaje que el cliente MQTT registra al conectarse y el broker publica si la conexión se pierde sin desconexión limpia.
- **Dónde:** `equipo-NN/<placa>/conexion = offline`. Ver [cap. 12](#).

Lint

- **Simple:** revisar el código buscando errores de estilo o cosas sospechosas, sin ejecutarlo (como sacarle la pelusa a la ropa).
- **Técnica:** análisis estático de código o configuración.
- **Dónde:** `yamllint` y `ansible-lint` en el CI. Ver [cap. 8](#).

Log (registro)

- **Simple:** el cuaderno donde un programa anota lo que va pasando.
- **Técnica:** flujo de eventos con marca de tiempo emitido por un proceso, usado para diagnóstico y auditoría.
- **Dónde:** `docker logs mqtt-aula` (quién se conectó y si lo rechazó). Ver [cap. 10](#).

Loki

- **Simple:** el archivo de logs: guarda los registros de todos los contenedores para buscarlos desde Grafana.
- **Técnica:** sistema de agregación de logs de Grafana Labs, indexado por etiquetas.

- **Dónde:** stack `logs` , prendido a pedido (`make logs-on`).

Madurez

- **Simple:** qué tan confiable y cuidada está una parte del sistema, del 0 (idea) al 5 (mejora sola).
- **Técnica:** grado de estabilidad, reproducibilidad, verificación y operación de un componente.
- **Dónde:** [cap. 15](#).

Mermaid

- **Simple:** una forma de escribir diagramas con texto.
- **Técnica:** lenguaje para diagramas embebibles en Markdown.
- **Dónde:** todos los diagramas de este manual.

Métrica

- **Simple:** un número medido en el tiempo (uso de CPU, RAM...).
- **Técnica:** serie temporal con nombre y etiquetas.
- **Dónde:** Prometheus las guarda; Grafana las muestra.

MicroPython

- **Simple:** una versión chica de Python que corre adentro de la placa.
- **Técnica:** implementación de Python 3 para microcontroladores.
- **Dónde:** las placas de Mijael y Jessi (versión 1.29).

Mínimo privilegio

- **Simple:** darle a cada persona o programa solo el permiso que necesita, y nada más.
- **Técnica:** principio de seguridad de asignar los permisos mínimos necesarios.
- **Dónde:** Telegraf solo lee; los alumnos sin sudo. Ver [cap. 7](#).

MQTT

- **Simple:** un idioma liviano para que aparatos chicos se manden mensajes cortos, tipo grupo de WhatsApp.
- **Técnica:** *Message Queuing Telemetry Transport*: protocolo publicar/suscribir sobre TCP, pensado para dispositivos con pocos recursos.
- **Dónde:** todo el aula habla MQTT con `mqtt-aula` . Ver [cap. 11](#).

MQTT Explorer

- **Simple:** un programa para tu compu que se conecta al broker y te muestra los mensajes de tus topics en vivo.
- **Técnica:** cliente MQTT de escritorio con vista en árbol de topics.

- **En el aula:** la herramienta clave para saber si tu dato llegó al servidor. Ver [Diagnóstico](#).

Namespace

- **Simple:** las "anteojeras" de un contenedor: deciden qué puede ver (sus archivos, sus procesos, su red).
- **Técnica:** mecanismo del kernel Linux que aísla la vista de recursos de un grupo de procesos.
- **Dónde:** base del aislamiento de Docker. Ver [cap. 3](#).

NAT

- **Simple:** el router comparte una IP pública entre muchos dispositivos.
- **Técnica:** *Network Address Translation*.
- **Dónde:** por eso hace falta port-forward para el acceso público.

nginx

- **Simple:** un programa que le entrega páginas web al navegador.
- **Técnica:** servidor web y proxy inverso de alto rendimiento.
- **Dónde:** el servicio `web` de equipo-03 y equipo-04 (sus páginas propias).

node-exporter

- **Simple:** mide el estado del server (CPU, RAM, disco).
- **Técnica:** exporter de métricas del host para Prometheus.
- **Dónde:** stack `monitoring`.

Node-RED

- **Simple:** una herramienta para armar lógica conectando cajitas en vez de escribir código.
- **Técnica:** entorno de programación por flujos (*flow-based*) sobre Node.js.
- **Dónde:** el Node-RED de cada equipo (interruptores, `/api/led`).

OOM (Out Of Memory)

- **Simple:** cuando se acaba la memoria y el kernel mata procesos.
- **Técnica:** *Out Of Memory killer* del kernel.
- **Dónde:** lo previenen los límites de RAM por equipo.

Payload (carga)

- **Simple:** el **contenido** de un mensaje: `23.5`, `ON`, `abierto`.
- **Técnica:** datos útiles transportados por un mensaje MQTT, sin los encabezados del protocolo.

- **En el aula:** lo que tu placa manda en cada mensaje: `24.7`, `0N`, `abierto` o un JSON. Telegraf lo convierte en número para Grafana. Ver [regla 2](#).

Playbook

- **Simple:** la "receta" de Ansible: qué configurar y en qué orden.
- **Técnica:** archivo YAML que orquesta roles/tareas sobre un inventario.
- **Dónde:** `ansible/site.yml`.

PostgreSQL

- **Simple:** una base de datos.
- **Técnica:** motor de base de datos relacional.
- **Dónde:** servicio compartido multi-tenant (una DB por equipo).

Prometheus

- **Simple:** el que junta y guarda las métricas.
- **Técnica:** base de datos de series temporales + scraper + PromQL.
- **Dónde:** stack `monitoring`. Ver [cap. 6](#).

PromQL

- **Simple:** el idioma para consultar las métricas de Prometheus.
- **Técnica:** lenguaje de consulta de series temporales.
- **Dónde:** las queries de los dashboards.

Proxy inverso

- **Simple:** un portero que recibe todo y reparte a los servicios.
- **Técnica:** proxy del lado del servidor (single entry point).
- **En el aula:** **Caddy** es la puerta web: recibe `https://grafana-aula...`, pide el login del aula y te deriva a Grafana. Tu Node-RED **no** pasa por Caddy (no tiene login): se llega por túnel. Ver [Red y accesos](#).

Publicar / suscribir

- **Simple:** mandar un mensaje a un "grupo con nombre" / anotarse para recibir lo de ese grupo.
- **Técnica:** patrón *publish/subscribe*: productores y consumidores desacoplados a través de un broker.
- **Dónde:** toda la comunicación de las placas. Ver [cap. 11](#).

Puerto

- **Simple:** el "interno" de un servicio dentro de una IP.
- **Técnica:** número (0-65535) que identifica un endpoint TCP/UDP.

- **En el aula:** tu placa usa el `10000` (el broker, por Funnel); tu Node-RED escucha en un puerto propio del equipo (`1882` , `1884` ...), y lo traés a tu compu con el túnel. Ver [guía, paso 1.2](#).

Pull request (PR)

- **Simple:** pedir formalmente "sumen mis cambios a la versión principal", para que se revisen antes.
- **Técnica:** solicitud de merge de una rama, con revisión y pruebas automáticas.
- **Dónde:** flujo de cambios del repositorio. Ver [cap. 8](#).

Pull y push (métricas)

- **Simple: pull:** alguien pasa a buscar los datos (como leer el medidor de luz). **Push:** cada uno avisa cuando pasa algo.
- **Técnica:** modelos de recolección: el colector consulta al origen (pull) o el origen envía al colector (push).
- **Dónde:** Prometheus hace pull; las placas hacen push por MQTT. Ver [cap. 6](#).

QoS (Quality of Service)

- **Simple:** cuánto esfuerzo pone MQTT en que un mensaje llegue (como el doble tilde).
- **Técnica:** nivel de garantía de entrega MQTT: 0 (como mucho una vez), 1 (al menos una vez), 2 (exactamente una vez).
- **Dónde:** Telegraf usa QoS 0; las órdenes de Node-RED, QoS 1.

RAM (Random Access Memory)

- **Simple:** la "mesada" de la computadora: donde está lo que se usa ahora. Es rápida, chica y se vacía al apagar.
- **Técnica:** memoria principal volátil de acceso aleatorio.
- **Dónde:** el servidor tiene 15 GiB. Ver [cap. 2](#).

Rama (branch)

- **Simple:** una línea de trabajo aparte, para probar cambios sin tocar lo que funciona.
- **Técnica:** puntero a una línea de commits en Git.
- **Dónde:** `feat/aula-iot` , `main` . Ver [cap. 8](#).

Relé

- **Simple:** un interruptor que se acciona con una señal eléctrica chiquita y puede prender o cortar un aparato grande.
- **Técnica:** conmutador electromecánico o de estado sólido controlado por una señal de baja potencia.
- **Dónde:** el enchufe de Jorge (simulado con un LED en GPIO 5).

Retenido (mensaje)

- **Simple:** un mensaje "fijado": el broker guarda el último y se lo da a quien llega después.
- **Técnica:** *retained message*: el broker conserva el último mensaje con la marca *retain* de cada topic y lo entrega a nuevos suscriptores.
- **Dónde:** `equipo-04/enchufe/estado` : al reconectarse, el dashboard ve el último estado.

Role (Ansible)

- **Simple:** una carpeta con todo lo necesario para configurar una cosa.
- **Técnica:** unidad reutilizable de tareas/plantillas/variables.
- **Dónde:** `ansible/roles/*`.

root

- **Simple:** el usuario que puede todo.
- **Técnica:** superusuario, UID 0.
- **Dónde:** se evita; los servicios usan cuentas sin privilegios.

Scrape

- **Simple:** cuando Prometheus pasa a "leer el medidor" de un servicio.
- **Técnica:** consulta periódica HTTP de Prometheus al endpoint `/metrics` de un target.
- **Dónde:** cada 15 s a node-exporter y cAdvisor. Ver [cap. 6](#).

Sensor

- **Simple:** la parte que **mide** algo del mundo físico (los "sentidos").
- **Técnica:** transductor que convierte una magnitud física en una señal eléctrica legible.
- **Dónde:** el reed switch de Jessi. Ver [cap. 11](#).

Serie temporal

- **Simple:** una lista de valores con su hora, como una planilla "hora | temperatura".
- **Técnica:** secuencia de muestras (timestamp, valor) identificada por un nombre y etiquetas.
- **Dónde:** `mqtt_valor{equipo, dispositivo, magnitud}` en VictoriaMetrics.

Servicio

- **Simple:** un programa que está siempre corriendo, esperando pedidos (el "cocinero de guardia").
- **Técnica:** proceso de larga duración que ofrece una función a otros.
- **En el aula:** el broker, Grafana, tu Node-RED: cada uno es un servicio en su contenedor. Ver [cap. 5](#).

Shift-left

- **Simple:** buscar los errores lo antes posible, cuando arreglarlos es más barato.
- **Técnica:** práctica de mover las pruebas y controles a etapas tempranas del ciclo de desarrollo.
- **Dónde:** tests en la compu y en el CI antes del servidor. Ver [cap. 8](#).

Socket

- **Simple:** un "enchufe" para que dos programas se hablen; en la misma máquina es un archivo especial.
- **Técnica:** extremo de comunicación entre procesos (Unix domain socket o de red).
- **Dónde:** `/run/labctld.sock` entre `labctl` y `labctld`. Ver [cap. 9](#).

Split DNS

- **Simple:** el mismo nombre responde una dirección distinta según desde dónde preguntes.
- **Técnica:** resolución DNS condicionada a la red de origen.
- **Dónde:** `*.lucasland.duckdns.org` → IP de Tailscale dentro del tailnet. Ver [cap. 5](#).

SSH

- **Simple:** entrar a la terminal de otra máquina de forma segura.
- **Técnica:** *Secure Shell*; acceso remoto cifrado, normalmente con claves.
- **Dónde:** administración del server (rol `users_ssh`).

SSO (Single Sign-On)

- **Simple:** loguearte una sola vez y entrar a todo lo que tenés permitido.
- **Técnica:** inicio de sesión único federado entre aplicaciones.
- **Dónde:** Authelia. Ver [cap. 7](#).

Stack

- **Simple:** el conjunto de tecnologías con que está hecho un sistema, una arriba de otra.
- **Técnica:** pila tecnológica: lenguajes, servicios, bases de datos y plataformas que componen una solución.
- **Dónde:** [cap. 15](#).

Staging

- **Simple:** un servidor de ensayo, para probar antes del "estreno" en el servidor real.
- **Técnica:** entorno de preproducción que replica producción.
- **Dónde:** `inventories/staging`. Ver [cap. 8](#).

Superficie de ataque

- **Simple:** todo lo que alguien de afuera podría intentar atacar.
- **Técnica:** conjunto de puntos de entrada expuestos de un sistema.
- **Dónde:** se achica sin SSH público (solo Tailscale). Ver [cap. 7](#).

systemd

- **Simple:** el que prende y cuida los servicios del sistema.
- **Técnica:** init/manager de servicios (PID 1).
- **Dónde:** timers, slices, `labctld`, `dnsmasq`.

Tailscale / Tailnet

- **Simple:** tu red privada propia para llegar al server desde cualquier lado.
- **Técnica:** red mesh basada en WireGuard; el *tailnet* es tu red.
- **En el aula:** la red privada del aula: el servidor (`100.110.123.76`) y las compus de alumnos y profe. Las ESP32 **no** están (usan Funnel). Ver [Red y accesos](#).

TCP

- **Simple:** enviar datos por red de forma confiable.
- **Técnica:** protocolo de transporte con control de entrega/orden.
- **Dónde:** web, SSH, bases de datos.

Telegraf

- **Simple:** el "traductor y escribano": lee los mensajes, los pasa a números y los anota con la hora.
- **Técnica:** agente de recolección de métricas (InfluxData) con entradas, procesadores y salidas configurables.
- **Dónde:** `aula-telegraf`: de `mqtt-aula` a VictoriaMetrics.

Template (Ansible)

- **Simple:** un archivo de configuración con "huecos" que se completan solos.
- **Técnica:** plantilla Jinja2 (`.j2`) renderizada con variables.
- **Dónde:** `telegraf.conf.j2` arma un topic por equipo. Ver [cap. 4](#).

Test (prueba)

- **Simple:** un programita que comprueba que otro hace lo que debe.
- **Técnica:** verificación automatizada del comportamiento de un componente.
- **Dónde:** `tests/`, corridos por el CI. Ver [cap. 8](#).

Topic

- **Simple:** el "nombre del grupo" al que se manda un mensaje MQTT. Se escribe como una ruta: `equipo-04/enchufe/estado`.
- **Técnica:** cadena jerárquica separada por `/` que identifica el canal de un mensaje MQTT; admite comodines `+` y `#` al suscribirse.
- **En el aula:** la "dirección" de cada mensaje, con el contrato `equipo-NN/dispositivo/magnitud` (ej. `equipo-04/enchufe/estado`). El broker solo te deja usar los que empiezan con tu equipo. Ver [las 3 reglas](#).

Túnel

- **Simple:** un pasillo privado y cifrado entre tu compu y el servidor: lo que entra por una punta sale por la otra.
- **Técnica:** canal que encapsula tráfico dentro de otra conexión (p. ej. reenvío de puertos por SSH).
- **En el aula:** `ssh -L 1880:localhost:1884 ...` trae tu Node-RED a `localhost:1880`. Ver [Red y accesos](#).

UFW

- **Simple:** el firewall (decide qué puertos se abren).
- **Técnica:** *Uncomplicated Firewall*, front-end de iptables.
- **Dónde:** rol `firewall` (default deny).

Vault (Ansible Vault)

- **Simple:** una caja fuerte para las contraseñas dentro del repo.
- **Técnica:** archivo cifrado que Ansible descifra al aplicar.
- **Dónde:** `inventories/*/group_vars/all/vault.yml`.

VictoriaMetrics

- **Simple:** la base de datos que guarda el historial de los sensores (15 días).
- **Técnica:** base de datos de series temporales compatible con Prometheus.
- **Dónde:** `aula-victoriametrics`, consultada por el Grafana del aula.

VPN (Virtual Private Network)

- **Simple:** una red privada que viaja por adentro de internet, cifrada: como estar en la misma sala aunque estés lejos.
- **Técnica:** red privada virtual: enlaza dispositivos remotos en una red lógica cifrada.
- **En el aula:** Tailscale es la VPN del aula. Ver [Red y accesos](#).

WebSocket

- **Simple:** una conexión que queda **abierta** para que los datos lleguen solos, como una llamada en vez de cartas.
- **Técnica:** protocolo de comunicación bidireccional y persistente sobre una conexión HTTP.
- **Dónde:** `/ws/led` : la página de Mijael ve el LED en vivo.

WireGuard

- **Simple:** el sistema de túneles cifrados que usa Tailscale por adentro.
- **Técnica:** protocolo VPN moderno basado en criptografía de curva elíptica.
- **Dónde:** debajo de Tailscale.

YAML

- **Simple:** un formato de texto para configuraciones, sensible a la sangría.
- **Técnica:** lenguaje de serialización legible por humanos.
- **Dónde:** compose, playbooks, inventarios, dashboards.

¿Falta un término? Está anotado en [Mejoras futuras](#) para agregarlo. Este glosario crece con el manual.

Créditos y autoría

Autor

Lucas D. Gómez — arquitecto de software y docente del aula. Contacto: resourceldg@gmail.com

Diseño de la arquitectura (servidor, plataforma de aula, plano IoT, pañol), la infraestructura como código de este repositorio y este manual.

Los equipos y sus proyectos

Los proyectos que aparecen en el manual son de sus autores. Se cuentan con sus errores reales, porque de ahí se aprende.

Equipo	Integrantes	Proyecto en el manual
equipo-01	Jessi, Dardo	Sensor de puerta con <i>reed switch</i> (cap. 14)
equipo-02	Alan, Gabi	—
equipo-03	Santi, Mijael, Gino	LED controlado desde una página propia (cap. 14)
equipo-04	Mariano, Jorge	Enchufe inteligente con firmware modular (cap. 14)
equipo-05	Guido	—

Cómo se escribió

El manual y el código se escribieron con asistencia de **Claude** (un asistente de inteligencia artificial de Anthropic), bajo la dirección y revisión del autor. Las decisiones de arquitectura, los criterios y la responsabilidad son del autor.

Tecnologías

Este sistema se apoya en software libre: Ubuntu, Docker, Ansible, Mosquitto, Node-RED, Grafana, Prometheus, VictoriaMetrics, Telegraf, PostgreSQL, Caddy, Authelia, MicroPython y el ecosistema Arduino, entre otros. Cada uno tiene sus propios autores y licencias.

Mejoras futuras del manual

Registro honesto de lo que **falta** o se puede **mejorar** en esta documentación. Mantenerlo al día evita "documentación fantasma".

Hecho en la revisión de octubre 2026

- Parte III (IoT aplicado): capítulos 11 a 14, y capítulo 15 (ciclo de vida y madurez).
- Capítulo 10: seis casos reales de las placas y la red.
- Glosario: vocabulario IoT, MQTT y ciclo de vida.
- Guía del equipo: puertos por equipo, Tailscale, broker del aula.
- Créditos y autoría.

Hecho en la segunda tanda (octubre 2026)

- Capítulos 4 a 8 completos (Ansible, servicios, observabilidad, seguridad, pipeline).
- Pasada de lenguaje en los capítulos 2, 3 y 9: cada sigla explicada, partiendo de lo cercano.
- Documentos de operación traducidos al español y actualizados (`README` , `architecture` , `deployment-guide` , `runbook`).
- El CI ahora corre también los tests de `shared-data` y `aula-iot` .

Hecho en la tercera tanda: navegación para alumnos (octubre 2026)

- Rutas de lectura por necesidad en el inicio (muestra, arquitectura, visualizar, procesar, acceso, diagnóstico).
- Páginas nuevas: **Red y accesos** (Tailscale, Funnel, túneles, las 4 capas de autenticación), **¿Node-RED, Grafana o ambos?** y **Diagnóstico** (árbol paso a paso, también para la muestra).
- "Seguí un dato: 24,7 °C" en el cap. 11: recorrido real, quién conoce a quién y qué pasa si se cae cada pieza.
- El ciclo de vida del dato y del software del proyecto del alumno (cap. 15).
- Bloques de "Ahora deberías entender / Seguí por acá" en los capítulos clave.
- Glosario con definiciones "en el aula" (118 términos).

Hecho: imágenes y PDF (octubre 2026)

- Imágenes propias, generadas con código (`img/generar_imagenes.py`): el ciclo completo ida y vuelta con sus protocolos, la pila de protocolos, los dos caminos (Tailscale y Funnel), las herramientas y el árbol de diagnóstico.
- Fundamentos (cap. 2): protocolos y pila de protocolos; VPN y Tailscale.

- Manual en PDF ordenado desde Fundamentos (`scripts/manual-pdf.py`).

Pendiente

- **Índice del PDF con número de página** (hoy tiene enlaces, sin números).
- **Cuando se active el Grafana del aula:** sacar los avisos "en puesta en marcha" (caps. 9, 12, 13, guía, diagnóstico paso 8, node-red-o-grafana).
- **Salida pública para la muestra** (que un visitante sin Tailscale vea un tablero): hoy no existe; hay que decidirla.
- **Anclas en GitHub:** los enlaces a secciones (`#...`) del manual siguen el formato de MkDocs (sin acentos). En el sitio funcionan; leyendo el `.md` directo en GitHub, los de títulos con acentos o rayas no saltan a la sección.
- **Capítulo 10:** sumar los casos listados (disco lleno, Prometheus caído, contenedor `unhealthy` , DuckDNS, servidor sin internet) a medida que pasen.

Diagramas por agregar

- Flujo de despliegue de Ansible (paso a paso).
- Flujo de métricas (exporter → Prometheus → Grafana) en detalle.
- Flujo de backups (Borg/borgmatic).
- Flujo completo de un login SSO (navegador → Caddy → Authelia → servicio).

Glosario

- Revisar que **todo** término en negrita del libro tenga entrada.
- Agregar: namespace, WireGuard, DERP, forward-auth, argon2/sha512crypt, loopback, RSS/RES (memoria), recording rule, alerta.

Posibles mejoras arquitectónicas (marcadas en el libro)

- Rotación de los secretos de Authelia/SSO (se generaron en sesión).
- Alertas en Grafana/Prometheus (hoy hay dashboards, no alertas). Primera: "la placa del equipo X no publica hace 15 minutos".
- Tablero público para la muestra (salida filtrada, ver `docs/aula-iot.md`).
- Cuota de disco por-contenedor además de por-equipo.
- SMTP real para el reset de contraseñas de Authelia (hoy es file notifier).
- Tests de exposición de puertos más exhaustivos.

Formato / publicación

- Verificar el build con `mkdocs build --strict`.
- Exportar a PDF (por ejemplo con el plugin `mkdocs-with-pdf`).

- Portada, licencia y numeración de figuras para la versión libro.

¿Encontraste algo mal explicado o faltante? Anotalo acá con una línea; es parte del mantenimiento del manual.